

Vietnam National University Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



GRADUATION THESIS
REPORT

**ENCRYPTED PEER-TO-PEER
MESSAGING APP OVER
BLUETOOTH MESH NETWORKS**

Major: Computer Science

THESIS COMMITTEE: HCMUT Computer Science Council

SUPERVISOR: Dr. Trương Tuấn Anh

B.Eng. Dương Huỳnh Anh Đức

—o0o—

STUDENT 1: Trần Trung Vĩnh (2252914)

STUDENT 2: Nguyễn Trần Huy Việt (2252906)

STUDENT 3: Nguyễn Anh Quân (2252678)

Ho Chi Minh City, May 20, 2026

PROTESTATION

The group hereby certifies that this specialized project is the result of our own research and work, conducted under the supervision of Dr. Trương Tuấn Anh and Mr. Dương Huỳnh Anh Đức. All research content and results presented in this project are truthful, objective, and have not been previously submitted to any educational institution for the purpose of obtaining an academic degree. All referenced materials are fully and properly cited, including the authors' names, titles of the works, dates, and places of publication. The group also certifies compliance with all regulations regarding the protection of intellectual property rights. Should any violations be discovered, the group accepts full responsibility before the Faculty Board and the Board of Rectors of Ho Chi Minh City University of Technology for the content of this declaration.

Authors

Trần Trung Vĩnh

Nguyễn Trần Huy Việt

Nguyễn Anh Quân

ACKNOWLEDGEMENTS

During our study and research at Ho Chi Minh City University of Technology, Vietnam National University Ho Chi Minh City, we received valuable guidance and support from the faculty members. This specialized project reflects our efforts to apply the knowledge acquired to practical problems in the field of information technology.

We would like to express our sincere gratitude to Dr. Trương Tuấn Anh for his dedicated supervision, insightful guidance, and constructive feedback throughout the project. We also thank Mr. Dương Huỳnh Anh Đức for his support and valuable practical contributions.

Our appreciation is extended to all lecturers of the Faculty of Computer Science and Engineering for their instruction and knowledge shared during our academic journey. We are also grateful to our families and friends for their encouragement and support.

Despite our best efforts, this project may contain certain limitations. We sincerely welcome constructive comments and suggestions for further improvement.

Authors

Trần Trung Vĩnh

Nguyễn Trần Huy Việt

Nguyễn Anh Quân

Abstract

This project presents the design and development of a decentralized messaging and file-sharing application using a Bluetooth mesh network. The system enables communication in environments where conventional internet or cellular networks are unavailable, such as during natural disasters, protests, large public gatherings, or remote outdoor activities. The application supports peer-to-peer encrypted messaging and secure file transfers, ensuring user privacy and data protection. To enhance reliability and scalability, it utilizes multiple data transmission techniques—including relay, hopping, and flooding—allowing messages to propagate efficiently across devices within the mesh network. This solution demonstrates how local wireless connectivity can be leveraged to create a resilient, infrastructure-independent communication platform for emergency and off-grid scenarios.

Contents

1	Introduction	14
1.1	Motivation	14
1.2	Objectives	14
1.3	Scope	15
1.4	Significance of the Project	16
1.4.1	Significance from the Practical Perspectives	16
1.4.2	Significance from the Scientific Perspectives	16
1.5	Report Structure	17
2	Background Knowledge	18
2.1	Decentralized Communication Systems	18
2.1.1	Peer-to-Peer Networks	18
2.1.2	Comparison with Centralized Client–Server Architecture	18
2.2	Bluetooth Low Energy	19
2.2.1	Bluetooth Low Energy Communication	19
2.2.2	Bluetooth Low Energy Role	20
2.2.3	Bluetooth Low Energy Service and Characteristics	20
2.2.4	Bluetooth Low Energy Limitation	21
2.3	Mesh Topology	21
2.4	Cryptography and Security	22
2.4.1	End-to-End Encryption	22
2.4.2	Hash Functions	22
2.4.3	Symmetric-Key Cryptography	23
2.4.4	Asymmetric-Key Cryptography	23
2.4.5	Key Exchange Mechanisms	23
3	Related Work	24

3.1	Bitchat	24
3.2	Briar	25
3.3	Peer-to-Peer Networks for Content Sharing	26
3.3.1	Gnutella v0.4: Early Development	26
3.4	Research Summary	27
3.4.1	Overall Insights	27
3.4.2	Opportunities and Challenges	27
4	Requirements Elicitation	29
4.1	Functional Requirements	29
4.2	Non-Functional Requirements	29
5	System Analysis	31
5.1	Use Case Diagrams	31
5.1.1	Whole System	31
5.1.2	Bootstrapping	32
5.1.3	Open Single Communication Channel	33
5.1.4	Process Message	34
5.1.5	Send Message	35
5.1.6	Fragmentation	36
5.1.7	Form Packet	37
5.1.8	Reassembly	37
5.1.9	Mesh Maintainer	38
5.2	Use Case Description	38
5.2.1	Use Case 1: Bootstrapping	38
5.2.2	Use Case 2: Open single communication channel	40
5.2.3	Use Case 3: Process Message	42
5.2.4	Use Case 4: Send Message	44
5.2.5	Use Case 5: Fragmentation	45
5.2.6	Use Case 6: Form Packet	45
5.2.7	Use Case 7: Reassembly	46
5.2.8	Use Case 8: Mesh Maintainer	47

5.3	Activity Diagrams	51
5.4	Sequence Diagrams	59
6	System Design	66
6.1	System Architecture	66
6.1.1	Comparison of Architecture Styles and Architecture Decision	66
6.1.2	Dual-Write Architecture for Bidirectional Communication	67
6.1.3	Overall Architecture	68
6.2	Component Diagram and Class Diagram	71
6.2.1	Component Diagram	71
6.2.2	Class Diagram	74
6.3	Database Design	78
6.4	User Interface Design	81
6.4.1	UI/UX Flow	81
6.4.2	Onboarding Screens	82
6.4.3	User Profile	83
6.4.4	Chat Box Screens	84
6.4.5	Add a User	84
6.4.6	Send Messages	87
7	System Implementation	88
7.1	Technology Stack	88
7.1.1	React Native	88
7.1.2	Expo	89
7.1.3	Kotlin	89
7.1.4	Android Studio	90
7.2	Mobile Application	91
7.2.1	Onboarding Screens	91
7.2.2	User Profile	91
7.2.3	Global Chat	92
7.2.4	Device Scanning	93
7.2.5	QR Scanning	94

7.3	Feature Implementation	97
7.3.1	Secure Channel Establishment	97
7.3.2	Identity Management	98
7.3.3	Routing Protocol	100
7.3.4	Mesh Maintainer	102
7.3.5	Payload Structure and Protocol	103
8	Testing and Deployment	106
8.1	Unit Testing	106
8.1.1	Overview	106
8.1.2	Testing Approach	107
8.1.3	Unit Testing Test Case Summary	109
8.1.4	Unit Testing Results Summary	111
8.2	Integration Testing	112
8.3	Use Case Testing	113
8.3.1	Use Case 1: Initial Application Launch	113
8.3.2	Use Case 2: Bluetooth Availability Check	115
8.3.3	Use Case 3: Global Chat	117
8.3.4	Use Case 4: Bootstrap + 1:1 Chat	118
8.3.5	Use Case 5: QR Scanning	120
8.3.6	Use Case 6: Change Profile	123
8.3.7	Use Case 7: Test Input Payload	126
8.3.8	Use Case 8: Anti-entropy	126
8.4	Non-Functional Testing	128
8.4.1	P1: Peer Discovery Validation (10-Second Threshold)	128
8.4.2	P2: Message Delivery Latency Validation (1-Second Ceiling)	129
8.5	Deployment	129
9	Conclusion and Next Phase	131
9.1	Key Deliverables Achieved	131
9.2	Limitations and Constraints	132
9.3	Future Work	132



A	Noise Protocol Framework	134
A.1	Handshake Patterns Used in the Application	134
B	Bloom Filter	136
B.1	Bloom Filter Overview	136
B.2	Types of Bloom Filters	137
B.2.1	Standard Bloom Filter	137
B.2.2	Kirsch-Mitzenmacher Bloom Filter	137
B.2.3	Compact Bloom Filter	138
B.2.4	Yes No Bloom Filter	138
B.2.5	Project decision	138
	References	139

List of Figures

1	Decentralized Topology	22
2	Usecase diagram: Whole system	31
3	Usecase diagram: Bootstrapping	32
4	Usecase diagram: Open single communication channel	33
5	Usecase diagram: Process message	34
6	Usecase diagram: Send message	35
7	Usecase diagram: Fragmentation	36
8	Usecase diagram: Form packet	37
9	Activity diagram: Bootstrapping	51
10	Activity diagram: Open communication channel	52
11	Activity diagram: Process message	53
12	Activity diagram: Send message	54
13	Activity diagram: Fragmentation	55
14	Activity diagram: Form packet	56
15	Activity diagram: Reassembly	57
16	Activity diagram: Mesh maintainer	58
17	Sequence diagram: Bootstrapping	59
18	Sequence diagram: Open communication channel	60
19	Sequence diagram: Process message	61
20	Sequence diagram: Send message	62
21	Sequence diagram: Fragmentation message	62
22	Sequence diagram: Form packet	63
23	Sequence diagram: Reassembly	64
24	Sequence diagram: Mesh maintainer	65
25	Dual-write architecture for bidirectional BLE communication between two peers	68
26	Applications overall architecture	70

27	Session layer components for global chat and two-party sessions, including handshake and cipher state management	71
28	Epidemic routing component diagram showing packet exchange via summary vectors and duplicate suppression via packet caching	72
29	Transport layer components for peer discovery, connection setup, and relay data exchange over BLE	73
30	System class diagram highlighting the relationships between core controllers and data entities	74
31	Session layer class diagram	75
32	Network Layer Class Diagram for Epidemic Routing	76
33	Transport Layer Class Diagram	77
34	Enhanced Entity-Relationship Diagram	78
35	Relational Entity Diagram	79
36	Meshenger UI/UX Flow	81
37	Four onboarding screens of the chat app	82
38	Edit Profile Screen	83
39	All, Individual, Group and Emergency Chat Box Screens (from left to right)	84
40	Add a User Popup	85
41	Device Scanning screen	86
42	QR Scanning Screen	86
43	Messaging Screen	87
44	React Native	88
45	Expo	89
46	Kotlin	90
47	Android Studio	90
48	Onboarding Screens	91
49	Set Profile Section	92
50	Global Chat Security Level and Chat Screen	92
51	Device Scanning Screen	93
52	Device Scanning Process	94
53	Direct QR Scanning Process	95
54	Image Picker Screen	96

55	My QR Screen	97
56	Anti-entropy synchronization between two peers using a summary vector request and response	101
57	Payload Format	104
58	Username Validation	115
59	Bluetooth Availability Check	116
60	Global Chat	117
61	Bootstrapping in Device C	118
62	Username Validation	119
63	Messaging screen after QR scanning	121
64	Error message in QR scanning	123
65	Meshenger More Screen	124
66	User Profile Information Changing Process	125
67	Maximum Payload Test	126
68	Device A and B Anti-entropy Test	127
69	Device A and B Anti-entropy Test	127
70	Bloom Filter Insert and Queries Operations	137

List of Tables

1	Usecase table: Bootstrapping	40
2	Usecase table: Open single communication channel	42
3	Usecase table: Process message	44
4	Usecase table: Send message	45
5	Usecase table: Fragmentation	45
6	Usecase table: Form packet	46
7	Usecase table: Reassembly	47
8	Usecase table: Mesh maintainer	50
10	Comparison of Monolithic Architecture Styles	66
11	Test Case Summary Matrix for Unit Testing	109
12	Unit Testing Results Summary	112
13	Integration Testing Results Summary	113
14	Initial Application Launch Testing	114
15	Reject Permissions Testing	114
16	Missing Username Testing	114
17	Use Case 2 Main Flow Testing	116
18	Use Case 2 Alternative Flow Testing	117
19	Global Chat Testing Between Devices A, B, and C	118
20	Bootstrapping and 1:1 Chat Testing Between Devices B and C	120
21	Direct QR Scanning Main Flow Testing	121
22	Image Picker QR Scanning Alternative Flow Testing	122
23	QR Scanning Exception Flow Testing	122
24	Change Profile Main Flow Testing	125



Listings

1	Android Logcat: Mesh Linkage Discovery Lifecycle	128
2	Android Logcat: Device C Outbound Message Transmission Trace	129
3	Android Logcat: Device B Inbound Message Extraction Trace	129

List of Algorithms

1	Mesh Maintenance Loop	103
---	---------------------------------	-----

1 Introduction

1.1 Motivation

Communication has always been a fundamental human need. The desire to exchange information quickly, spontaneously, and across any distance has driven innovation for centuries, from carrier pigeons and handwritten letters to the invention of the telephone, and later, computers that revolutionized global connectivity. Today, modern technology and infrastructure allow us to communicate with anyone around the world in a matter of seconds.

However, there are situations where communication becomes critical but traditional infrastructure is unavailable or unreliable, such as natural disasters where internet and phone lines are destroyed or disrupted by floods, earthquakes, or storms; protests in which governments may deliberately restrict or shut down communication networks to limit coordination; large gatherings such as concerts or festivals where overloaded networks struggle to handle the surge of users; and remote activities like hiking or picnicking in isolated areas without mobile coverage.

In these scenarios, when only smartphones are available and no centralized service exists to connect them, alternative communication methods are essential. Fortunately, modern smartphones support technologies that enable direct device-to-device connections. One such technology is Bluetooth Low Energy (BLE), which provides a foundation for resilient peer-to-peer communication.

This project, *Encrypted Peer-to-Peer Messaging App over Bluetooth Mesh Networks*, is motivated by the need for a secure, privacy-preserving, and infrastructure-independent communication system. By leveraging BLE and integrating end-to-end encryption, the proposed application enables users to communicate directly, reliably, and privately even in the absence of mobile networks or Wi-Fi. The system is designed to ensure resilience in challenging environments, empowering individuals to stay connected when traditional infrastructure fails.

1.2 Objectives

The primary objectives of this project are as follows:

- **To design and develop** a decentralized peer-to-peer messaging application using

Bluetooth mesh communication.

- **To ensure security and privacy** through end-to-end encryption, authentication, and integrity protection.
- **To support offline communication** in environments where internet or cellular networks are unavailable or unreliable.
- **To optimize performance and battery efficiency** by using Bluetooth Low Energy (BLE) for background operation.
- **To provide a user-friendly interface** that allows non-technical users to communicate easily without complex setup.
- **To evaluate system scalability and reliability** across different real-world use cases such as disaster zones, remote communities, and crowded public events.

1.3 Scope

The scope of this project includes the design, implementation, and testing of a Bluetooth mesh-based communication system that enables short- to medium-range peer-to-peer messaging. The project focuses on local device-to-device networking without reliance on internet or mobile infrastructure.

Key features within the project scope include:

- Bluetooth mesh message routing, node discovery, and relay functionality.
- End-to-end encryption for all transmitted data.
- Text and small media (e.g., image) message exchange.
- Android-based implementation using Kotlin and BLE APIs.

Out of scope for this project:

- Integration with cloud or server-based systems.
- Long-range or cross-network message relaying via the internet.
- Support for iOS or other operating systems in the current version, which is reserved for future expansion.

The project will thus deliver a fully functional Android prototype demonstrating secure, decentralized, and offline communication over Bluetooth mesh networks.

1.4 Significance of the Project

1.4.1 Significance from the Practical Perspectives

From a practical standpoint, the success of this project will deliver:

- **Alternative communication solutions:** offering users in challenging environments such as disaster zones, remote areas, or crowded events a reliable means of communication when conventional infrastructure is unavailable.
- **A secure messaging platform:** through end-to-end encryption, authentication, integrity protection, and identity management, users can be assured that their conversations remain private, trustworthy, and verifiable.
- **Scalable connectivity:** the system is designed to expand seamlessly as the number of users grows. Increased participation will not compromise reliability, ensuring that messages are consistently delivered without loss or delay.
- **Cost-effective deployment:** by leveraging existing smartphones, the solution reduces reliance on expensive satellite phones or specialized equipment, making secure communication more accessible.

1.4.2 Significance from the Scientific Perspectives

From a scientific perspective, the success of this project will contribute to:

- **Exploring alternative communication paradigms:** while current systems depend heavily on the internet, radio signals, phone lines, cellular networks, and satellite communication, this project demonstrates the feasibility of infrastructure-independent approaches.
- **Evaluating security and networking protocols:** serving as a detailed case study, the project generates insights into the effectiveness of cryptographic and communication protocols in decentralized environments.
- **Advancing trust and identity management research:** the system investigates practical methods for establishing and maintaining trust among peers in distributed networks.
- **Informing potential standardization:** the findings may contribute to future standards for secure, decentralized communication technologies.

1.5 Report Structure

This report is organized into eight chapters, structured as follows:

- **Chapter 1: Introduction** outlines the motivation, objectives, scope, and significance of the project.
- **Chapter 2: Background Knowledge** provides the theoretical foundation required to understand the system, including peer-to-peer communication, BLE, mesh topology, and cryptographic concepts.
- **Chapter 3: Related Work** reviews existing systems and prior work relevant to decentralized messaging and content sharing.
- **Chapter 4: Requirements Elicitation** defines the functional and non-functional requirements of the system.
- **Chapter 5: System Analysis** presents the behavioral analysis through use case, activity, and sequence diagrams.
- **Chapter 6: System Design** describes the architecture, data design and user interface design.
- **Chapter 7: System Implementation** discusses the tech stack, the application and the feature implementation.
- **Chapter 8: Testing and Deployment** describes the testing approach, test cases, test result and the application deployment.
- **Chapter 9: Conclusion and Next Phase** conclusion from the project and next phase discussions.

2 Background Knowledge

2.1 Decentralized Communication Systems

2.1.1 Peer-to-Peer Networks

A peer-to-peer (P2P) network is a decentralized system in which two or more computers are directly connected to exchange data. Unlike traditional client–server models, each computer in a P2P network can function as a client, a server, or both simultaneously. When a device joins the network, it can request data from other peers while also providing data to them, depending on the application in use.

The applications of P2P networking are diverse and far-reaching. Early examples include file-sharing systems such as Napster and BitTorrent, which revolutionized digital content distribution. Today, P2P principles underpin critical technologies such as cryptocurrencies and blockchain platforms [8].

2.1.2 Comparison with Centralized Client–Server Architecture

Both centralized and decentralized architectures present distinct advantages and disadvantages when applied to messaging applications.

Centralized Client–Server Systems

Advantages

- Ease of governance through a central administrative authority.
- Simplified implementation for synchronization, group management, and backup.
- Easier performance optimization with predictable latency and throughput.
- Uniform security policies because all traffic passes through a single control point.

Disadvantages

- Single point of failure during outages, attacks, or maintenance.
- Limited scalability when the number of users grows rapidly.
- Data ownership concerns when providers can access traffic or metadata.

- Censorship and surveillance risks due to centralized control.
- Dependence on stable internet connectivity and server infrastructure.

Decentralized Topologies

Advantages

- Scalability potential because each new peer contributes resources.
- High availability without a central point of failure.
- Better user data ownership when end-to-end encryption is applied.
- Resilience in restrictive or infrastructure-poor environments.
- Cost efficiency due to the absence of expensive centralized infrastructure.

Disadvantages

- Greater implementation complexity for routing, synchronization, and consistency.
- Harder integrity and reliability guarantees across distributed peers.
- Variable performance depending on peer availability and topology.
- Security risks from malicious peers when identity management is weak.
- Increased battery and CPU usage due to peer discovery and relaying.

2.2 Bluetooth Low Energy

2.2.1 Bluetooth Low Energy Communication

In Bluetooth Low Energy (BLE), there are two main communication modes:

- **Broadcast communication:** devices communicate without first establishing a connection by broadcasting data packets that can be received by nearby devices.
- **Connection-oriented communication:** devices establish a dedicated connection before exchanging data, enabling bidirectional communication.

The primary advantage of a connected topology is the higher throughput achieved through a direct link established before communication begins. In addition, communication is bidirectional, meaning that both the central and the peripheral can exchange data with each other. In contrast, in broadcast communication, the peripheral only transmits advertising data and does not receive data back through the same interaction. This project therefore focuses on connection-oriented communication to reduce data loss, support bidirectional exchange, and improve throughput, all of which are important for maintaining a healthy mesh in a P2P system.

2.2.2 Bluetooth Low Energy Role

In connection-oriented BLE communication, two actions correspond to two fundamental operating roles:

- **Advertising:** the process of transmitting advertising packets, either to broadcast data or to allow discovery by another device.
- **Scanning:** the process of listening for advertising packets from nearby devices.

The associated device roles are as follows:

- **Central (Client):** a device that scans for peripherals and initiates connections.
- **Peripheral (Server):** a device that advertises and accepts connections from centrals.

On Android, a single device can operate in multiple BLE roles simultaneously. For example, the same device may act as a peripheral in one interaction while acting as a central in another. This project uses that capability to allow the application to switch roles as needed in order to help maintain a healthy mesh.

2.2.3 Bluetooth Low Energy Service and Characteristics

In connection-oriented BLE communication, a peripheral device must define the services and characteristics that central devices use to discover and interact with it. A service is a collection of related data and behaviors, represented by characteristics, that organizes information exposed by the peripheral device. Both services and characteristics are identified using unique UUIDs, allowing compatible devices to recognize them, establish a connection, and exchange data.

Among the different characteristic types available in BLE, this project focuses on write characteristics, which enable data transfer from the central device to the peripheral device.

2.2.4 Bluetooth Low Energy Limitation

As its name suggests, BLE is designed to minimize power consumption on mobile devices, and this design introduces several limitations that must be addressed in the proposed system:

- **Payload size limit:** exchanged data payloads are typically limited to between 20 and 512 bytes depending on device capabilities. To address this constraint, the project implements fragmentation and reassembly mechanisms.
- **Connection limit:** the number of simultaneous BLE connections usually ranges from approximately 5 to 25 depending on hardware. To manage this limitation, the project introduces a lightweight protocol that enables devices to attempt reconnection whenever the number of active links falls to a critically low level.

Bluetooth Low Energy (BLE) was introduced in 2010 as an alternative to Bluetooth BR/EDR, specifically optimized for low power consumption. This efficiency makes BLE well-suited for battery-constrained devices such as smartphones and IoT systems. On Android, BLE is supported through official APIs that enable device discovery, connection management, and data exchange. However, when devices communicate via BLE, transmitted data may be accessible to applications running on the same device. To address this concern, the proposed system integrates strong cryptographic mechanisms to ensure that all exchanged data remains confidential and protected against unauthorized access or interception [2, 4].

2.3 Mesh Topology

In a decentralized topology, any machine can connect directly to any other machine, as illustrated in Figure 1. To join the network, a peer typically begins by contacting and advertising itself to a neighboring peer — a process known as *bootstrapping*. Through bootstrapping, the peer may also receive additional network information depending on the application's implementation. A well-known example of this topology is the early versions of the Gnutella file-sharing system, which relied on peers discovering and connecting to one another without centralized coordination.

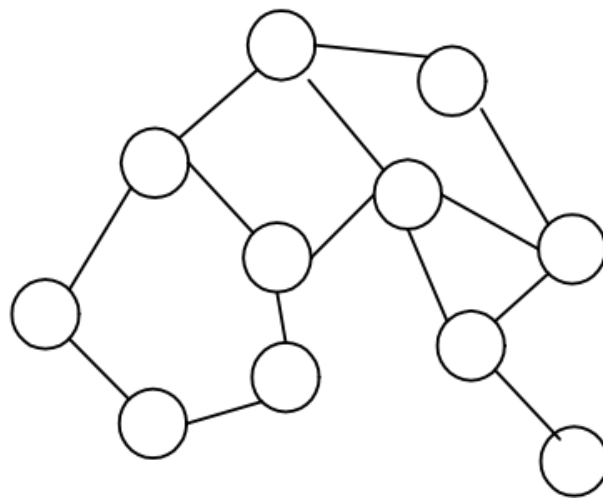


Figure 1: *Decentralized Topology*

2.4 Cryptography and Security

2.4.1 End-to-End Encryption

End-to-end encryption (E2EE) is a secure communication technique in which a message is encrypted at the sender's device and remains encrypted while traversing the network, including any intermediate nodes. Only the intended recipient, possessing the correct decryption key, can decrypt and read the message. This ensures that no third party — including service providers, network operators, or potential attackers — can access the content of the communication [11].

Over the years, several protocols have been developed to support end-to-end encryption, including:

- **Signal Protocol:** widely adopted in modern messaging applications such as Signal and WhatsApp, with forward secrecy and strong authentication.
- **TLS in E2EE mode:** although Transport Layer Security is usually used for client-server communication, it can also be configured to provide end-to-end protection between communicating parties.
- **Matrix Protocol (Olm/Megolm):** used in decentralized messaging platforms such as Element for both one-to-one and group communication.

2.4.2 Hash Functions

Hash functions convert arbitrary-length input into fixed-size outputs, enabling integrity verification. Common hash algorithms include SHA-2 and SHA-3, while legacy algorithms such as MD5 and SHA-1 are now considered insecure.

2.4.3 Symmetric-Key Cryptography

Symmetric-key algorithms use a single shared key for encryption and decryption. Popular examples include AES and ChaCha20, while DES is considered obsolete.

2.4.4 Asymmetric-Key Cryptography

Asymmetric-key cryptography employs public and private key pairs, enabling secure communication without prior key exchange. Widely used algorithms include RSA, elliptic-curve cryptography (ECC), and DSA.

2.4.5 Key Exchange Mechanisms

Key exchange protocols allow secure establishment of shared secrets over insecure channels. Notable mechanisms include Diffie–Hellman (DH), Elliptic Curve Diffie–Hellman (ECDH), and Internet Key Exchange (IKE).

3 Related Work

This section reviews existing research and applications that address similar use cases to the proposed system. The surveyed works include applications that utilize peer-to-peer communication, Bluetooth Low Energy (BLE), and security mechanisms in distributed environments.

3.1 Bitchat

Bitchat, developed by Jack Dorsey, is a decentralized peer-to-peer messaging application that employs a dual transport architecture. It supports local Bluetooth mesh networks for offline communication and the internet-based Nostr protocol for broader reach. This design enables operation in both connected environments and scenarios where internet access is unavailable, such as disasters or remote areas [9].

Notable features of Bitchat include:

- **End-to-end encryption:** it uses the Noise Protocol Framework for authentication, key exchange, and forward secrecy within the Bluetooth mesh.
- **Local communication:** it supports direct peer-to-peer messaging within Bluetooth range.
- **Multi-hop relay:** it forwards messages across nearby devices, up to several hops.
- **Offline capability:** it functions without internet connectivity.
- **Binary protocol:** it employs a compact packet format optimized for BLE constraints.
- **Automatic discovery:** it includes built-in peer discovery and connection management.
- **Adaptive power management:** it uses battery-optimized duty cycling to reduce energy consumption.

Advantages

- Dual communication modes that support both offline Bluetooth mesh and online communication.

- Strong security through end-to-end encryption.
- Good energy efficiency on mobile devices.

Disadvantages

- No explicit peer presence detection beyond discovery.
- Susceptibility to flooding-based denial-of-service behavior.
- Limited identity management for local peer verification.
- No file-sharing support in the analyzed version.

3.2 Briar

Briar is an open-source peer-to-peer messaging application designed for secure and resilient communication. Unlike conventional messaging platforms, Briar does not rely on centralized servers. Instead, messages are synchronized directly between users' devices via Bluetooth, Wi-Fi, Tor over the internet, or removable storage such as USB drives [5].

Key features of Briar include:

- **End-to-end encryption:** all communications are encrypted by default.
- **Offline communication:** it supports Bluetooth and Wi-Fi messaging without internet access.
- **Tor integration:** it can route traffic through the Tor network to improve anonymity.
- **Decentralized synchronization:** data is stored in encrypted form on participating devices.
- **Resilient transport options:** it also supports data exchange via removable media.
- **Open source distribution:** it is released under GPL-3.0.

Advantages

- Strong privacy protections through encryption and anonymized routing.
- Strong offline capability in disaster or restricted environments.
- Decentralized design without a single point of failure.

- Cross-platform support that extends beyond Android.
- A design focus on activists, journalists, and high-risk users.

Disadvantages

- Limited discoverability because communication is largely restricted to pre-established contacts.
- No explicit online presence indicators.
- Feature limitations such as lack of voice or video communication.
- Full functionality is primarily centered on Android.
- Synchronization can be slower than centralized alternatives.

3.3 Peer-to-Peer Networks for Content Sharing

Peer-to-peer content-sharing systems provide foundational insights into decentralized network behavior. One of the earliest and most influential examples is Gnutella, which pioneered large-scale decentralized file sharing.

3.3.1 Gnutella v0.4: Early Development

In its early versions, Gnutella v0.4 implemented a fully decentralized mesh network [10]. Key characteristics included:

- **Bootstrapping:** new nodes joined the network by connecting to known peers.
- **Ping and Pong messages:** these were used as heartbeat packets to track peer liveness.
- **Flooding and adaptive routing:** queries were broadcast across the network to locate content.

Advantages

- Guaranteed content discovery when content existed and the network remained connected.
- A simple decentralized model with no central index.

Disadvantages

- Bottlenecks, where weak peers could act as cut vertices and fragment the network.
- Network overload due to excessive control traffic and flooding overhead.

3.4 Research Summary

3.4.1 Overall Insights

Based on the analysis of existing applications and systems, several key insights emerge:

- Few applications provide reliable messaging over Bluetooth-based peer-to-peer networks.
- Peer-to-peer innovation has shifted from early file-sharing systems toward blockchain, decentralized finance, decentralized social platforms, and secure messaging.

3.4.2 Opportunities and Challenges

Opportunities

- Combining offline Bluetooth mesh and online communication fills a notable gap.
- Advances in encryption and identity management can be leveraged in modern designs.
- The problem domain is highly relevant for disaster recovery and censorship-heavy regions.
- Rising concern over surveillance creates demand for privacy-preserving communication tools.
- Activists, journalists, and low-connectivity communities form a viable user base.

Challenges

- Bluetooth remains limited in range, bandwidth, and interference tolerance.
- User trust and adoption may be difficult to achieve.
- Flooding and routing overhead create scalability challenges.
- Identity verification and attack mitigation remain non-trivial.
- Rich media, file sharing, and presence detection are harder to implement robustly.



- Fewer active research communities focus specifically on Bluetooth-based messaging systems.

4 Requirements Elicitation

4.1 Functional Requirements

User Requirements

- U1. Bootstrapping:** The system enables users to scan the mesh network for all available peers and select a specific recipient for communication.
- U2. Peer Detection:** the system shall continuously monitor and maintain awareness of all active devices within the mesh network.
- U3. Mesh Maintenance:** the device must support autonomous network entry upon startup. Additionally, it must feature a self-healing mechanism that triggers a re-join request whenever the peer-to-peer connection count drops below the operational minimum.
- U4. Offline Chat:** users shall be able to chat without an internet connection.
- U5. User Profile:** users shall be able to choose and change their nickname and profile picture.
- U6. Chat History:** the system shall support chat history backup and export.
- U7. Notifications:** the system shall provide chat notifications.
- U8. Tutorial/Onboarding:** the system shall provide a tutorial or onboarding flow for first-time users.
- U9. Appearance:** the system shall support dark mode and accessibility options.
- U10. Relay Functionality:** the user's device must help forward messages to other peers if it is not the destination.

4.2 Non-Functional Requirements

Security

- S1.** All messages and data must be encrypted end-to-end using a secure protocol.
- S2.** The system must ensure message and data traceability by reliably associating each transmission with its originating sender.

- S3.** The system shall guarantee data integrity by preventing unauthorized modification or tampering.

Scalability

- SC1.** The system must be able to handle a network of 100–500 nodes.

Performance

- P1.** Peer discovery should complete within 10 seconds under typical mesh conditions.
- P2.** Message delivery latency should be under 1 second for nearby peers.

Archivability

- A1.** The system shall persistently store user-associated messages on the user's device until explicitly deleted or modified by user-defined retention settings.
- A2.** Messages not directly associated with the user shall be temporarily buffered when the intended recipient is unavailable and automatically forwarded once the recipient becomes reachable. After successful delivery, buffered copies shall be deleted automatically.

5 System Analysis

5.1 Use Case Diagrams

5.1.1 Whole System

The Bluetooth Mesh Chat App allows users to communicate with each other through Bluetooth connections without relying on the internet. The system consists of two main actors: the *User* and the *Device*. The user interacts directly with the application to perform actions such as scanning for nearby devices, selecting who to chat with, and sending messages. Meanwhile, devices in the network act as peers or relays, helping transmit messages across multiple nodes to maintain connectivity in the mesh network.

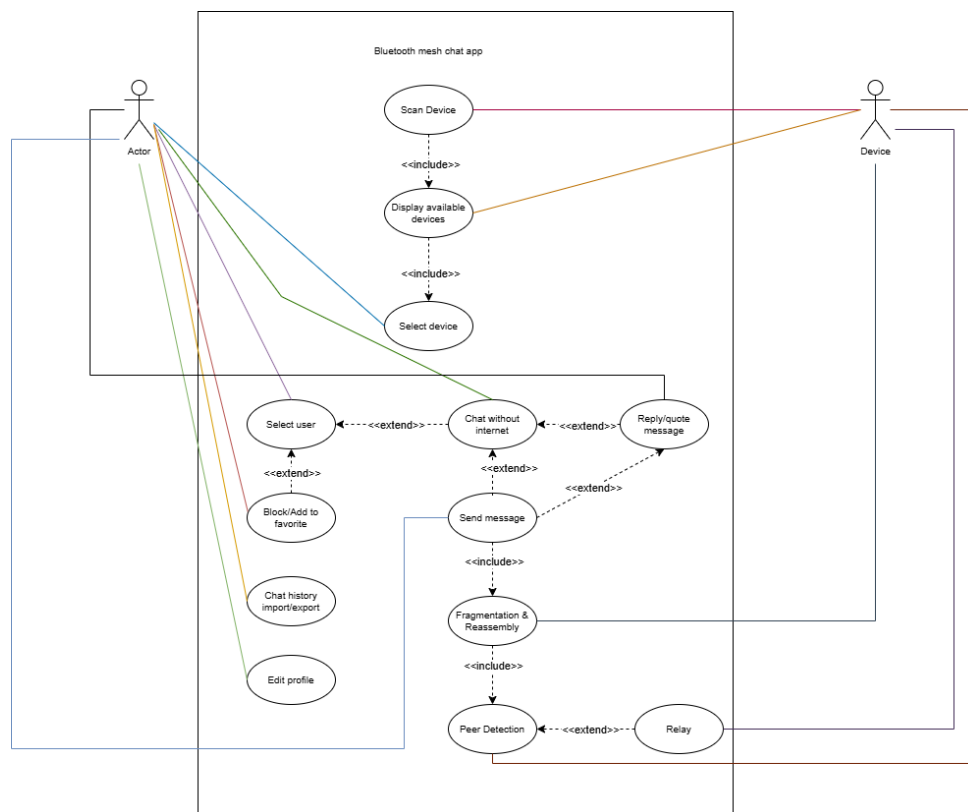


Figure 2: Usecase diagram: Whole system

The process usually begins when the user scans for nearby devices. This use case includes displaying a list of available devices and selecting one to connect with. Once connected, the user can initiate communication through the chat-without-internet feature, which is the core function of the system. This use case extends related functions such as sending

messages, replying to or quoting messages, and selecting users. It represents the ability to exchange messages directly between devices without requiring mobile data or Wi-Fi.

When the user chooses to send a message, the system ensures efficient and reliable delivery through two important processes: fragmentation and reassembly, and peer detection. Fragmentation and reassembly handle the breaking down of large messages into smaller packets suitable for Bluetooth transmission and reassembling them correctly on the receiving device. Peer detection identifies available mesh network nodes that can receive or forward messages. If a direct connection is not possible, the system uses the relay function, where another device helps forward the message to its destination, maintaining the mesh network's multi-hop communication feature.

Beyond chatting, the user can perform several additional actions to enhance their experience. The select-user use case allows choosing specific users to chat with and can extend to features such as blocking or adding to favorites, enabling users to manage their contact preferences. The app also supports replying to and quoting messages, which allows referencing previous messages in a conversation for clarity. Furthermore, users can edit their profiles and can import or export chat history to save important conversations or transfer data between devices.

5.1.2 Bootstrapping

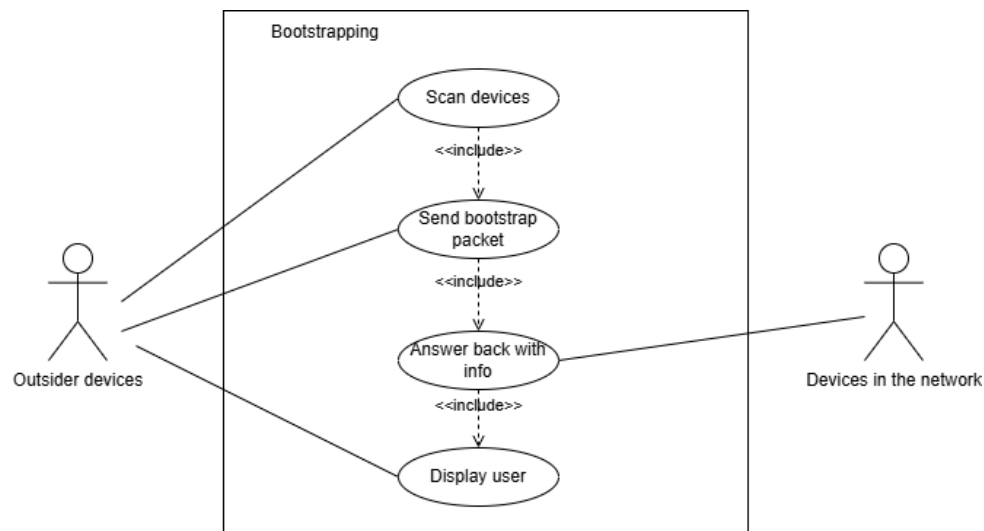


Figure 3: Usecase diagram: Bootstrapping

The bootstrapping system begins when an outsider device initiates a scan for devices to find other nodes in its surroundings. This scanning process includes the broadcast-device-presence use case, where the outsider device announces its availability to nearby devices. This broadcast allows devices already in the network to recognize and potentially connect with the outsider device. The relationship here is mutual: while the outsider device scans

and broadcasts its presence, devices already in the network also listen for new broadcasts to maintain awareness of potential peers.

Once the broadcast is received, devices in the network can respond in one of two ways: accept user or decline user. These two actions represent the decision-making step that determines whether the outsider device will be integrated into the mesh network. Both of these use cases extend from broadcast-device-presence, meaning they are optional outcomes triggered depending on the system's or user's decision.

If the device is accepted, the system proceeds to update the contact database and cache. This ensures that the newly accepted device is stored locally so that future communication can occur smoothly without re-scanning or re-broadcasting. On the other hand, if the device is declined, the system still executes display user, which allows existing users or devices to view the detected outsider device without integrating it into the network.

5.1.3 Open Single Communication Channel

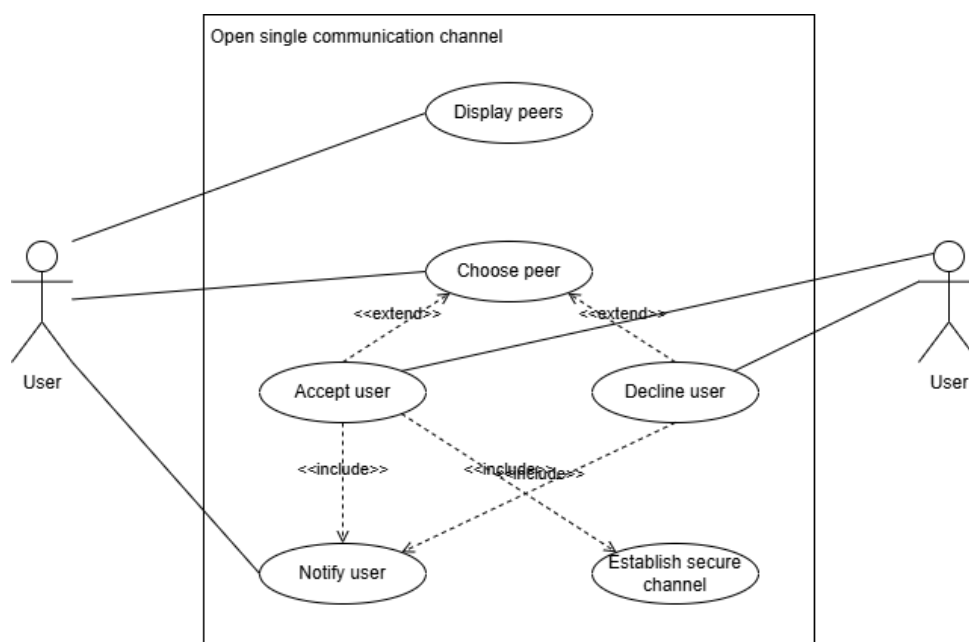


Figure 4: Usecase diagram: Open single communication channel

When a user performs the display-peers use case, the process begins. This step allows the initiating user to view nearby or available peers within Bluetooth range or reachable via the mesh network. Once peers are displayed, the user proceeds to choose a peer, selecting a specific device or person they wish to communicate with. This marks the start of an attempt to open a single, direct communication channel between the two devices.

After choosing a peer, the system transitions to a decision phase involving two possible actions: accept user or decline user. These use cases extend from choose peer, as they are

conditional outcomes that depend on how the receiving user responds to the connection request. If the receiving user accepts the request, the system proceeds with establish secure channel, which ensures that the communication link is encrypted and authenticated to prevent unauthorized access or data interception.

Whether the request is accepted or declined, the system also includes notify user. This function ensures that the initiating user is informed about the outcome of the connection request — whether the communication channel has been successfully established or denied. Both accept-user and decline-user include this notification feature, ensuring that users receive consistent feedback on their connection attempts.

5.1.4 Process Message

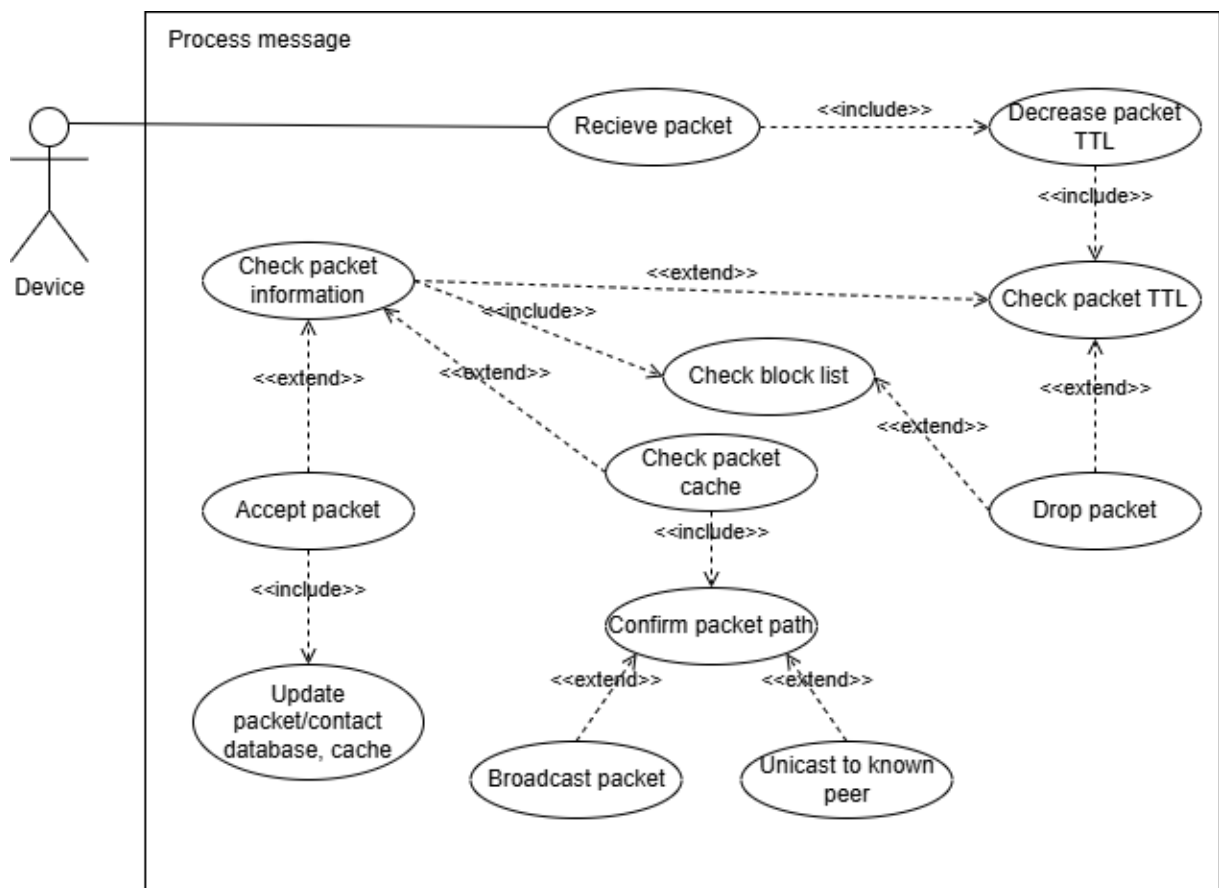


Figure 5: Usecase diagram: Process message

When a device receives a packet from another peer, the process begins with receive packet. Immediately after receiving, the system executes decrease packet TTL and check packet TTL. The TTL value determines how many hops a packet can make before being discarded, preventing infinite circulation within the mesh. If the TTL reaches zero, the system executes drop packet, ending the process for that data unit.

If the packet remains valid, the system proceeds to inspect packet information such as sender ID, destination ID, and sequence number to determine whether the packet should be processed or ignored. Several optional or conditional actions extend from this stage. For instance, checking the block list ensures that packets from blocked devices are filtered out, while checking the packet cache prevents duplicate forwarding by verifying whether the packet has already been processed recently. Both checks help maintain security and reduce unnecessary traffic in the mesh network.

Once the packet passes these checks, the system moves to accept packet, which indicates that the packet is legitimate and ready to be processed or forwarded. Accepting the packet includes updating the packet/contact database and cache, allowing the system to store new peer information and maintain an updated record of recently seen packets and nodes. After the packet is accepted, the device confirms the packet path and either broadcasts the packet to the mesh or unicasts it to a known peer depending on routing information.

5.1.5 Send Message

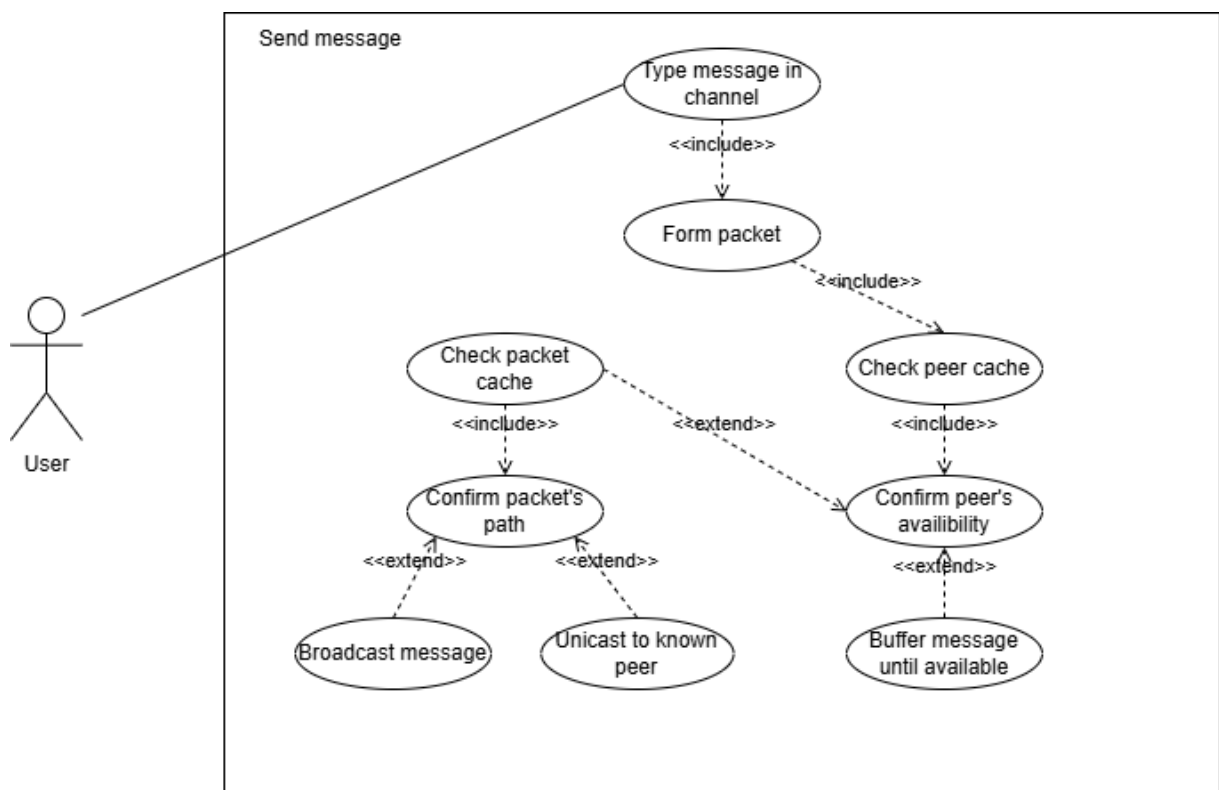


Figure 6: Usecase diagram: Send message

The process starts when the user initiates type message in the channel. Once the user types a message, the system automatically proceeds to form packet, which packages the message content together with metadata such as source ID, destination ID, timestamp, and sequence number.

After the packet is formed, the system performs two verification checks: check packet cache and check peer cache. Checking the packet cache ensures that the message is not a duplicate of a recently sent or processed packet, preventing redundancy and network congestion. At the same time, checking the peer cache verifies that the target peer is known and that its address and status are stored locally.

Next, the system confirms the packet path and the recipient's readiness. If a known path is available, the message may be unicasted to a known peer. If no known route exists, the system may instead broadcast the message so that intermediate peers can help forward it. In parallel, confirming peer availability checks whether the target peer is currently reachable. If the peer is temporarily unavailable, the system buffers the message until the peer becomes reachable again.

5.1.6 Fragmentation

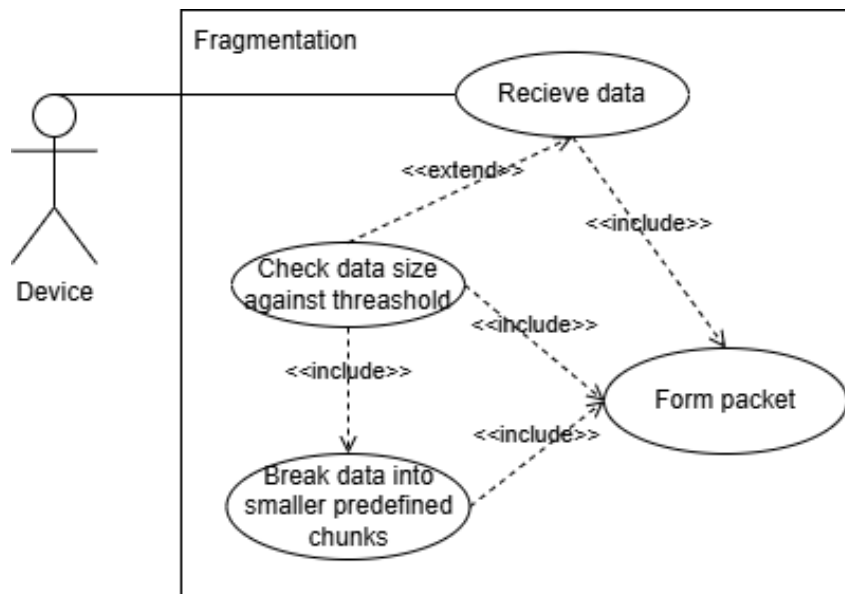


Figure 7: Usecase diagram: Fragmentation

The main actor in this use case is the device, which may be any node in the mesh network. The process begins when the device receives data, such as a message or file, that needs to be prepared for transmission. The system then checks the data size against the predefined threshold. If the data size exceeds this threshold, the system breaks the data into smaller predefined chunks so that each chunk conforms to the mesh packet size limit.

After fragmentation, or if the data already fits within the size limit, the process proceeds to form packet, which organizes the data into transmission-ready packets that include addressing, integrity checks, and sequence metadata.

5.1.7 Form Packet

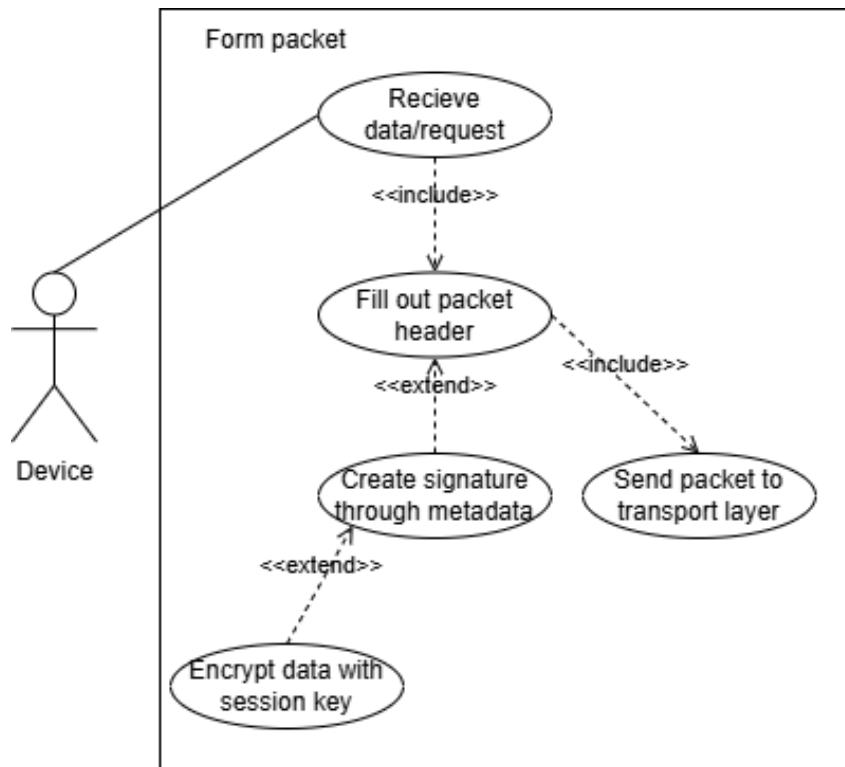


Figure 8: Usecase diagram: Form packet

The process begins when the device receives data or a request, which could originate from higher-level processes such as user input, fragmented data chunks, or control messages. After receiving the data, the system fills out the packet header, where it constructs the essential metadata such as source and destination addresses, message type, sequence number, and other routing-related information.

Once the header is prepared, the system may create a metadata-based signature to support integrity checking and may encrypt the payload using a session key to preserve confidentiality. Finally, the completed packet is sent to the transport layer for unicast or broadcast transmission.

5.1.8 Reassembly

Reassembly complements the fragmentation process on the receiving side of the mesh network. After a device receives and accepts an incoming packet, the system inspects the fragment-related metadata to determine whether the payload belongs to a multi-part message. If the packet is not fragmented, the complete payload can be returned directly to the upper processing layer.

If the packet is part of a fragmented payload, the system creates a fragment key from

stable packet metadata so that all fragments from the same original message are grouped together consistently. Each fragment is stored in a temporary queue associated with that key, and the system checks whether the full fragment set has already been collected. Once all required fragments are available, the device orders them correctly and rebuilds the original payload before handing the completed packet to the next stage for decryption, validation, and message delivery.

5.1.9 Mesh Maintainer

The Mesh Maintainer is a background system use case responsible for preserving the minimum number of active links required for stable mesh participation. Instead of being triggered directly by the user, it runs continuously while the application is active and periodically checks whether the current number of active peer connections has fallen below the configured operating threshold.

When the mesh becomes under-connected, the system scans for nearby devices that are already recognized as part of the mesh, evaluates which peers are suitable candidates for reconnection, and temporarily resets its advertising state before attempting new links. This behavior supports the self-healing requirement of the network by allowing each node to recover from peer mobility, temporary disconnections, or topology changes without manual intervention.

If a connection attempt succeeds, the system stores the updated connection state and restores the normal in-mesh advertising behavior. If the attempt fails or no suitable devices are found, the failure is logged and the maintenance loop retries after a delay.

5.2 Use Case Description

5.2.1 Use Case 1: Bootstrapping

Use Case ID	UC-01
Use Case Name	Bootstrapping - Device Network Connection
Actors	• Outsider Device (Primary Actor) • Devices in the Network (Secondary Actor)
Description	The bootstrapping process where an outsider device discovers, authenticates, and joins the Bluetooth mesh network.

Preconditions	• Bluetooth is enabled on both devices • Network devices are broadcasting • Application is installed
Postconditions	• New device is provisioned and can communicate securely • Contact database is updated on all devices • User appears in network member list
Main Flow	1. Outsider device scans for available networks 2. Network devices broadcast their presence 3. Outsider device displays list of discovered networks 4. User selects network and sends join request 5. Network device validates authentication credentials 6. Network device accepts request and sends provisioning data 7. Outsider device receives encryption keys and network configuration 8. All network devices update their contact database 9. System displays updated user list on all devices

Alternative Flow	3a. No devices found 1. System displays "No devices found" 2. User can retry scanning 5a. Authentication fails 1. Network device declines the request 2. System sends rejection notification 3. Use case ends with failure
-------------------------	---

Table 1: *Usecase table: Bootstrapping*

5.2.2 Use Case 2: Open single communication channel

Use Case ID	UC-02
Use Case Name	Open Single Communication Channel
Actors	User (Sender), User (Receiver)
Description	Establishing a secure one-to-one communication channel between two users in the mesh network.
Preconditions	• Both users are provisioned in the network • Users are within mesh network range
Postconditions	• Secure communication channel established • Both users can exchange encrypted messages • Connection status displayed to both users

Main Flow	1. Sender opens the application 2. System displays list of available peers 3. Sender chooses peer from the list 4. System sends connection request to receiver 5. Receiver's device receives connection request 6. Receiver accepts the connection request 7. System establishes secure encrypted channel 8. System notifies both users of successful connection 9. Communication channel is ready for messaging
Alternative Flow	2a. No peers available 1. System displays "No peers found" 2. System suggests scanning for new devices 3. Use case ends 6a. Receiver declines connection 1. Receiver selects decline option 2. System sends decline notification to sender 3. System displays "Connection declined" message 4. Use case ends without establishing channel

Exception Flow	4a. Connection timeout 1. No response from receiver within 30 seconds 2. System notifies sender of timeout 3. Use case ends 7a. Channel establishment fails 1. System retries establishing secure channel 2. If retry fails, display error message 3. Use case ends
-----------------------	---

Table 2: *Usecase table: Open single communication channel*

5.2.3 Use Case 3: Process Message

Use Case ID	UC-03
Use Case Name	Process Message
Actor	Device (System)
Description	Receiving, validating, and routing incoming packets through the mesh network.
Preconditions	Device is part of mesh network, packet received from another peer.
Postconditions	Packet delivered to destination or forwarded to next hop.

Main Flow	1. Device receives packet from mesh network 2. System decreases packet TTL (Time-To-Live) 3. System checks packet TTL validity 4. System checks packet information (headers, integrity) 5. System checks packet cache for duplicates 6. System confirms packet routing path 7. System accepts packet for processing 8. System updates packet/contact database cache 9. System unicasts to known peer or forwards to destination
Alternative Flow	4a. Sender is in block list 1. System checks block list 2. System drops packet 3. Use case ends 5a. Packet is duplicate (found in cache) 1. System drops packet to prevent loop 2. Use case ends 7a. Routing path unknown 1. System broadcasts packet to mesh 2. Use case ends

Exception Flow	3a. TTL expired (reaches 0) 1. System drops packet 2. Use case ends 4b. Packet corrupted or invalid 1. System drops packet 2. Use case ends
-----------------------	---

Table 3: Usecase table: Process message

5.2.4 Use Case 4: Send Message

Use Case ID	UC-04
Use Case Name	Send Message
Actor	User
Description	Sending encrypted message to peer(s) through the mesh network.
Preconditions	User is connected to mesh network, communication channel established.
Postconditions	Message delivered to recipient(s), delivery status confirmed.
Main Flow	1. User types message in channel 2. System forms encrypted packet 3. System checks packet cache to avoid duplication 4. System checks peer cache for recipient availability 5. System confirms peer's availability 6. System confirms optimal routing path 7. System unicasts message to known peer

Alternative Flow	5a. Peer not immediately available 1. System buffers message until peer available 2. Message delivered when peer comes online
	6a. Direct path unavailable 1. System broadcasts message through mesh 2. Message routed via intermediate peers

Table 4: *Usecase table: Send message*

5.2.5 Use Case 5: Fragmentation

Use Case ID	UC-05
Use Case Name	Fragmentation
Actor	Device (System)
Description	Breaking large data into smaller chunks to fit Bluetooth mesh packet size limits.
Preconditions	Data ready to transmit, packet size threshold defined.
Postconditions	Data fragmented into multiple packets ready for transmission.
Main Flow	1. Device receives data to transmit 2. System checks data size against threshold 3. System breaks data into smaller predefined chunks 4. System forms packets from each chunk
Alternative Flow	2a. Data size within threshold 1. System forms single packet directly 2. Skip fragmentation step 3. Use case ends

Table 5: *Usecase table: Fragmentation*

5.2.6 Use Case 6: Form Packet

Use Case ID	UC-06
Use Case Name	Form Packet
Actor	Device (System)
Description	Creating network packet with headers, encryption, and metadata for transmission.
Preconditions	Data/request ready, session keys available.
Postconditions	Complete encrypted packet ready for transport layer.
Main Flow	1. Device receives data or request 2. System fills out packet header (sender, receiver, timestamp, sequence) 3. System encrypts data with session key 4. System creates signature through metadata (integrity check) 5. System sends packet to transport layer

Table 6: *Usecase table: Form packet*

5.2.7 Use Case 7: Reassembly

Use Case ID	UC-07
Use Case Name	Reassembly
Actor	Device (System)
Description	Collecting fragmented packets and reconstructing the original payload before delivery to the upper layer.
Preconditions	• An incoming packet has been received. • Fragment metadata is available. • The fragment queue is accessible.
Postconditions	• The complete payload is returned for further processing, or partial fragments are buffered until the remaining fragments arrive.

Main Flow	1. Device receives an incoming packet. 2. System checks whether the packet is fragmented. 3. System generates a fragment key from the packet metadata. 4. System adds the fragment to the queue associated with the key. 5. System checks whether all fragments for the payload are available. 6. System orders fragments by sequence. 7. System reassembles the fragments into a complete payload. 8. System returns the complete packet to the upper layer.
Alternative Flow	2a. Packet is not fragmented. 1. System bypasses the reassembly queue. 2. System returns the packet directly for normal processing. 5a. Missing fragments. 1. System keeps the available fragments in the queue. 2. System returns no completed payload yet. 3. Use case resumes when the remaining fragments arrive.

Table 7: *Usecase table: Reassembly*

5.2.8 Use Case 8: Mesh Maintainer

Use Case ID	UC-08
Use Case Name	Mesh Maintainer

Actor	Device (System)
Description	Monitoring the current number of active peer connections and autonomously restoring mesh connectivity when the connection count drops below the operational minimum.
Preconditions	• The application is running. • Bluetooth is enabled. • The device already has cached information about known mesh peers.
Postconditions	• Connection state is updated. • Advertising state is reset appropriately. • The node either rejoins enough peers or records the failure for a future retry.

Main Flow	1. Background service checks the number of active peer connections. 2. System determines that the number of active connections is below the configured limit. 3. System scans for nearby devices that belong to the mesh. 4. System verifies whether suitable in-mesh devices are available. 5. System waits for a short randomized backoff interval. 6. System updates the in-mesh state to false and resets the advertiser. 7. System selects suitable discovered peers. 8. System attempts to establish or restore connections. 9. System saves the connection state when a connection succeeds. 10. System waits for stabilization, restores the in-mesh state, and resumes monitoring until the app is closed.
------------------	---

Alternative Flow	2a. Active connections are still sufficient. 1. System skips the recovery procedure. 2. System keeps the current advertiser state. 3. System continues periodic monitoring. 4a. No suitable in-mesh devices are found. 1. System restores the advertiser state. 2. System waits for the next maintenance interval. 3. Use case resumes from the monitoring step. 8a. Connection attempt fails. 1. System logs the failed attempt. 2. System waits before retrying. 3. Use case resumes from the monitoring step.
Exception Flow	1a. Application is closed during maintenance. 1. System stops the maintenance loop. 2. Use case ends.

Table 8: *Usecase table: Mesh maintainer*

5.3 Activity Diagrams

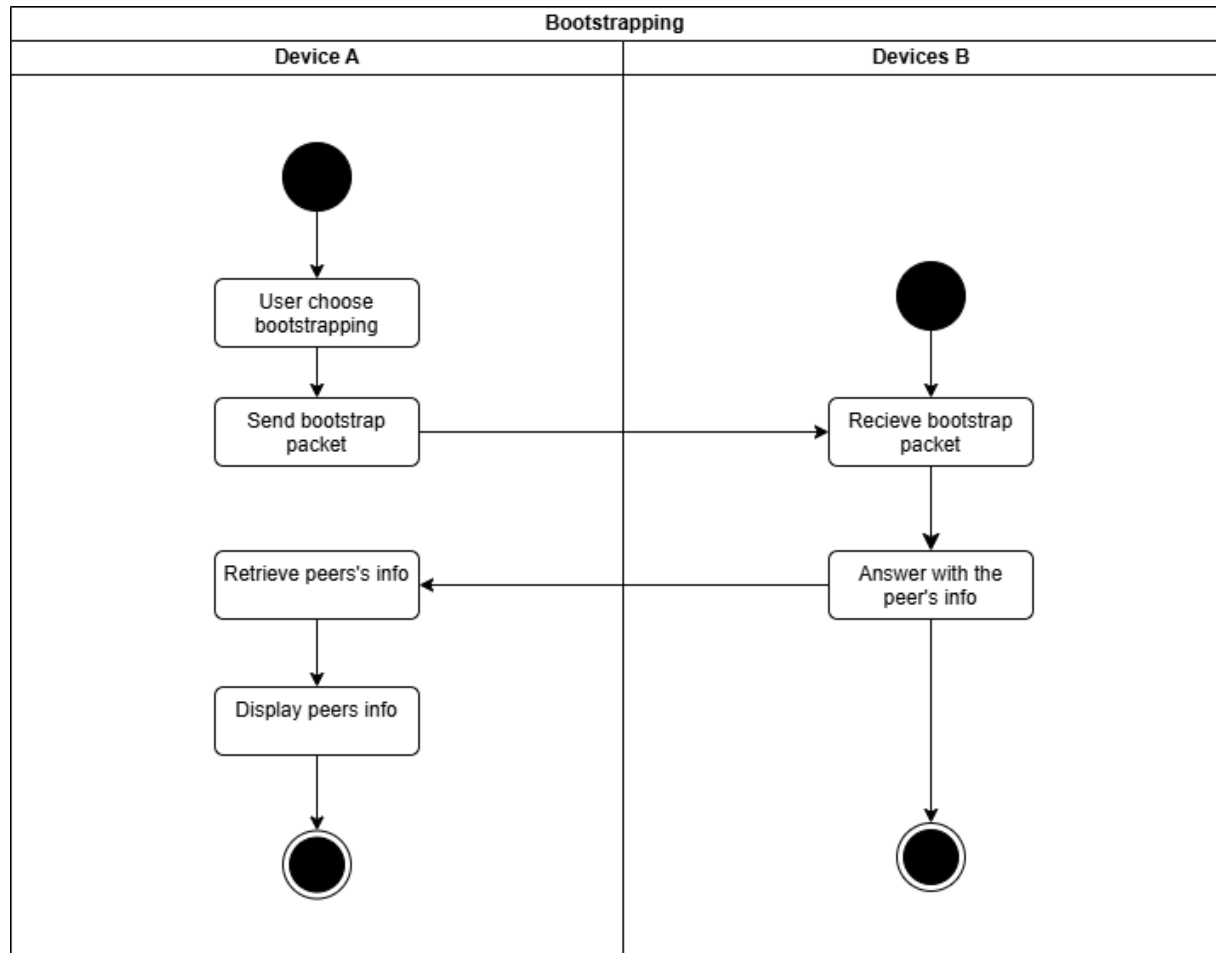


Figure 9: Activity diagram: Bootstrapping

The application continuously scans nearby devices over Bluetooth and reads each peer's basic information from advertisements or GATT while also broadcasting its own presence. When the user accepts a peer, the app fetches that peer's network details, such as capabilities and keys, then updates a local peer cache with a fresh timestamp and TTL. If the user declines, the temporary record is discarded. The interface then displays the current list of discovered users. This loop repeats so that the list stays up to date as devices appear or leave.

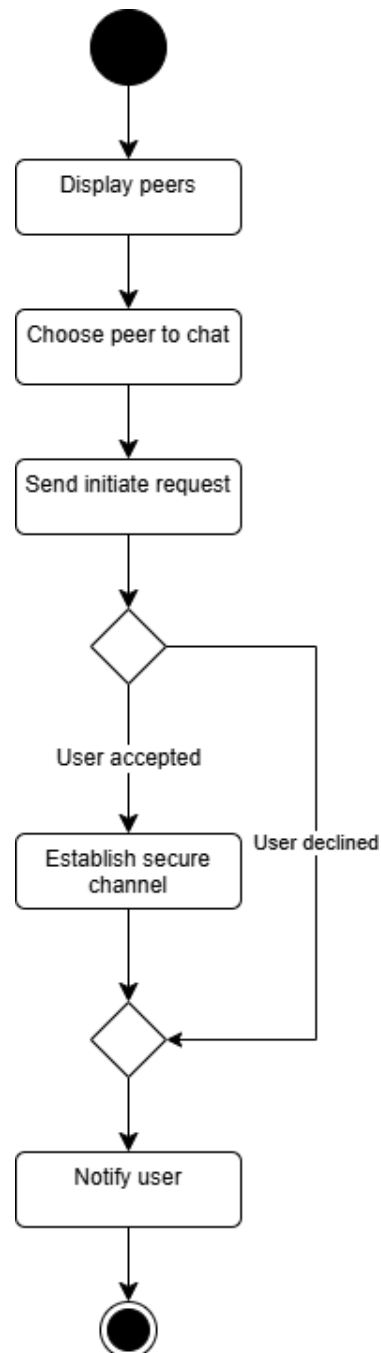


Figure 10: *Activity diagram: Open communication channel*

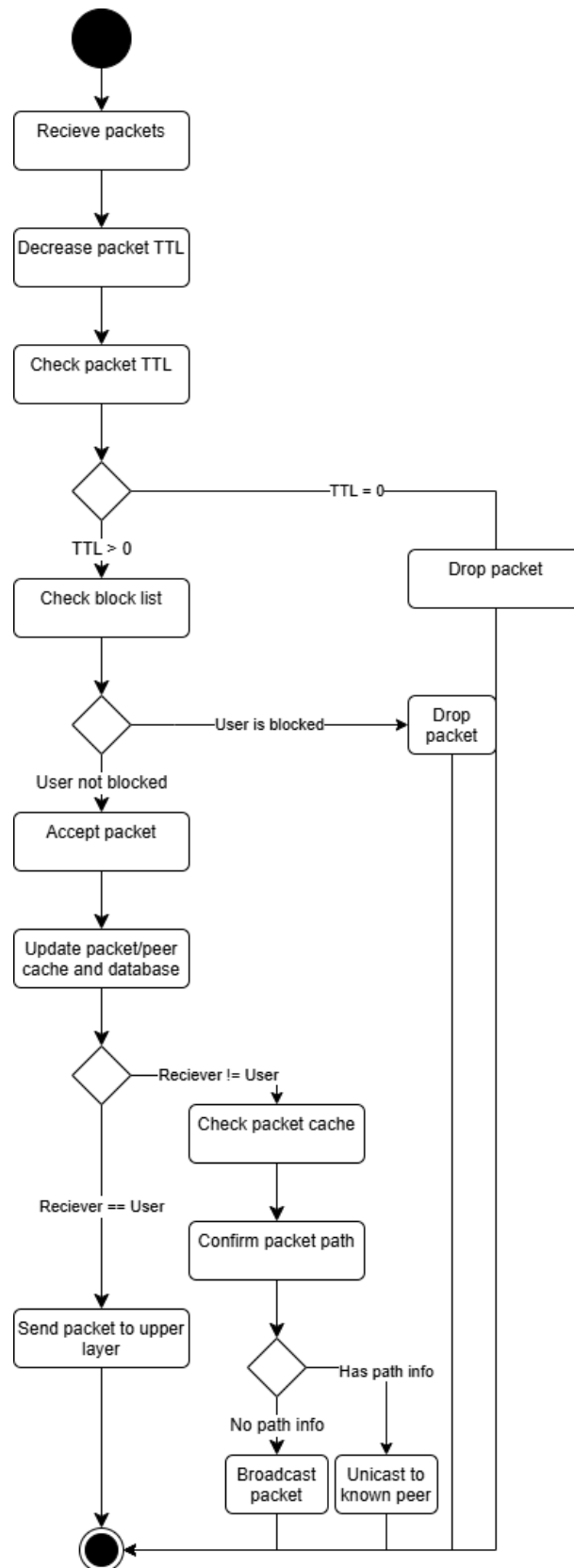


Figure 11: Activity diagram: Process message

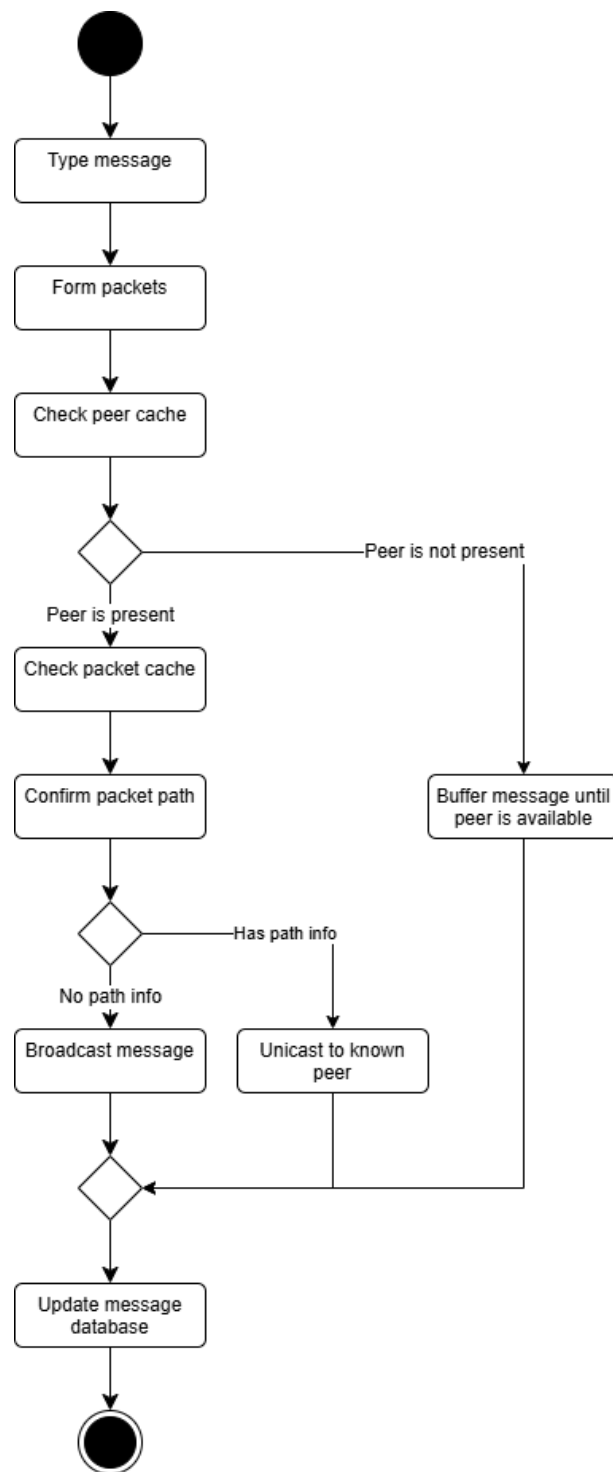


Figure 12: Activity diagram: Send message

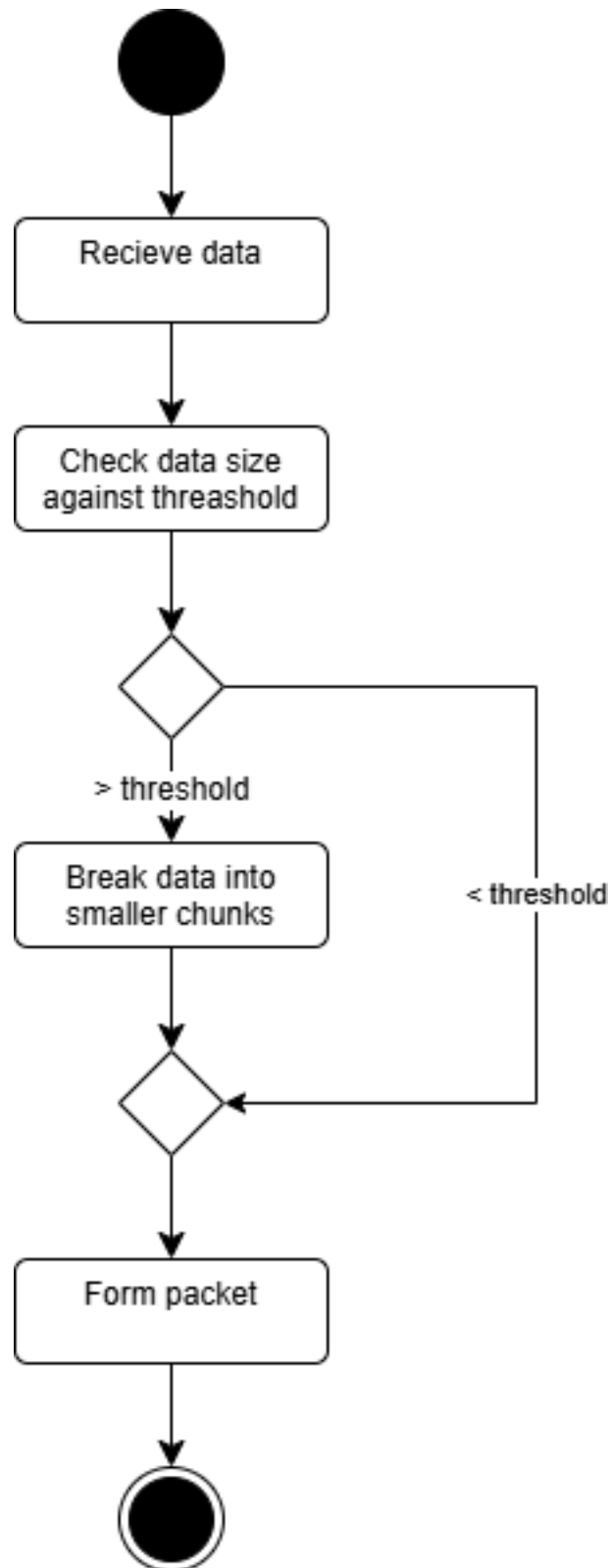


Figure 13: *Activity diagram: Fragmentation*

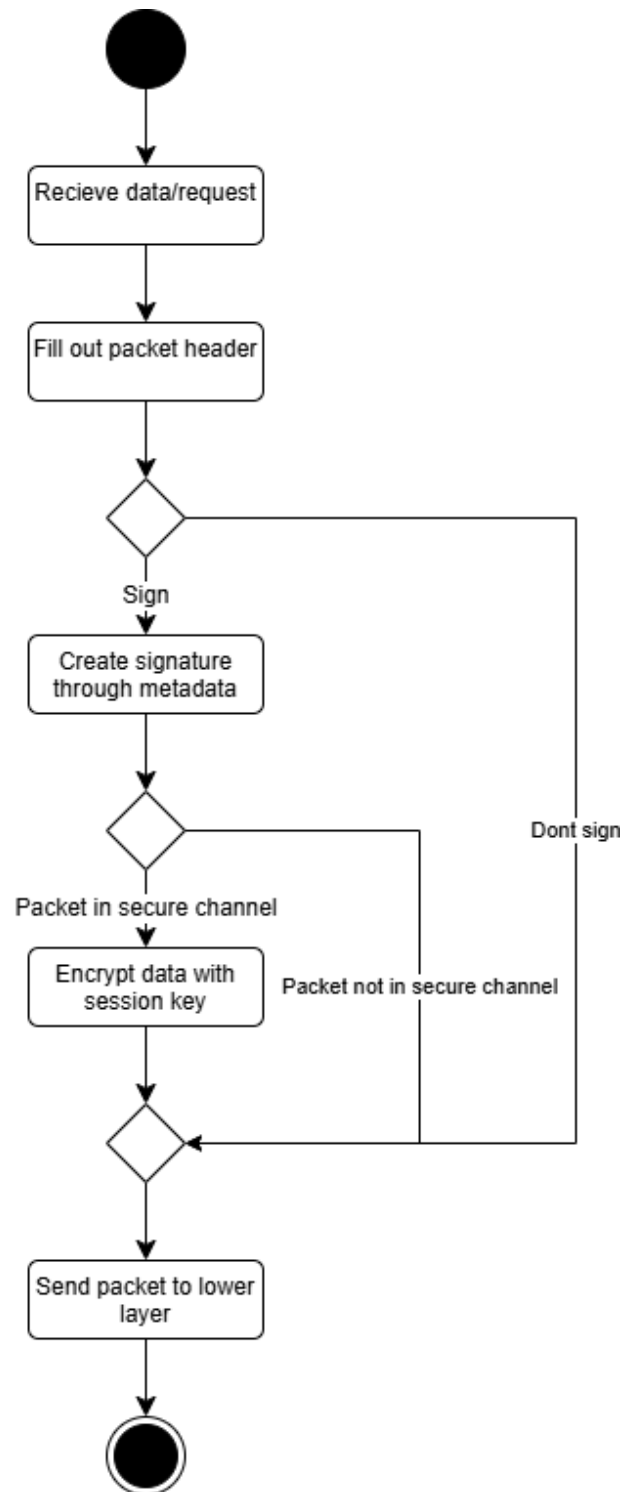


Figure 14: Activity diagram: Form packet

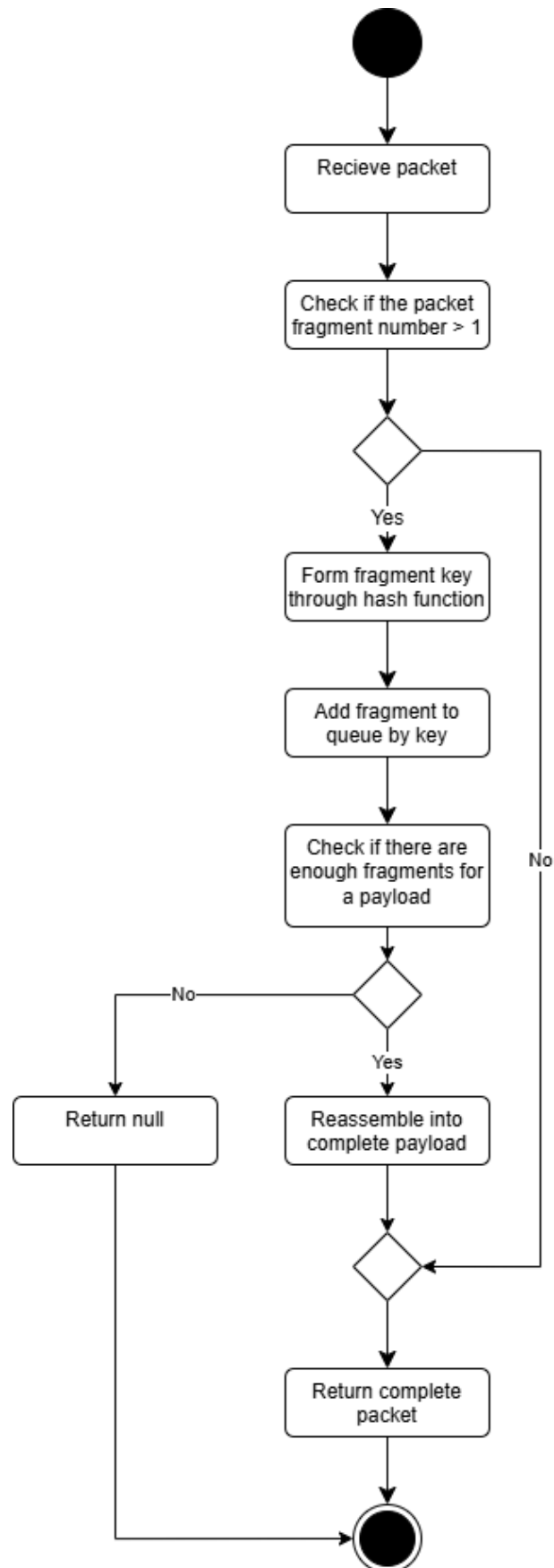


Figure 15: Activity diagram: Reassembly

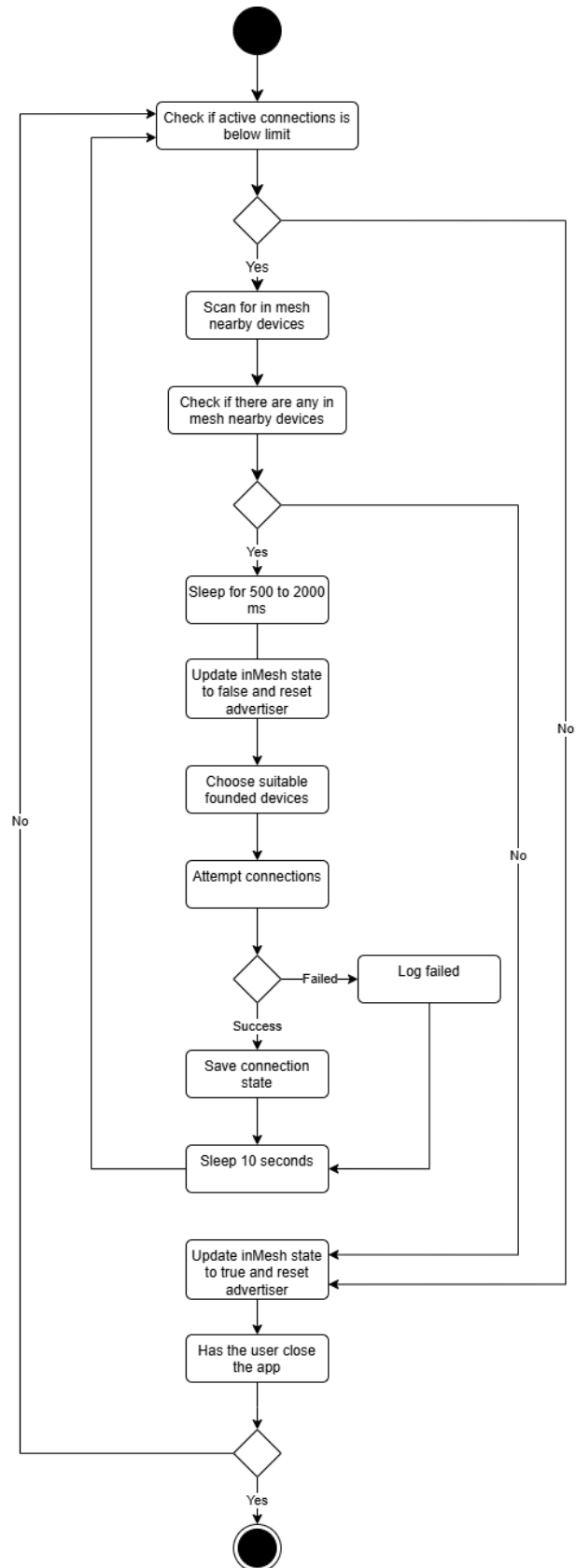


Figure 16: Activity diagram: Mesh maintainer

5.4 Sequence Diagrams

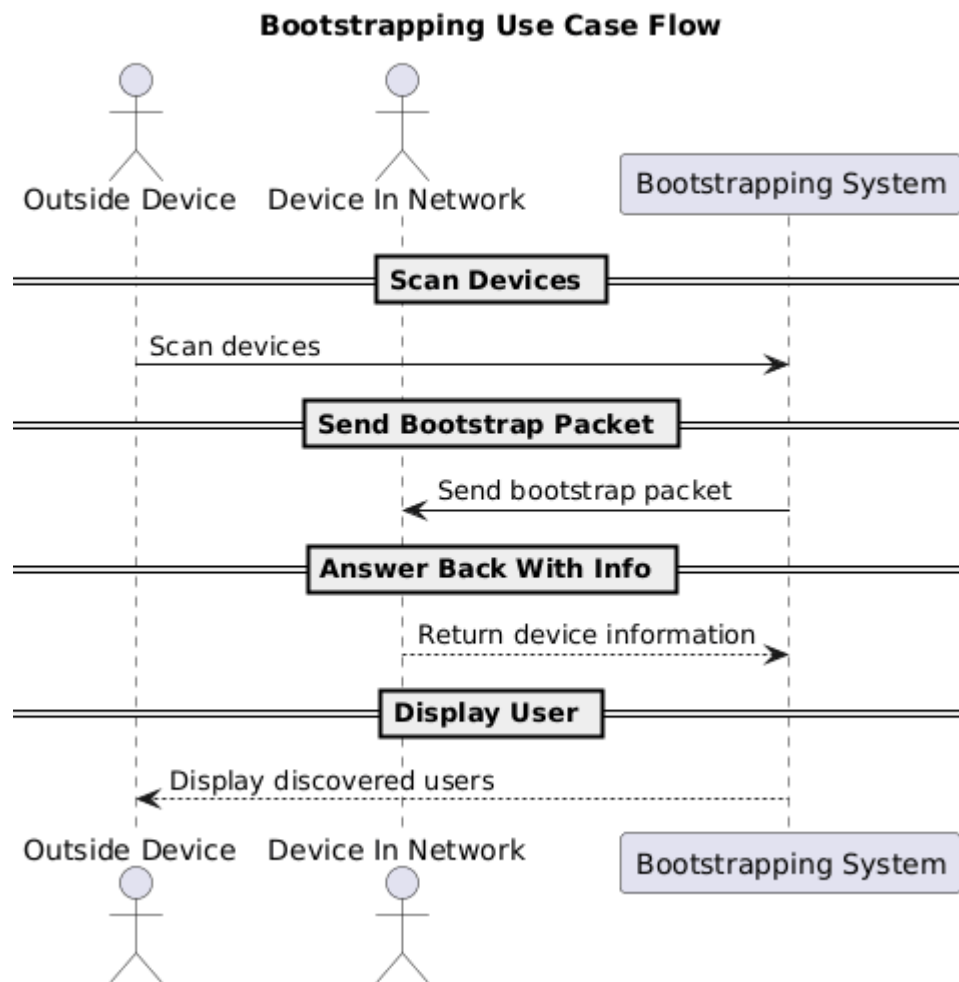


Figure 17: Sequence diagram: Bootstrapping

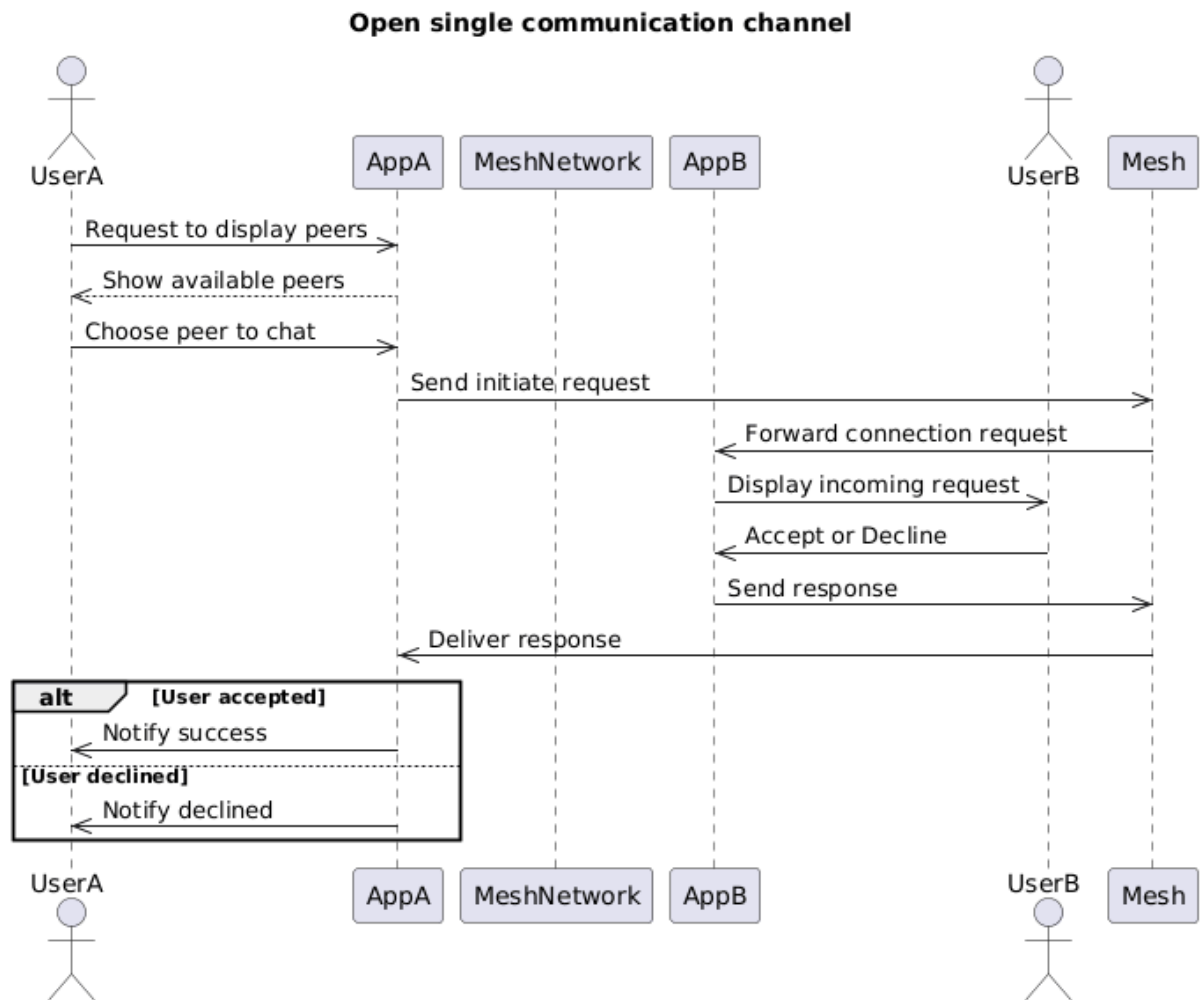


Figure 18: Sequence diagram: Open communication channel

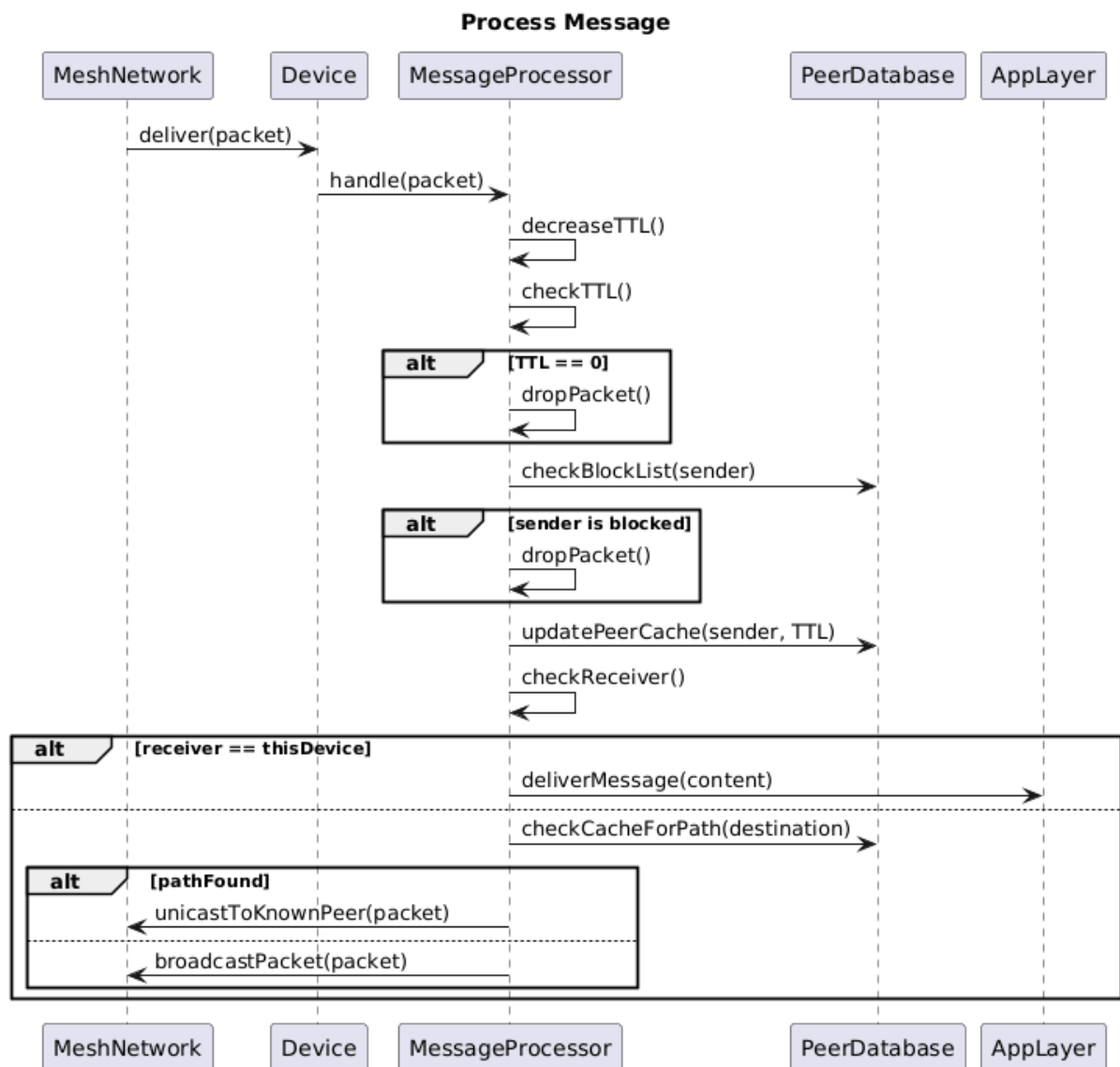


Figure 19: Sequence diagram: Process message

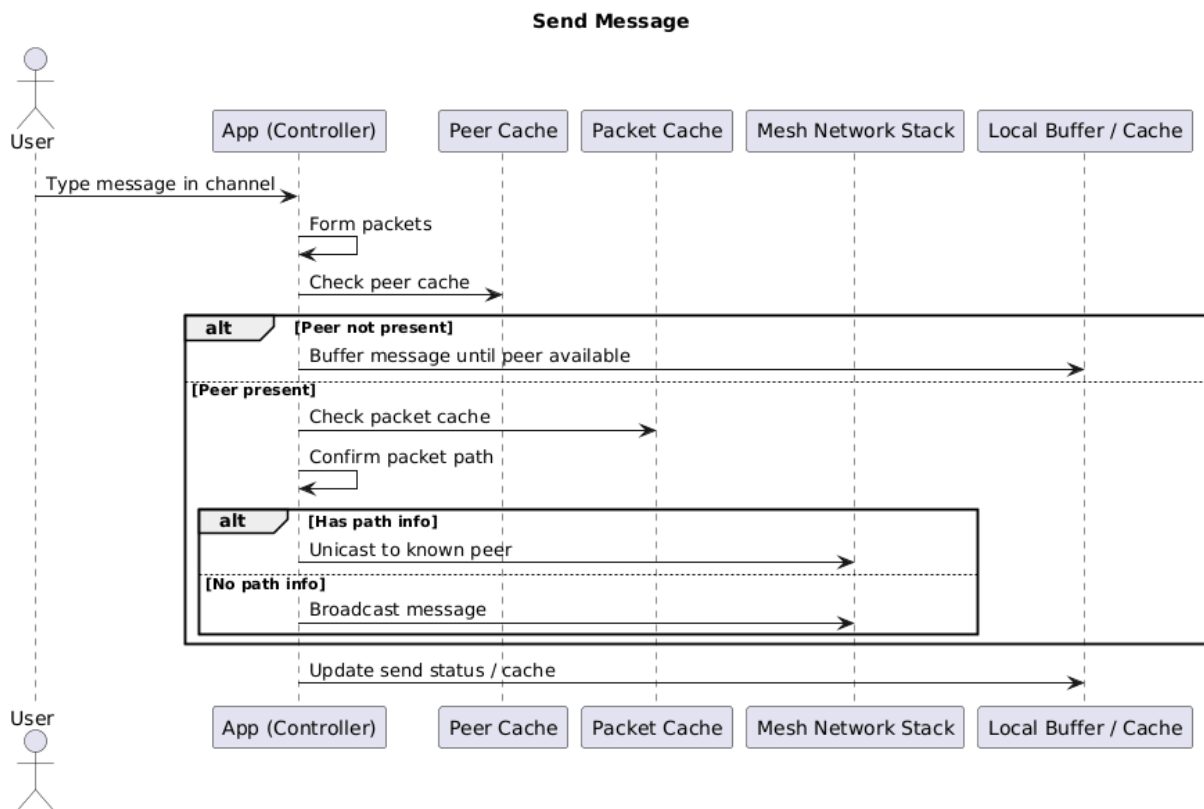


Figure 20: Sequence diagram: Send message

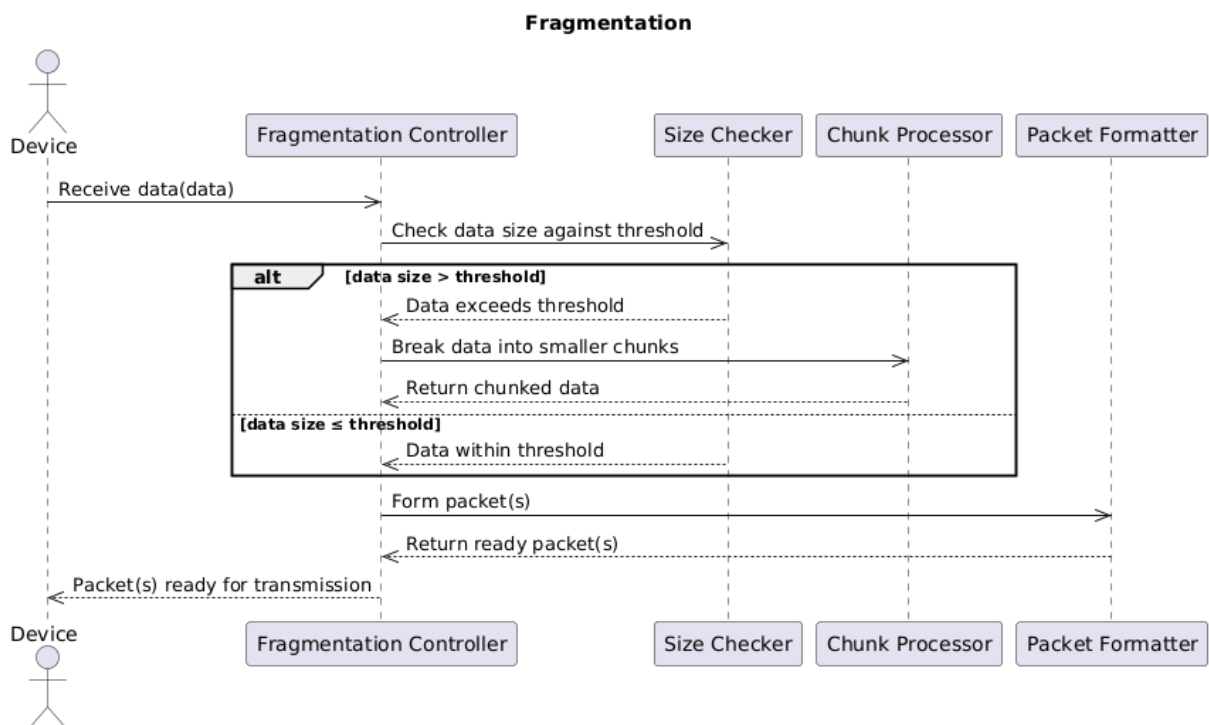


Figure 21: Sequence diagram: Fragmentation message

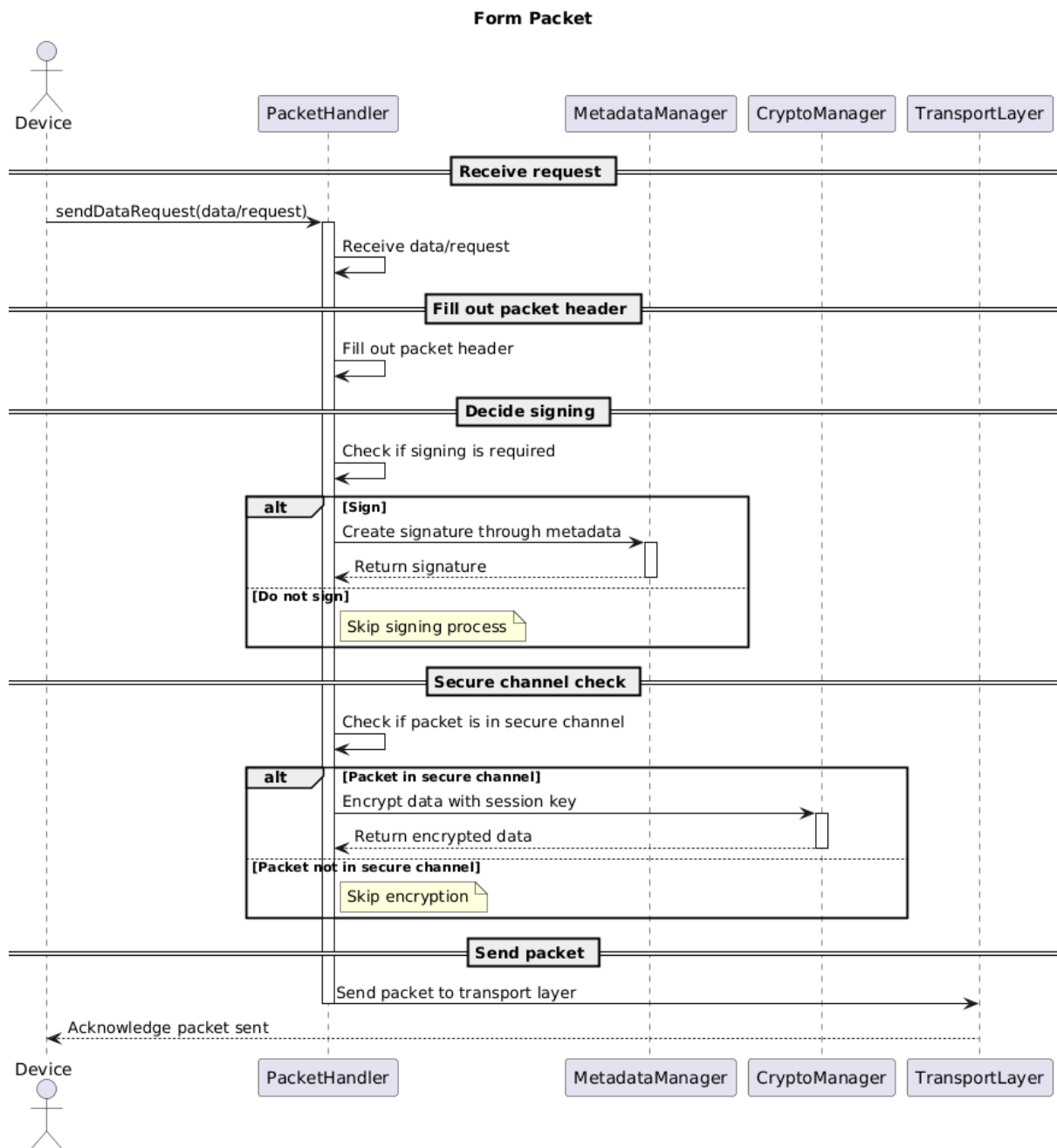


Figure 22: Sequence diagram: Form packet

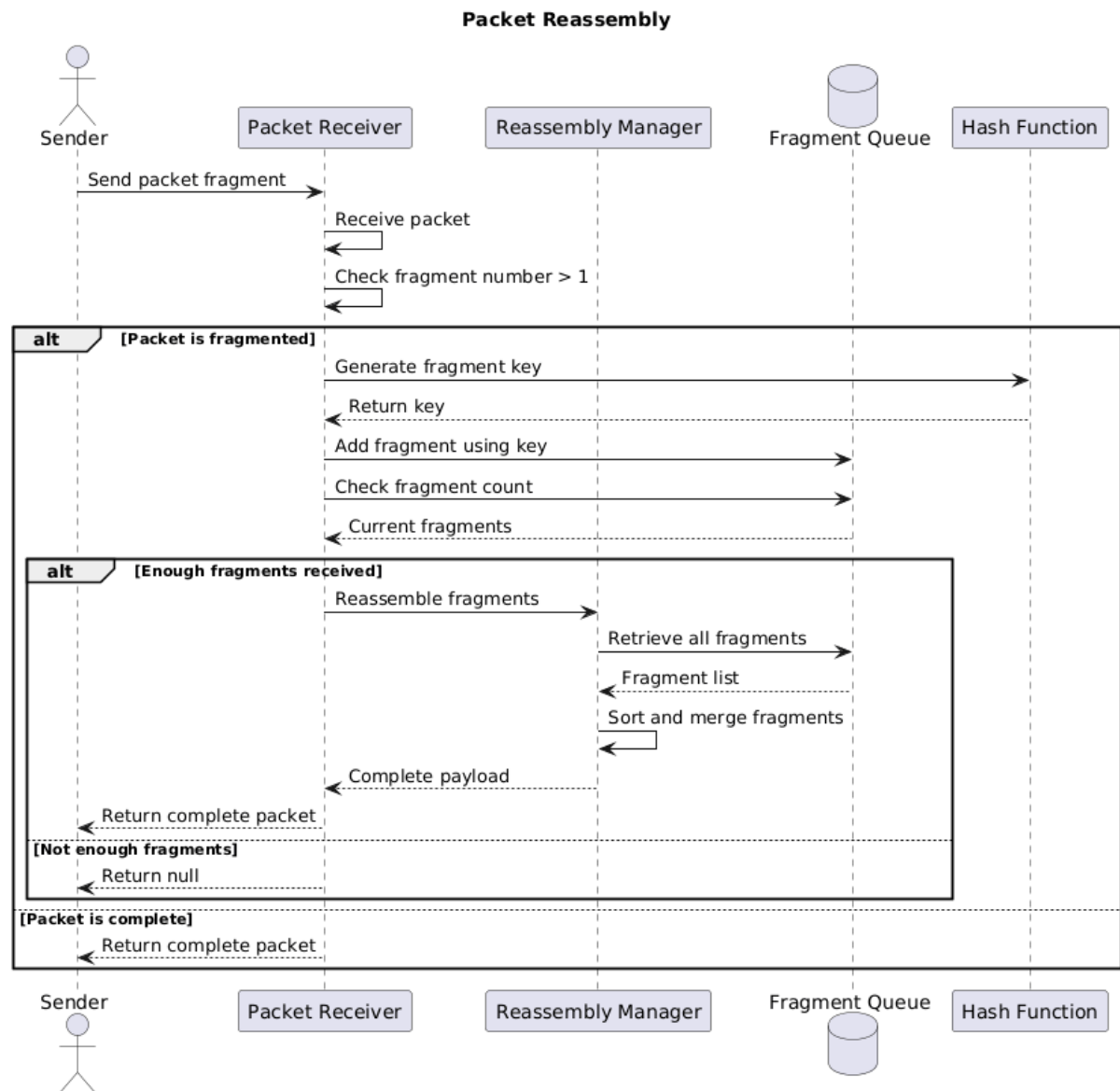


Figure 23: Sequence diagram: Reassembly

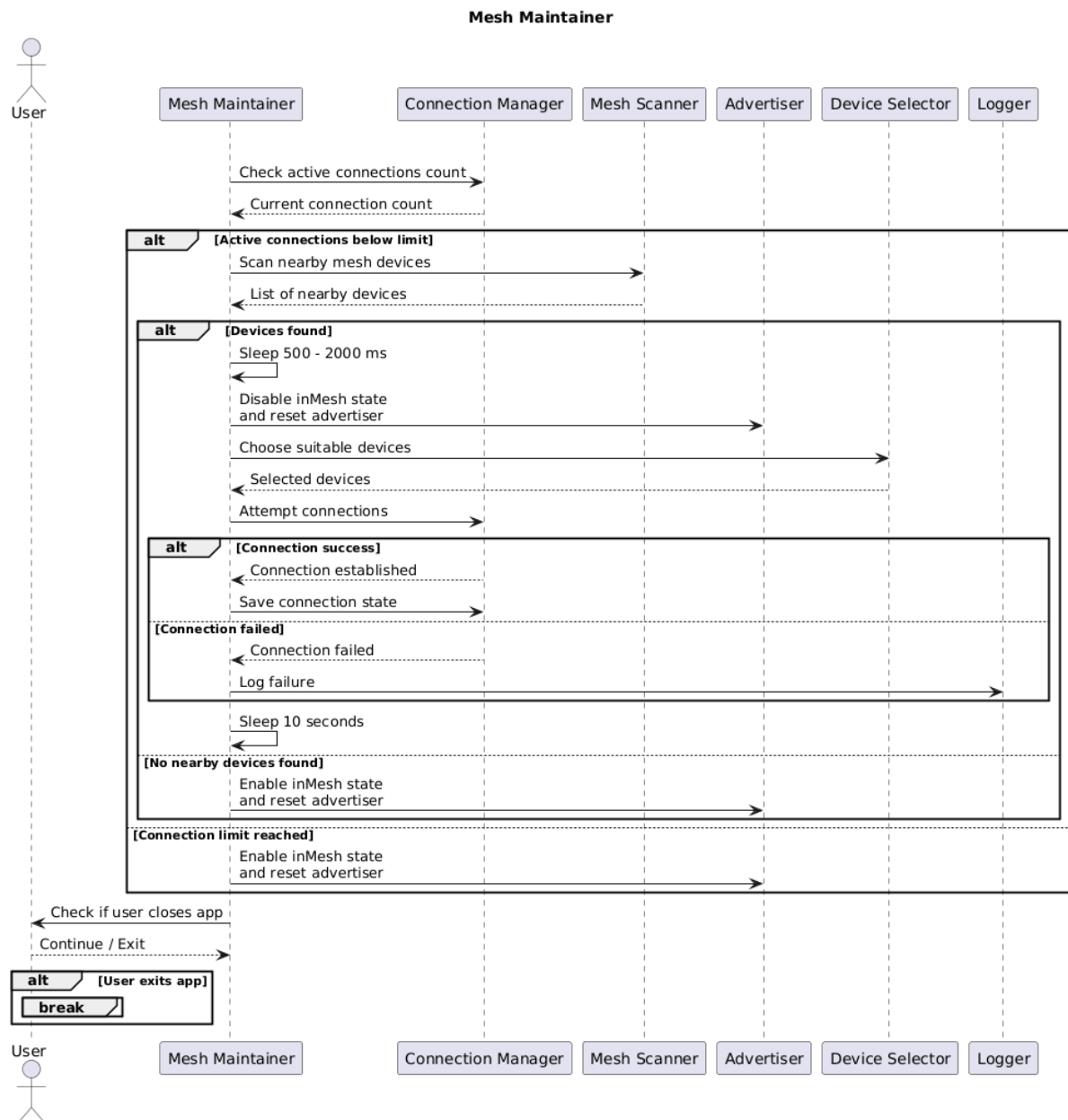


Figure 24: Sequence diagram: Mesh maintainer

6 System Design

6.1 System Architecture

6.1.1 Comparison of Architecture Styles and Architecture Decision

Since this project focuses on developing a mobile messaging application over a Bluetooth mesh network, only monolithic architectural styles are considered. This choice is motivated by the constraints of mobile environments, including limited resources, offline operation, and the need for tight integration between networking, security, and application logic. Table 10 compares three common monolithic architecture styles and evaluates their suitability for the Meshenger system.

Pattern	Pros	Cons	Meshenger Suitability
Layered	Clear separation of concerns across UI, business logic, and data; easy collaboration for teams with different specializations; widely adopted and well understood; strong modularity and maintainability.	Dependencies between layers can become rigid; performance overhead appears if layering is overly strict; some changes ripple across multiple layers.	Highly suitable (recommended). Core services such as networking, security, backend, and frontend can be structured into layers, improving clarity, maintainability, and testability.
Pipeline	Effective for data processing and transformation stages; stages are reusable and can run in parallel.	Error handling becomes more complex; the model adds coordination overhead for non-streaming and stateful tasks.	Partially suitable. Useful for packet transformation workflows, but less effective for interactive and stateful components such as peer tables and message queues.
Microkernel	Lightweight and stable core; flexible through plug-ins and extensions; suitable for frequent feature updates.	Plug-in contracts are complex; indirection introduces overhead; it is difficult to apply to tightly coupled services.	Moderately suitable. Good for extensibility, but unnecessarily complex for core services where simplicity and stability are prioritized.

Table 10: *Comparison of Monolithic Architecture Styles*

Based on the comparison, the layered architecture emerges as the most suitable choice

for this project. In the context of Meshenger, each layer is designed as a collection of loosely coupled yet highly cohesive modules, ensuring a clear separation of responsibilities. This approach enhances maintainability and scalability while simplifying testing because individual modules can be validated independently without affecting other layers.

6.1.2 Dual-Write Architecture for Bidirectional Communication

To support bidirectional data exchange between two physical peers over Bluetooth Low Energy (BLE), this project evaluated two primary architectural patterns. Although BLE is natively based on a client–server GATT model, achieving practical full-duplex communication requires an explicit strategy for handling traffic in both directions. Figure 25 illustrates the symmetrical dual-write pattern adopted in the final design.

Option 1: Asymmetrical Link (Write/Notify)

This approach uses a single BLE connection in which one device acts as the GATT client and the other acts as the GATT server. Data is transmitted from the client to the server through a writable characteristic, while the return path from the server to the client is implemented through notifications.

Advantages. This pattern is resource-efficient because it maintains only one active BLE link, thereby reducing power consumption, radio contention, and CPU overhead.

Disadvantages. Its practical deployment is constrained by software fragmentation. Reliable behavior for notification descriptors and more advanced MTU negotiation is more consistent on newer Android releases, which reduces backward compatibility for older devices and narrows the reachable user base.

Option 2: Symmetrical Dual-Write Architecture

The dual-write method allows both devices to operate simultaneously as GATT clients and GATT servers. Each peer establishes an independent connection to the other peer’s server, so Device A connects to Device B while Device B also connects to Device A. As a result, two logical BLE links are maintained between the same pair of physical peers, and both traffic directions rely on write operations.

Advantages. This pattern depends only on characteristic writes, which are among the most stable and widely supported BLE features on Android. By avoiding reliance on notification descriptors, the protocol remains compatible with older Android versions and improves accessibility across a wider range of hardware.

Disadvantages. Maintaining two concurrent connections for each peer pair increases resource utilization, especially in terms of radio-frequency time sharing and memory overhead for connection management.

Implementation Decision

For this project, the dual-write method was selected. Despite its higher resource cost, priority was given to network inclusivity and backward compatibility. This decision helps ensure that the mesh remains functional across diverse hardware profiles, including older Android devices that may not reliably support newer BLE notification behavior.

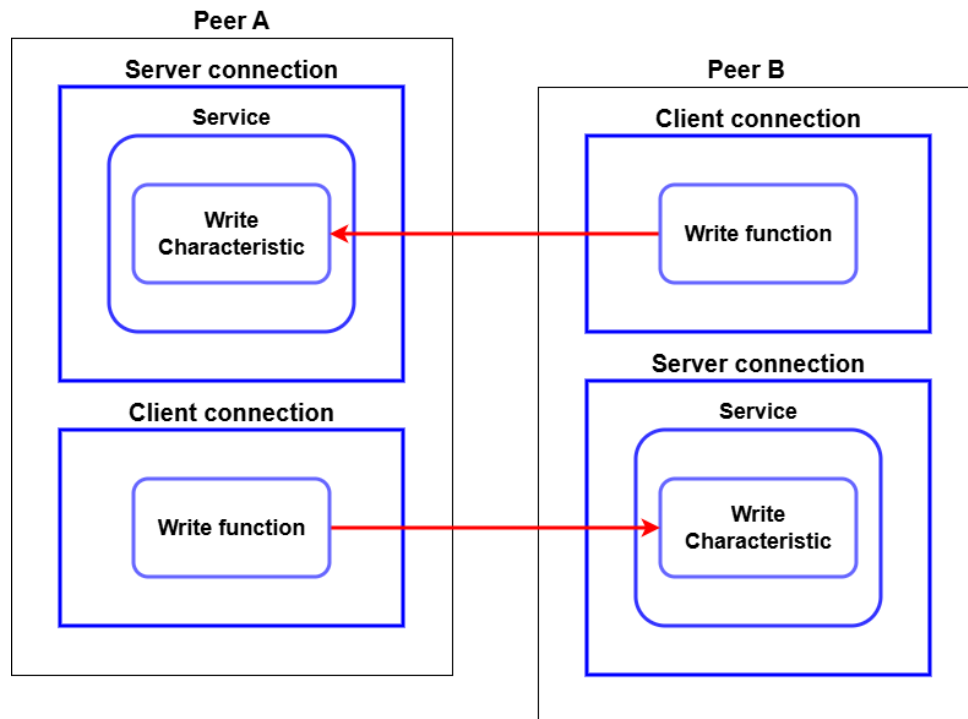


Figure 25: *Dual-write architecture for bidirectional BLE communication between two peers*

6.1.3 Overall Architecture

The architecture of the application is structured into layered components that separate user interaction, session management, mesh routing, and Bluetooth transport responsibilities. The figure 26 illustrates our application overall architecture.

- **UI/UX (Purple):** The UI/UX layer is responsible for rendering results and capturing user input. Detailed design specifications are available in Section 6.4.
- **Application (Red):** The Application Layer is the native façade between the React Native UI and the Kotlin backend. It exposes a Native Module bridge so JavaScript can invoke typed operations (peers, profiles, messaging, global chat, mesh scan, and two-party sessions) and subscribe to device events for asynchronous updates (new messages, handshake callbacks, profile changes). Internally it orchestrates work: it validates inputs, forwards mesh/session traffic to the Session and Network stacks, and persists durable application state. Persistence is centralized in a SQLite helper

(MeshengerDbHelper) accessed through singleton stores—a Messaging Store for out-bound/inbound ciphertext rows and conversation reads, and a User Store for the local profile and peer directory—so UI-oriented code stays decoupled from raw SQL. Where applicable, optional peer key-wrapping logic can persist encrypted key blobs alongside Android Keystore aliases, keeping long-lived secrets out of plaintext storage. This layer therefore owns cross-cutting application concerns (bridge API, local database, and delegation into lower layers) while leaving routing, epidemic flooding, and BLE mechanics to the Network and Transport tiers below.

- **Session Layer (Blue):** The Session Layer manages end-to-end communication channels between users. It is divided into two primary sub-components:
 - **Global Session:** Responsible for broadcast-style communication across the entire mesh. This supports the *Global Chat* feature and initial peer discovery (bootstrapping). It employs symmetric key encryption to balance security with the high-speed requirements of multi-peer broadcasting.
 - **Two-Party Session:** Dedicated to secure, point-to-point messaging between two specific users. A unique session key is generated for every new interaction to provide forward secrecy.
- **Network Layer (Green):** This layer handles packet routing, reliability, and data integrity:
 - **Epidemic Flooding:** The core routing engine that manages data propagation. In addition to standard flooding, it implements a store-and-forward mechanism to buffer data during mesh instability and deliver messages once a reliable path is restored.
 - **Packet Cache:** A local repository that tracks processed packets. This prevents replay attacks and reduces resource usage by avoiding duplicate processing.
 - **Reassembly Queue:** A temporary staging area for fragmented data. It monitors incoming packet fragments and reconstructs the original payload once all pieces are received.
 - **Packet Operations:** The low-level data handler responsible for packet creation, verification, fragmentation, encoding, and decoding.
- **Transport Layer (Yellow):** The Transport Layer abstracts the Bluetooth Low Energy (BLE) radio and manages peer connections:
 - **Mesh Maintainer:** The autonomous controller for network stability. It supports discovery, mesh integration, and self-healing by triggering reconnection

logic when peer density falls below a predefined threshold.

- **Mesh Connection Registry:** A centralized database that tracks active inbound and outbound connections in real time.
- **BLE Advertiser & Scanner:** The advertiser broadcasts the device's presence to the mesh, while the scanner identifies other compatible peers.
- **BLE Server:** Listens for and acknowledges incoming connection requests.
- **BLE Client & Server Connections:** The client connection initiates handshakes and pushes outbound data, while the server connection manages reception of data pushed from other peers.

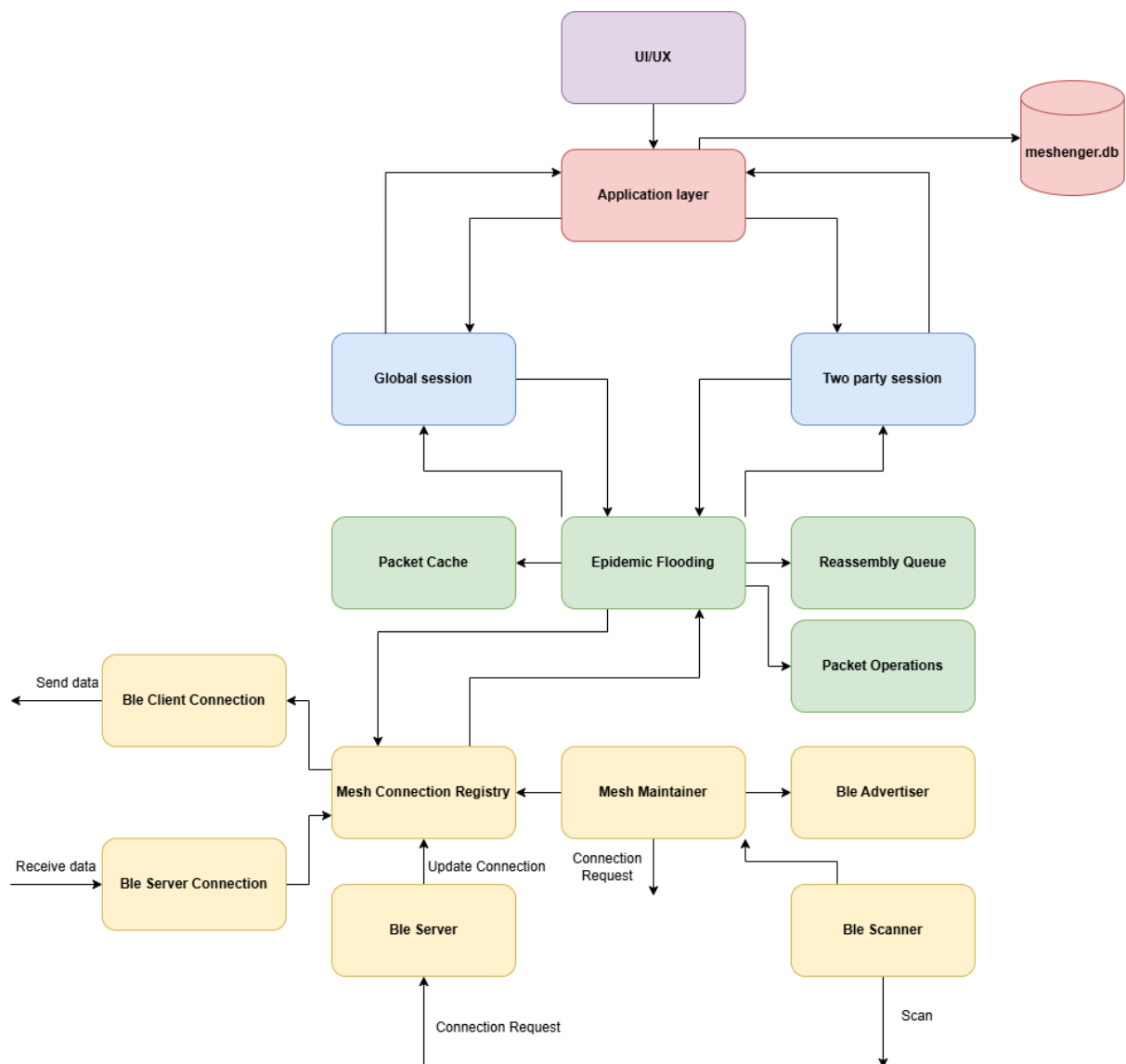


Figure 26: *Applications overall architecture*

6.2 Component Diagram and Class Diagram

6.2.1 Component Diagram

This section presents a component-level view of the communication stack, describing how plaintext messages are transformed into encrypted payloads (session layer), disseminated across the mesh (network layer), and delivered over Bluetooth Low Energy (transport layer). Each diagram highlights the main processing modules, the state they maintain, and the interfaces between them.

Session Layer

Figure 27 shows how the system establishes secure sessions and provides a uniform encryption/decryption interface to the upper layers.

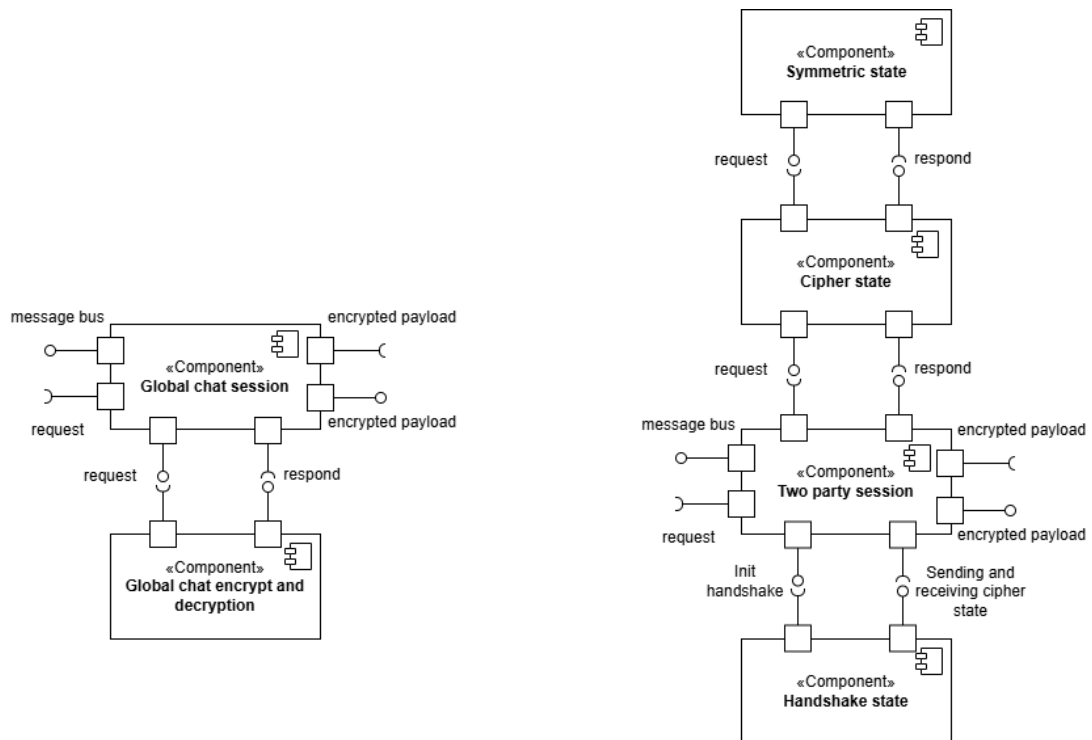


Figure 27: Session layer components for global chat and two-party sessions, including handshake and cipher state management

The session layer supports two modes of operation:

- **Global chat session:** A lightweight session intended for mesh-wide (broadcast) communication. Plaintext messages arrive through the message bus and are processed by **Global chat encrypt and decryption**, which exposes a request/response API and outputs an *encrypted payload* to the lower layers.

- **Two-party session:** A point-to-point session that maintains per-peer cryptographic context. Session establishment starts in **Handshake state** (*init handshake*) and transitions to **Cipher state** for steady-state message protection. Shared key material is negotiated and refreshed through **Symmetric state** used by the cipher state.

In both cases, the session layer hides cryptographic details from the rest of the system: upper layers submit plaintext, while lower layers only handle the resulting *encrypted payload*.

Network Layer – Epidemic Routing

Figure 28 depicts an epidemic (store-carry-forward) routing pipeline designed for intermittently connected meshes. Nodes exchange metadata upon contact and opportunistically forward packets that the peer does not yet possess.

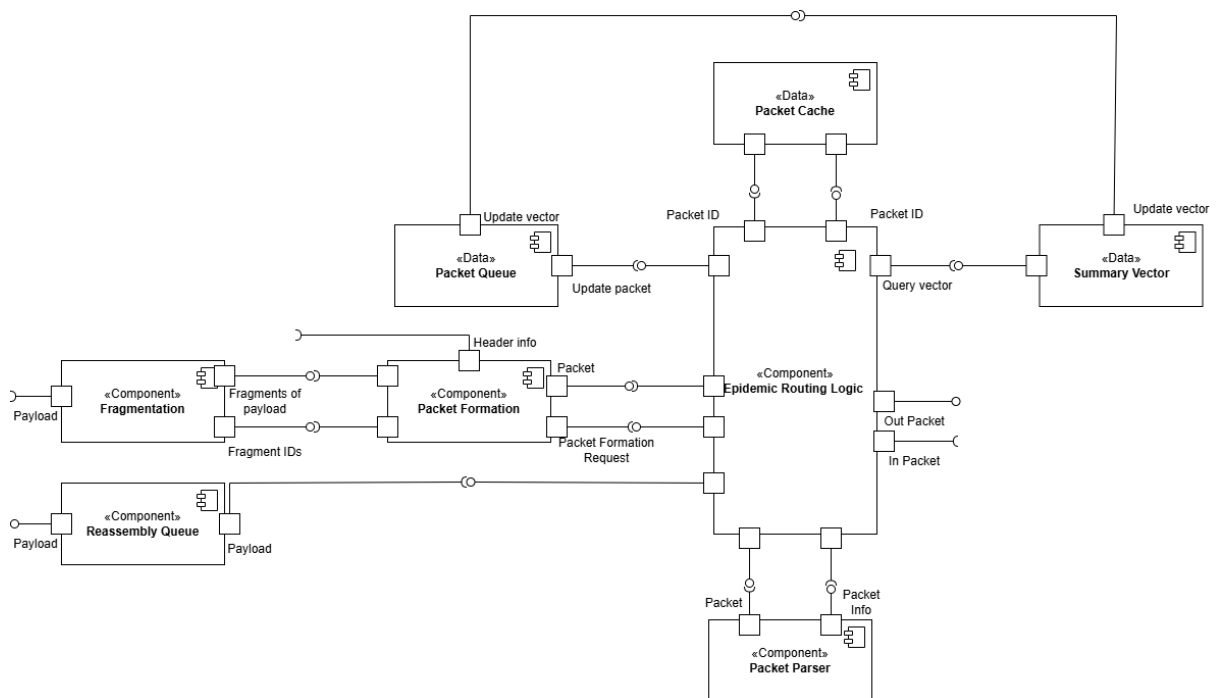


Figure 28: Epidemic routing component diagram showing packet exchange via summary vectors and duplicate suppression via packet caching

The Epidemic Routing Logic maintains a **Summary Vector**—a compact representation of locally stored packet IDs. During a contact opportunity, the node queries the neighbour’s vector, computes the set difference, and decides which packets to request or forward. To avoid redundant transmissions and repeated processing, the **Packet Cache** tracks recently observed packet IDs.

Outbound messages enter the network layer as a payload that is split by fragmentation

and encoded by packet formation (header creation and fragment IDs). The generated packets are buffered in the epidemic packet queue until a suitable forwarding opportunity arises; enqueue/dequeue operations trigger corresponding updates to the summary vector. For inbound traffic, the **Packet Parser** extracts identifiers and metadata. After cache validation, fragments are passed to **Reassembly Queue** to reconstruct the original content.

Transport Layer

Figure 29 shows how BLE discovery and connection management are organized to support a self-healing mesh.

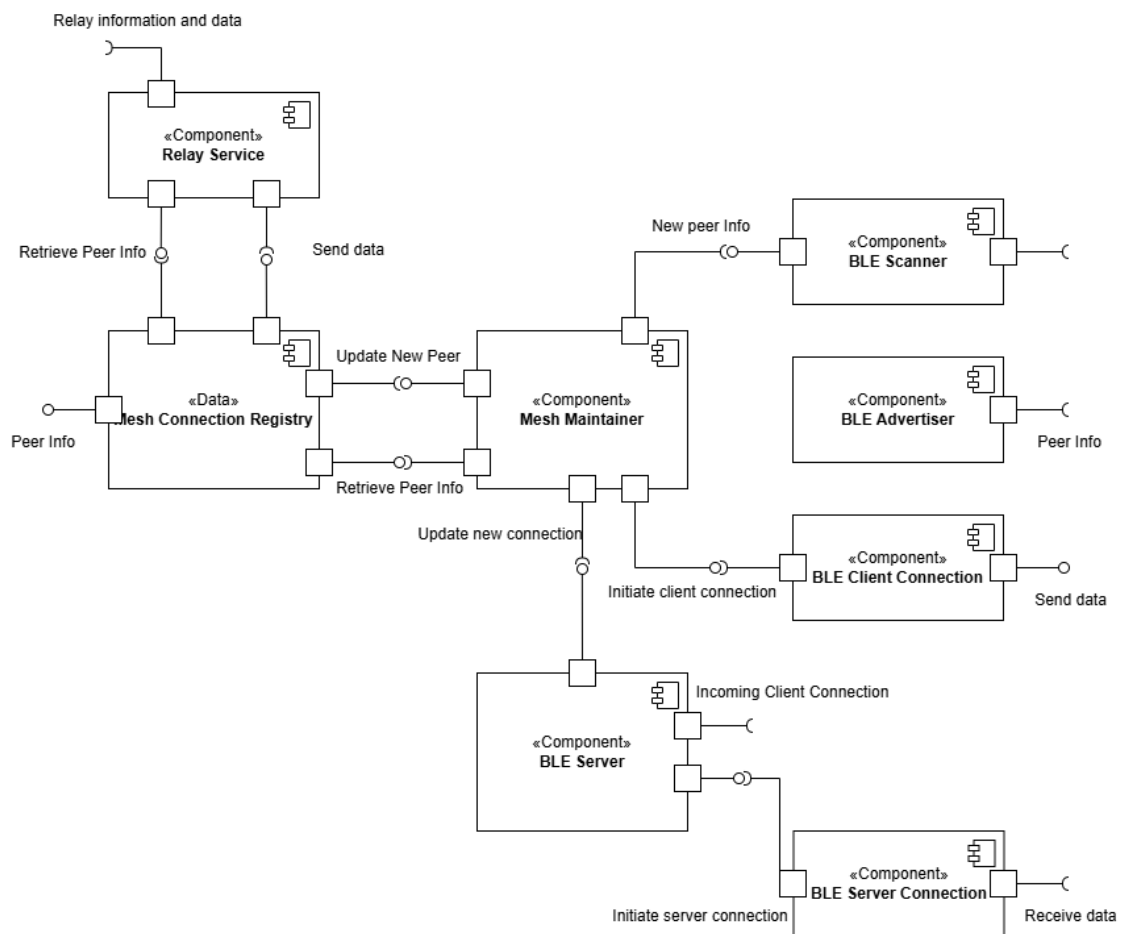


Figure 29: Transport layer components for peer discovery, connection setup, and relay data exchange over BLE

The **Mesh Maintainer** orchestrates transport operations. It consumes *new peer information* produced by **BLE Scanner** and records discovered peers in the **Mesh Connection Registry**. In parallel, **BLE Advertiser** broadcasts the local node's *peer information* to make it discoverable.

To create links, the maintainer initiates outbound sessions via **BLE Client Connection** and accepts inbound sessions through **BLE Server**, which spawns a **BLE Server Connection** for data reception. The registry provides centralized peer lookup and stores updates about newly established connections, enabling reconnection and link selection decisions.

Finally, the **Relay Service** bridges routing decisions and transport delivery by retrieving peer details from the registry and performing *send data* actions for packet forwarding and control information dissemination.

6.2.2 Class Diagram

The class diagram presents a static view of the system by listing the main classes, their key attributes and operations, and the relationships that connect them (associations, compositions, and dependencies).

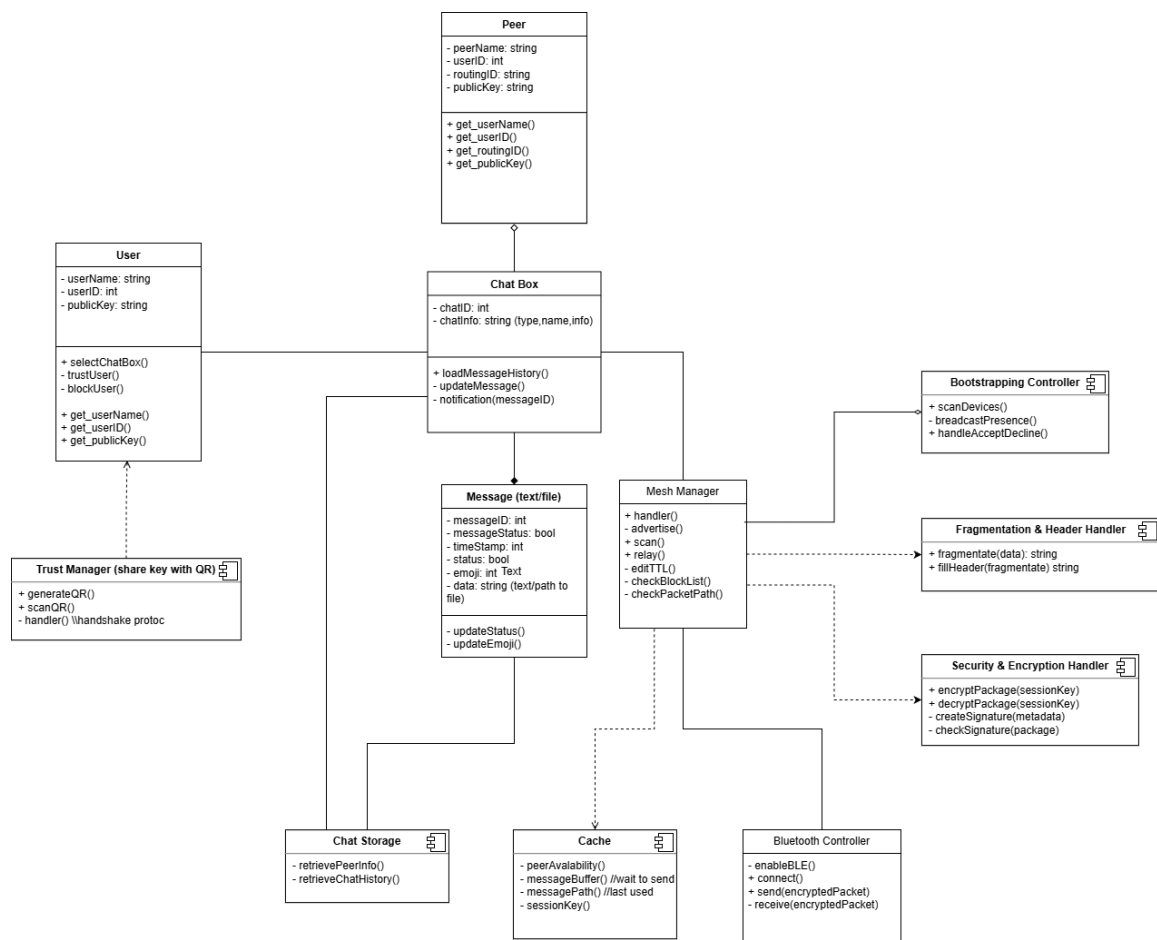


Figure 30: System class diagram highlighting the relationships between core controllers and data entities

Figure 30 consolidates the roles introduced in the sequence diagrams into a single architectural model. The most important responsibilities can be grouped as follows:

- **UI layer:** user-facing classes (e.g., `User`, `ChatBox`) that capture interactions and input events.
- **Mesh control:** `MeshManager` coordinates overall mesh behaviour and works with the `BootstrappingController` to discover and connect to peers.
- **Message processing:** `MessageProcessor` prepares data for transmission and delegates specialized tasks to the security and fragmentation components.
- **Persistence:** `ChatStorage` and `Cache` provide local storage for messages and routing-related information to support offline operation.

Session Layer

The session layer provides the system’s **secure communication context**. It is responsible for creating, maintaining, and terminating a **Session** between peers, while hiding cryptographic details from upper layers. In practice, it standardizes the session lifecycle (initialization, membership updates, and packet/message processing) and encapsulates all state required for end-to-end protection.

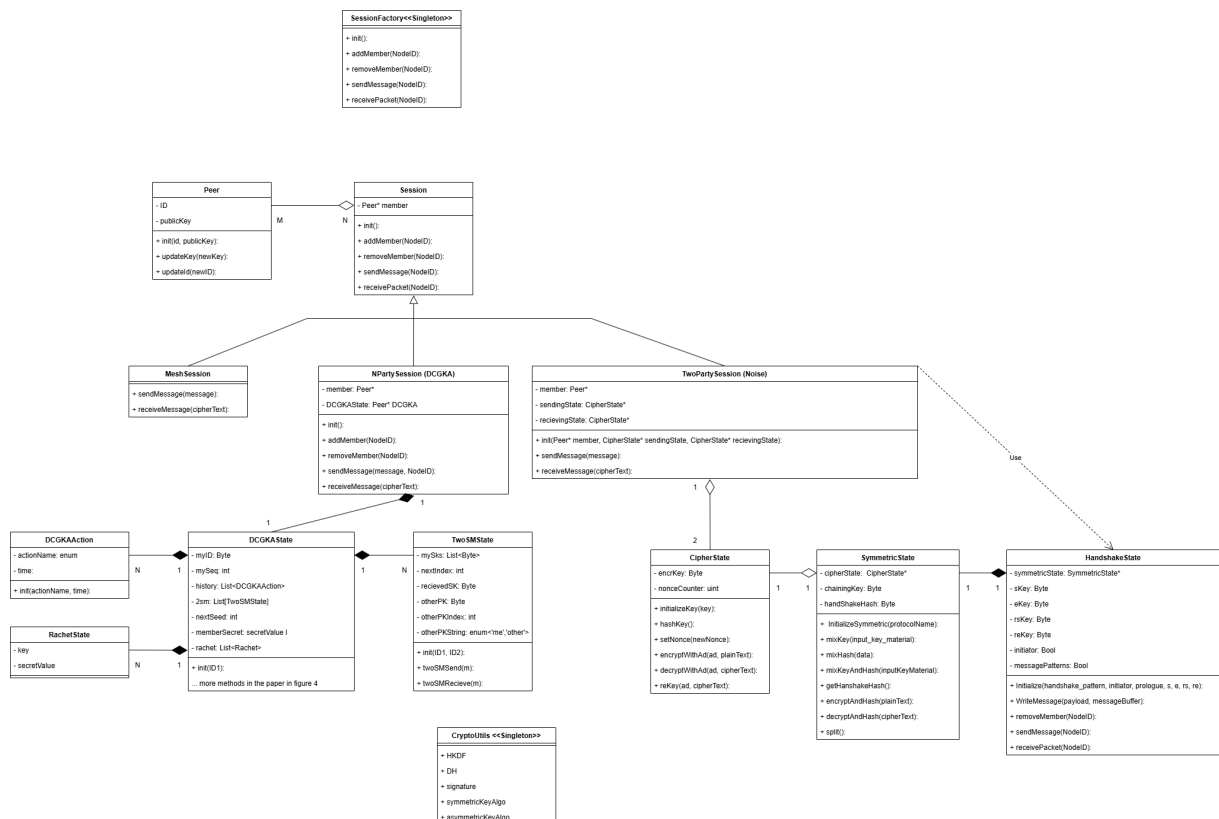


Figure 31: *Session layer class diagram*

Figure 31 shows `SessionFactory` (singleton) as the main entry point for creating and

managing sessions for a set of **Peer** instances. The abstract **Session** interface offers common operations such as **addMember**, **removeMember**, **sendMessage**, and **receivePacket**. Two concrete variants are modeled: **TwoPartySession** (Noise) for pairwise channels and **NPartySession** (DCGKA) for group communication with dynamic membership. The underlying security is represented explicitly through state objects—**HandshakeState**, **SymmetricState**, and **CipherState** for the Noise handshake, and **DCGKASession** (with **TwoSMState** history/ratchet material) for group key agreement and rekeying.

Network Layer - Epidemic Routing

The epidemic routing design follows a *store-carry-forward* model in which nodes opportunistically exchange packets whenever a contact becomes available. Instead of maintaining fixed end-to-end paths, each node keeps local knowledge about what it has already seen and uses an anti-entropy exchange to reduce duplicates while still maximizing delivery probability under intermittent connectivity.

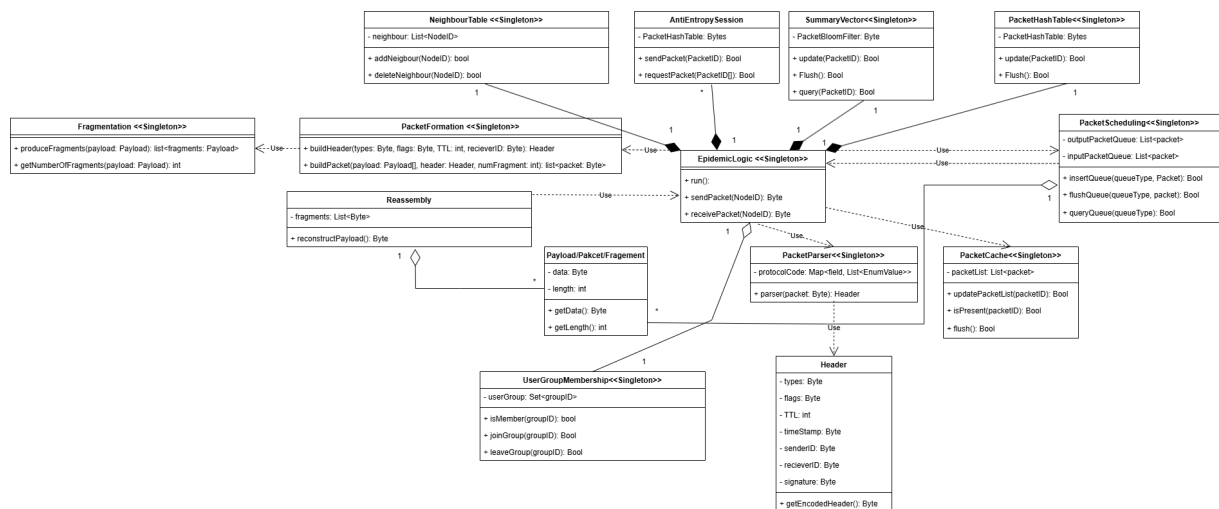


Figure 32: Network Layer Class Diagram for Epidemic Routing

Figure 32 places **EpidemicLogic** (singleton) at the center of the network layer. It interacts with **NeighbourTable** to track reachable peers and coordinates an **AntiEntropySession** when exchanging state with a neighbour. To summarize local packet knowledge efficiently, the design provides compact metadata structures such as **SummaryVector** (e.g., bloom-filter-like representation) and a **PacketHashTable** for packet identifiers; **PacketCache** additionally suppresses duplicates and prevents repeated processing.

For data-plane processing, outbound payloads are converted to packets by **PacketFormation** (header creation and packet construction) and may be split by

Fragmentation into multiple fragments. Incoming traffic is interpreted by **PacketParser**, which extracts a **Header** and packet metadata before passing fragments to **Reassembly** to reconstruct the original **Payload/Package/Fragment**. **PacketScheduling** manages input/output queues and prioritizes which packets are transmitted or requested during a contact, enabling epidemic dissemination while respecting bandwidth and buffer constraints.

Transport Layer

The transport layer is responsible for peer discovery, connection establishment, and reliable packet delivery over Bluetooth Low Energy (BLE). It provides the link abstraction used by the network layer by managing scanning/advertising, maintaining connection state for both inbound and outbound links, and exposing send/receive operations to upper layers.

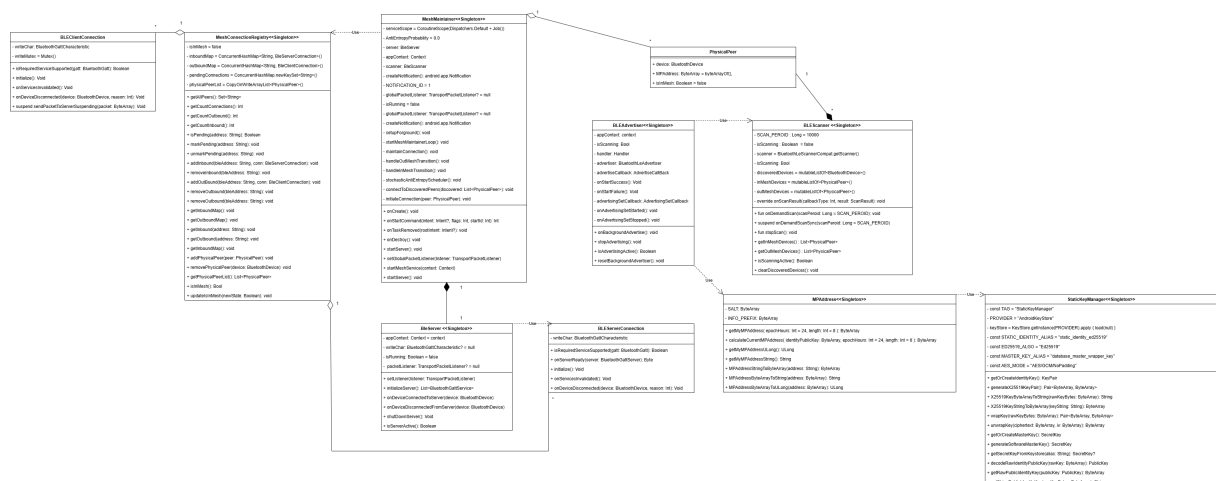


Figure 33: Transport Layer Class Diagram

As shown in Figure 33, **MeshMaintainer** orchestrates the BLE workflow by coordinating **BLEScanner** and **BLEAdvertiser** (singletons) to discover nearby devices and broadcast the local presence. Discovered devices are represented as **PhysicalPeer** entries and recorded in the **MeshConnectionRegistry**, which tracks pending and active connections as well as inbound and outbound maps. For data transfer, the maintainer relies on **BLEClientConnection** for outbound GATT links and on **BLEServer** to accept inbound connections, creating **BLEServerConnection** instances for reception. The diagram also shows utility components such as **MPAddress** and **StaticKeyManager** for address/key material used during secure identification and connection setup.

Diagrams

To facilitate enhanced readability and future modifications, the editable source files for the architectural diagrams presented in this section are hosted on diagrams.net. These vector files provide granular detail and are accessible via the following links:

- **Class Diagram Repository:** [View Class Diagram](#)
- **Component Diagram Repository:** [View Component Diagram](#)

6.3 Database Design

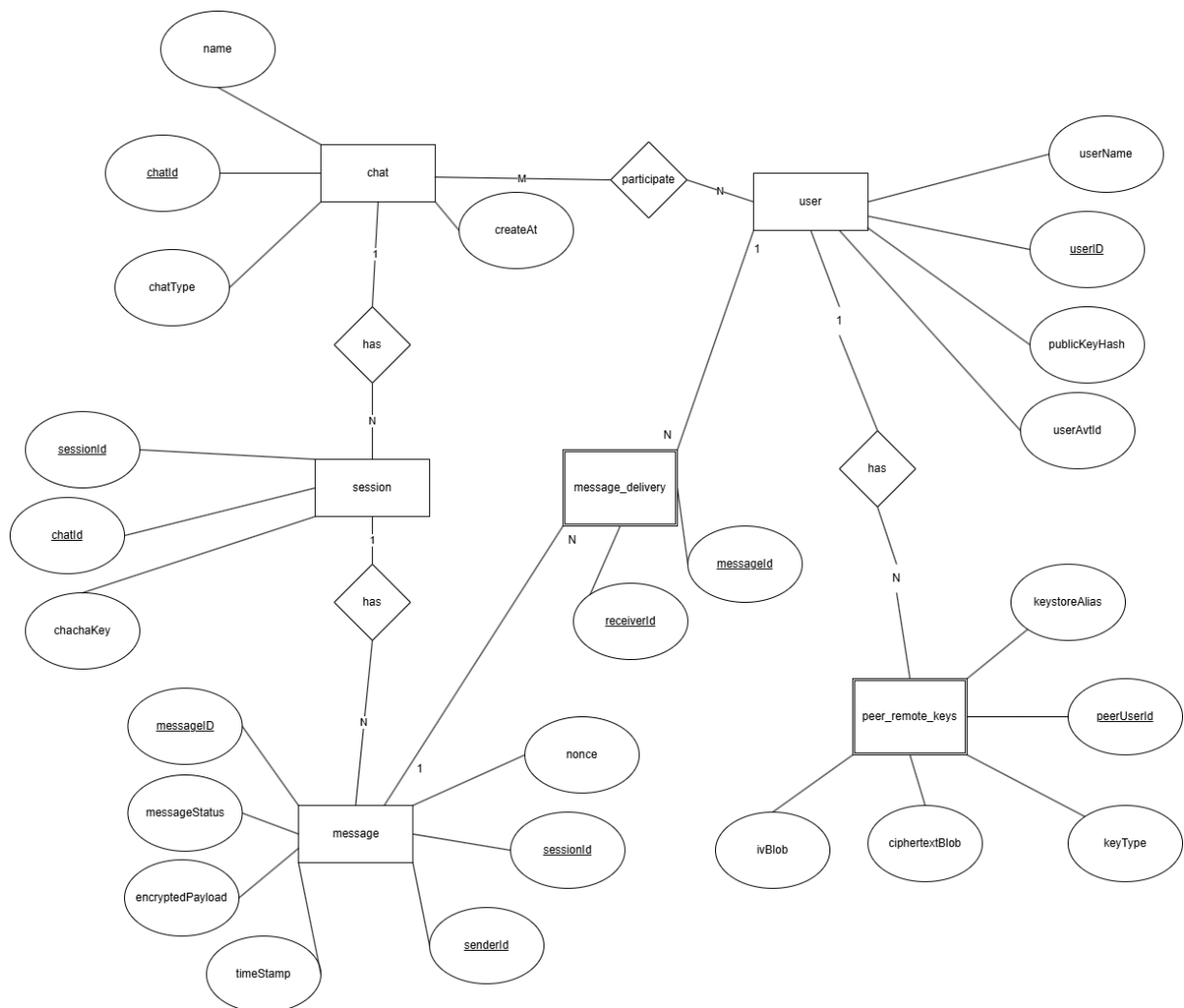


Figure 34: *Enhanced Entity-Relationship Diagram*

The Entity Relationship Diagram (ERD) illustrates the database structure of a secure messaging application. The core entities include *user*, *chat*, *session*, *message*, *message_delivery*, and *peer_remote_keys*. A user can participate in multiple chats through

the *participate* relationship, while each chat may contain many users, representing a many-to-many association. Each chat also manages multiple communication sessions, and every session stores cryptographic information such as the *chachaKey* used for secure message encryption. The database is designed to support both group and private conversations while maintaining clear relationships between users, chats, and encrypted communication sessions.

The *message* entity stores encrypted message data, including the encrypted payload, timestamp, sender information, nonce, and message status. Each message belongs to a session and can be delivered to multiple receivers through the *message_delivery* entity, enabling message tracking and delivery management. Additionally, the *peer_remote_keys* entity manages remote cryptographic keys associated with users, including key aliases, initialization vectors, ciphertext blobs, and key types. This structure ensures secure key storage and supports end-to-end encrypted communication within the system.

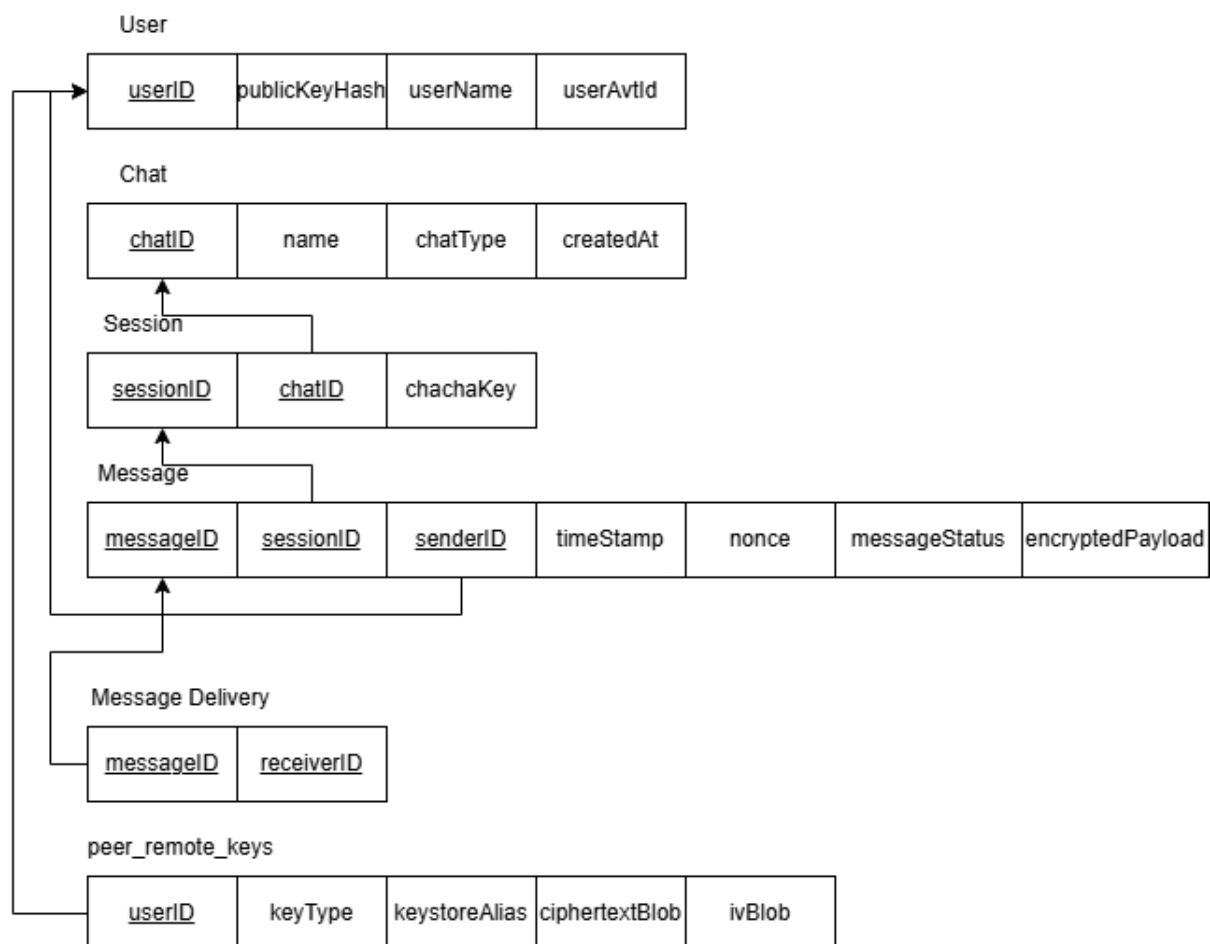


Figure 35: Relational Entity Diagram

The relational schema represents the database structure of the secure messaging system by defining tables, attributes, and relationships between entities. The *User* table stores

user information such as unique identifiers, usernames, avatar identifiers, and public key hashes for authentication and encryption purposes. The *Chat* table manages conversation information, including chat names, chat types, and creation timestamps. Each chat can contain multiple communication sessions through the *Session* table, which stores encrypted session keys used for secure communication.

The *Message* table is the core component responsible for storing encrypted messages, sender information, timestamps, nonces, message statuses, and encrypted payloads. Message delivery tracking is handled by the *Message Delivery* table, which connects messages to their receivers and supports multi-user communication. In addition, the *peer_remote_keys* table stores remote cryptographic key information, including key aliases, ciphertext blobs, initialization vectors, and key types. Overall, the relational schema ensures organized data management, secure key storage, and efficient handling of encrypted messaging operations within the system.

6.4 User Interface Design

6.4.1 UI/UX Flow

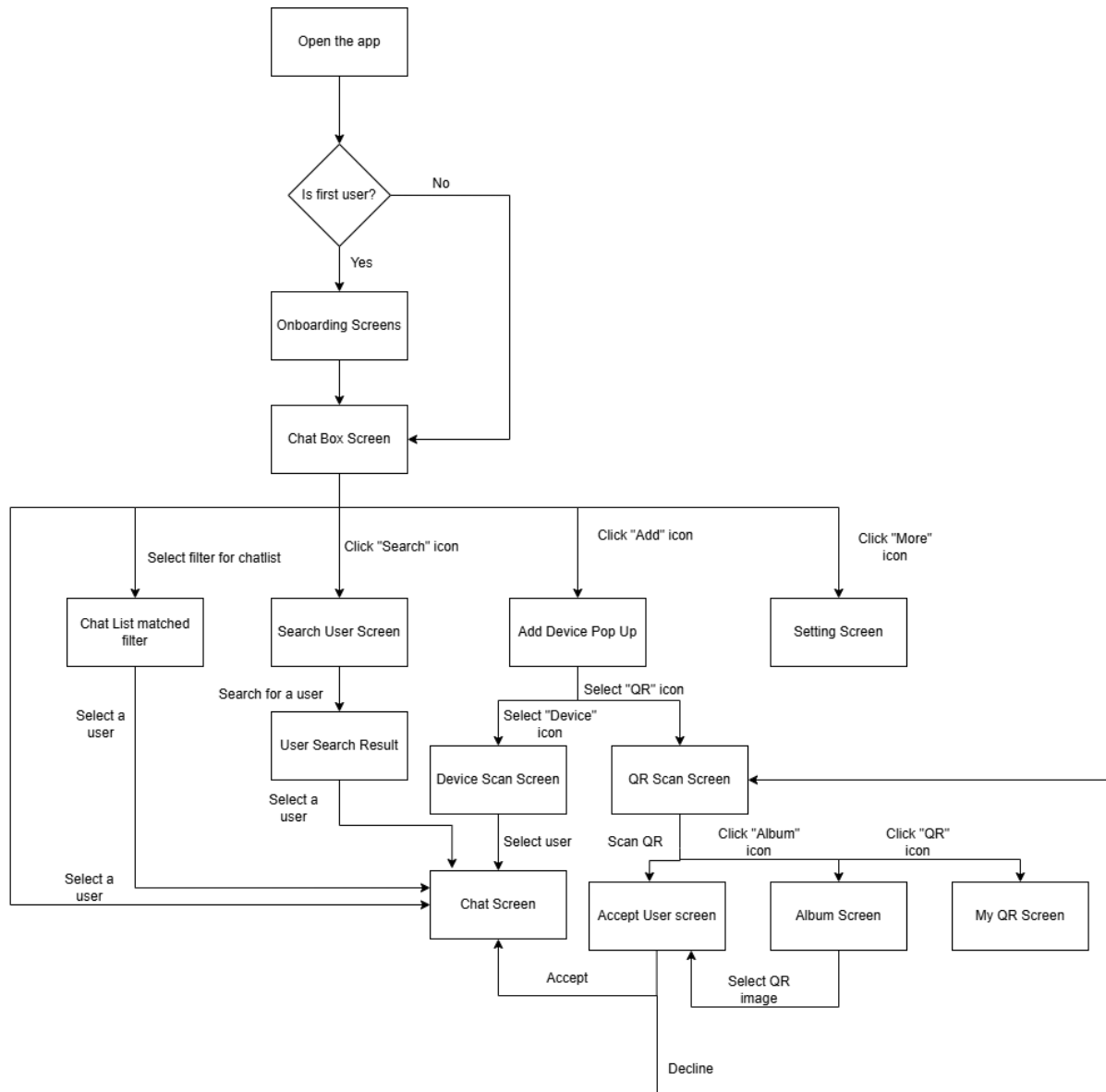


Figure 36: *Meshenger UI/UX Flow*

The Meshenger UI Flow illustrates how users interact with the Android application to access core features such as connecting to other users' devices through device scanning and QR code scanning methods, searching for users, and chatting. The flow is designed to provide a simple and intuitive user experience by guiding users through onboarding, connection setup, and communication processes efficiently.

6.4.2 Onboarding Screens

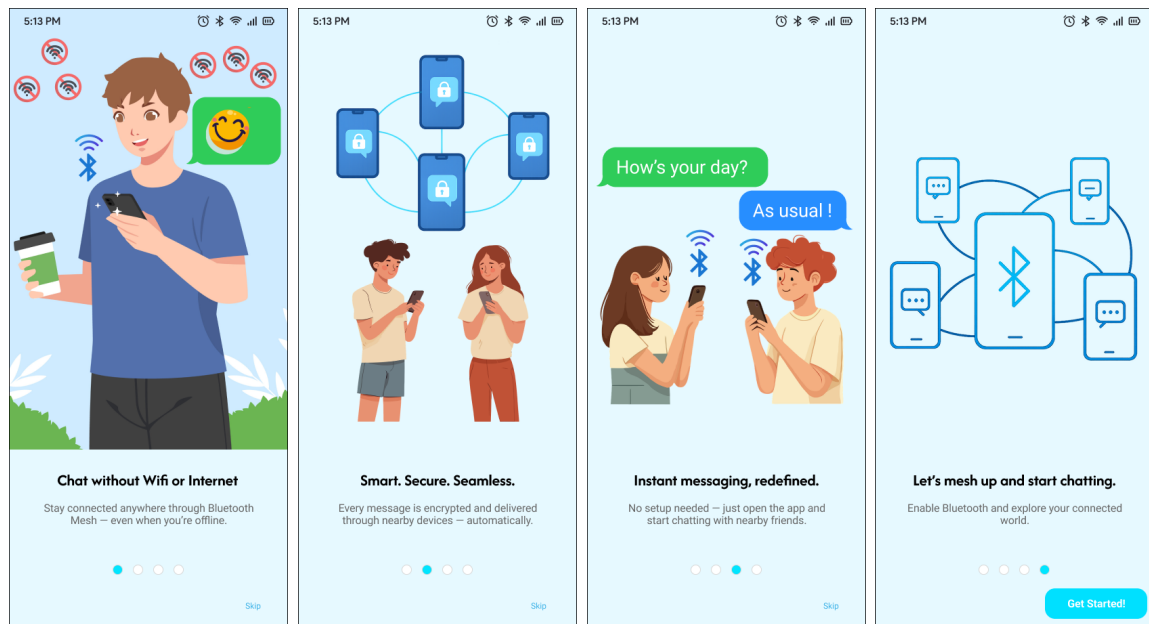


Figure 37: *Four onboarding screens of the chat app*

When the user completes installation of the Android application, the system displays four onboarding screens only once to introduce the chat application's services and features.

After the fourth onboarding screen, the "Get Started" button appears at the bottom-right corner, and the user is required to click it to navigate to the user profile editing section.

6.4.3 User Profile

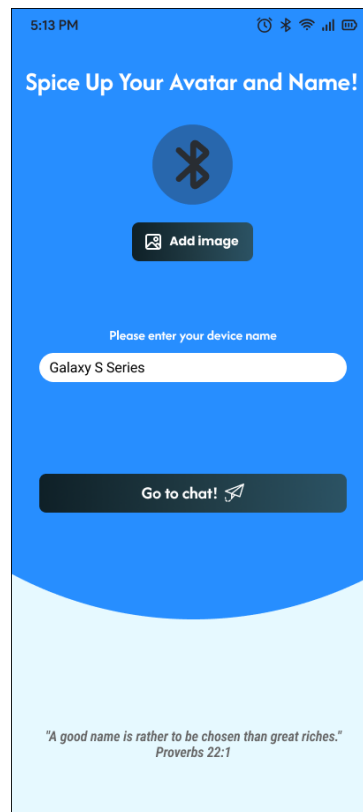


Figure 38: *Edit Profile Screen*

Before navigating to the user profile section, the system renders the user profile section, which includes a profile image and a username input field for profile initialization. Profile initialization is optional. If the user leaves the fields unchanged and selects “Go to Chat”, the system saves the default profile image and the device name as the username in the local database. Otherwise, the system saves the customized profile information.

6.4.4 Chat Box Screens

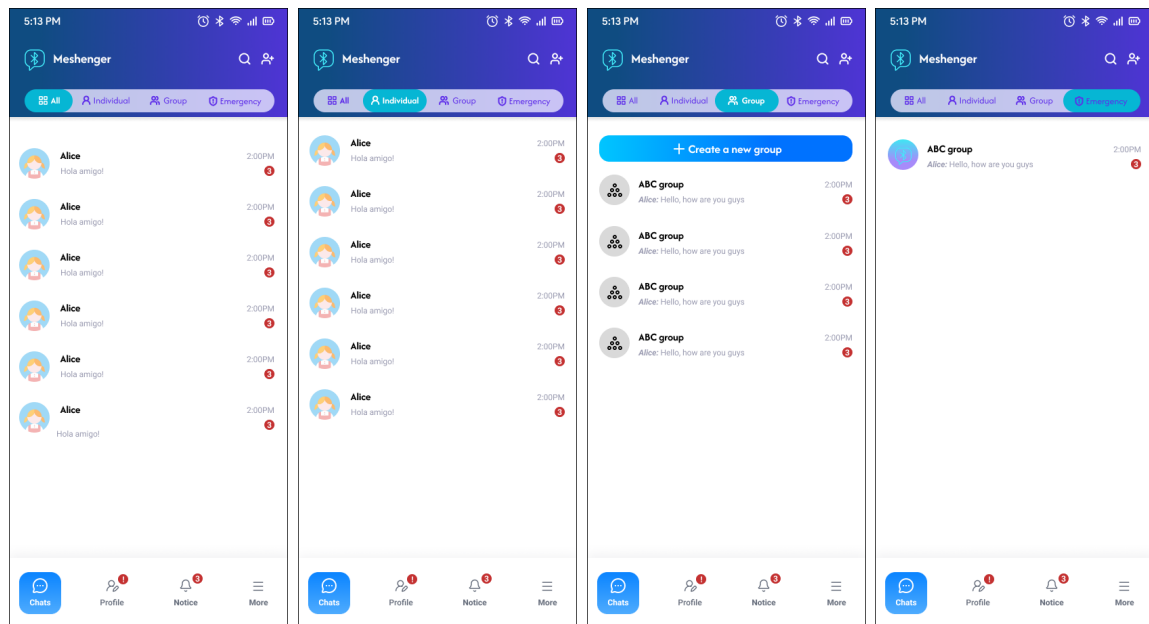


Figure 39: All, Individual, Group and Emergency Chat Box Screens (from left to right)

After exiting the user profile section, the Chat Box interface is displayed with four chat filters: All Chat, Individual Chat, Group Chat, and Emergency Chat. Users can switch between filters by swiping across the body section or by selecting a filter directly. The displayed user list changes depending on the selected filter:

- **All Chat:** Displays all previously connected users and groups. In particular, the Global Chat is always placed at the top of the chat list to ensure users can quickly connect with others when necessary.
- **Individual Chat:** : Displays all previously connected users' chat history.
- **Group Chat:** : Displays all joined groups' chat history
- **Emergency Chat:** : Displays Global Chat to communicate with everyone in the mesh.

6.4.5 Add a User

To add a user, the application provides two connection methods: Device Scanning and QR Scanning. Users can access these methods by clicking the Add User icon at the top-right corner of the Chat Box screen and selecting one of the available options.

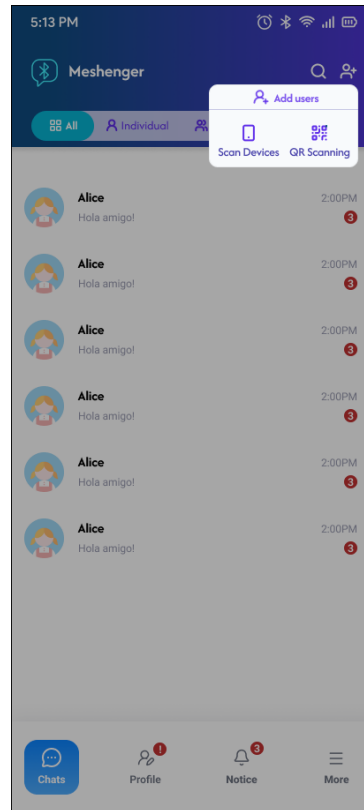


Figure 40: *Add a User Popup*

For the Device Scanning method, users are navigated to the Device Scanning screen, which displays a list of nearby mesh devices available for chatting.

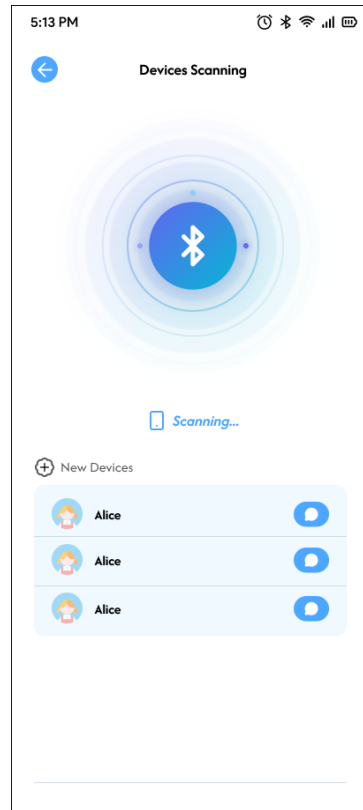


Figure 41: *Device Scanning screen*

For the QR Scanning method, users are navigated to the QR Scanning screen, which provides two ways to scan for a new device: direct scanning and image picker scanning. On the other hand, users can select “My QR” to generate a QR code containing their personal information for scanning by another device.



Figure 42: *QR Scanning Screen*

6.4.6 Send Messages

Similar to other messaging applications, users can communicate by sending messages and receive notifications when they get a reply.

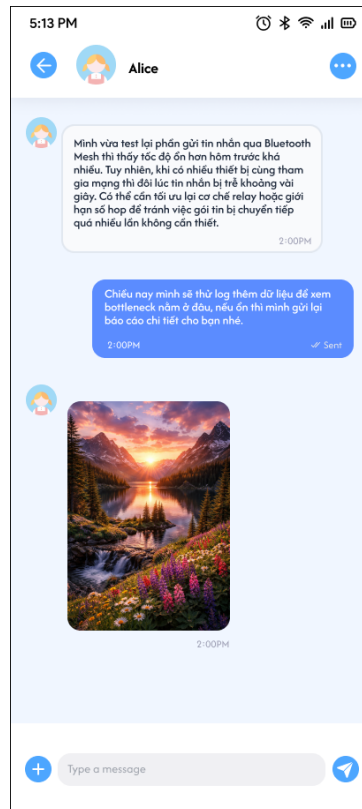


Figure 43: *Messaging Screen*

7 System Implementation

7.1 Technology Stack

This section presents the main technologies used to implement the application. The selected stack combines cross-platform interface development with Android-native integration so that the system can provide a modern user experience while still supporting low-level device capabilities such as Bluetooth communication.

7.1.1 React Native

React Native is an open-source framework developed by Meta Platforms for building cross-platform mobile applications using JavaScript and React. It allows developers to create native mobile applications for both iOS and Android using a single codebase, which helps reduce development time and implementation cost.

React Native follows a component-based architecture in which the user interface is constructed from reusable components. Unlike traditional hybrid frameworks that depend primarily on web views, React Native renders native UI elements, resulting in better performance and a more natural user experience.

The framework also adopts React's declarative programming model, enabling developers to describe the interface based on application state. In addition, features such as hot reloading accelerate development by allowing changes to be reflected immediately during implementation and testing.

For this project, React Native is used to structure the main user-facing screens, including onboarding, profile management, chat views, and navigation flows. Its reusable component model improves maintainability while supporting a consistent experience across the mobile interface.



Figure 44: *React Native*

7.1.2 Expo

Expo is an open-source platform and toolchain built around React Native, developed to simplify and accelerate mobile application development. It provides a managed workflow in which much of the project configuration is handled automatically, reducing setup complexity and enabling faster iteration.

Expo improves development efficiency through features such as live reloading and hot reloading, which allow developers to view interface updates in real time. It also streamlines asset management, application builds, and deployment tasks, helping the development team move from prototyping to testing more efficiently.

Another important capability of Expo is its support for over-the-air updates, which allows non-store updates to be distributed more quickly when appropriate. In this project, Expo is used to support rapid prototyping, UI testing, and project configuration, while Android-specific features can still be integrated through native modules when needed.



Figure 45: *Expo*

7.1.3 Kotlin

Kotlin is a modern, statically typed programming language developed by JetBrains and officially supported by Google as a primary language for Android development. Its full interoperability with Java allows developers to use existing Java libraries and frameworks while gradually adopting Kotlin in new or evolving modules.

One of Kotlin's major strengths is its concise and expressive syntax, which reduces boilerplate code and improves readability. It also provides modern language features such as null safety, data classes, extension functions, and coroutines for asynchronous programming, all of which help developers write safer and more efficient software.

Within this project, Kotlin is particularly valuable for Android-native functionality, especially where deeper platform integration is required. This includes native Android services, platform-level BLE handling, and interoperability with Android libraries that support communication, lifecycle management, and performance optimization.



Figure 46: *Kotlin*

7.1.4 Android Studio

Android Studio is the official integrated development environment for Android application development, developed by Google and built on IntelliJ IDEA by JetBrains. It provides a comprehensive environment for coding, testing, debugging, and packaging Android applications.

The IDE includes intelligent code completion, syntax highlighting, refactoring tools, and integrated Gradle support for build automation and dependency management. Android Studio also provides an emulator for testing applications on virtual Android devices, together with profiling and debugging tools for monitoring CPU usage, memory consumption, and runtime behavior.

For this project, Android Studio serves as the main development environment for building and validating the Android side of the application. It is especially important for testing device integration, debugging native modules, managing Gradle-based builds, and examining application behavior during implementation.



Figure 47: *Android Studio*

7.2 Mobile Application

7.2.1 Onboarding Screens

When a user first opens the application, the system automatically translates the interface language based on the device's localization settings. Currently, the system supports only English and Vietnamese. If the device location is outside regions that primarily use these two languages, the application defaults to Vietnamese. The **Figure 48** illustrates the onboarding screens displayed when the application is first launched on a device located in Vietnam.

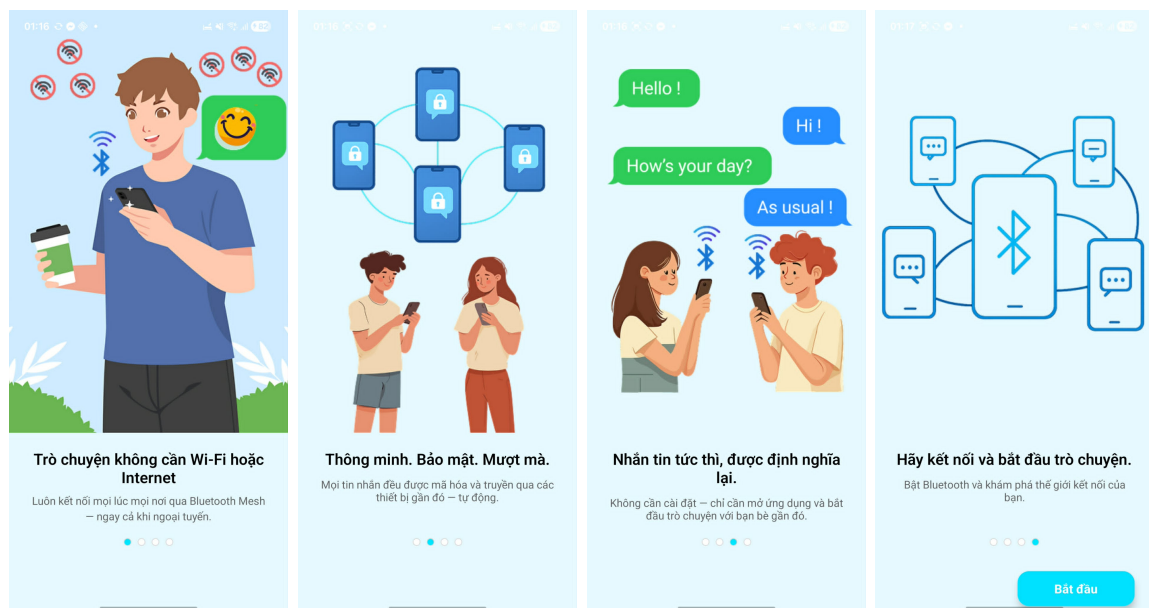


Figure 48: Onboarding Screens

7.2.2 User Profile

After the user has viewed all four onboarding screens, they are directed to the profile setup screen, where they are required to provide a username and select an avatar image.

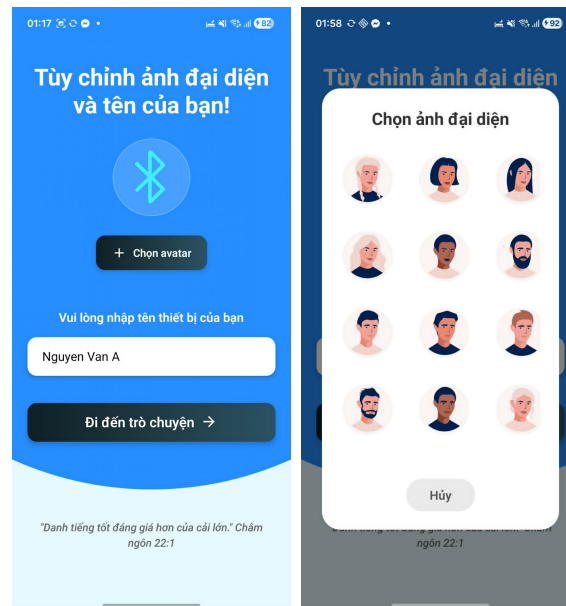


Figure 49: Set Profile Section

7.2.3 Global Chat

Whenever the user accesses the application, the system automatically creates a mesh network to advertise the device for messaging and peer discovery. The Global Chat feature enables all devices within the mesh network to communicate with one another. However, due to the large volume of communications and the open nature of the channel, the security level of Global Chat is lower than that of one-to-one messaging established through device scanning or QR scanning.

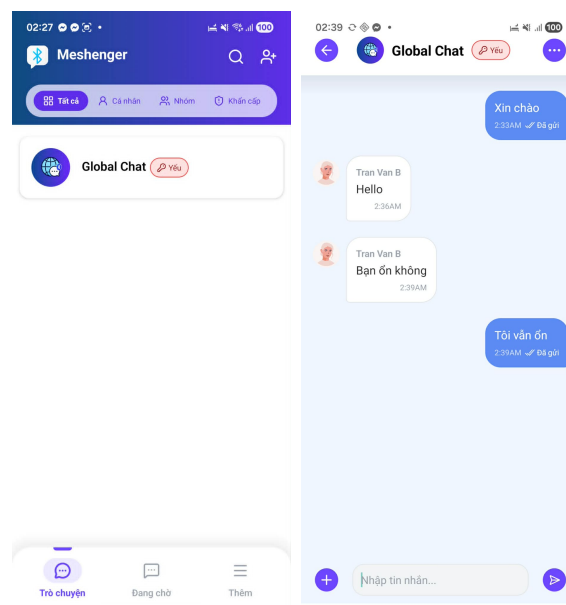


Figure 50: Global Chat Security Level and Chat Screen

7.2.4 Device Scanning

Device Scanning is one of the methods used to connect with other users. Through the bootstrapping process, the device can recognize nearby devices within the mesh network. The Device Scanning screen can be accessed by selecting the pop-up menu from the add icon located at the top-right corner of the interface and then pressing the device icon.

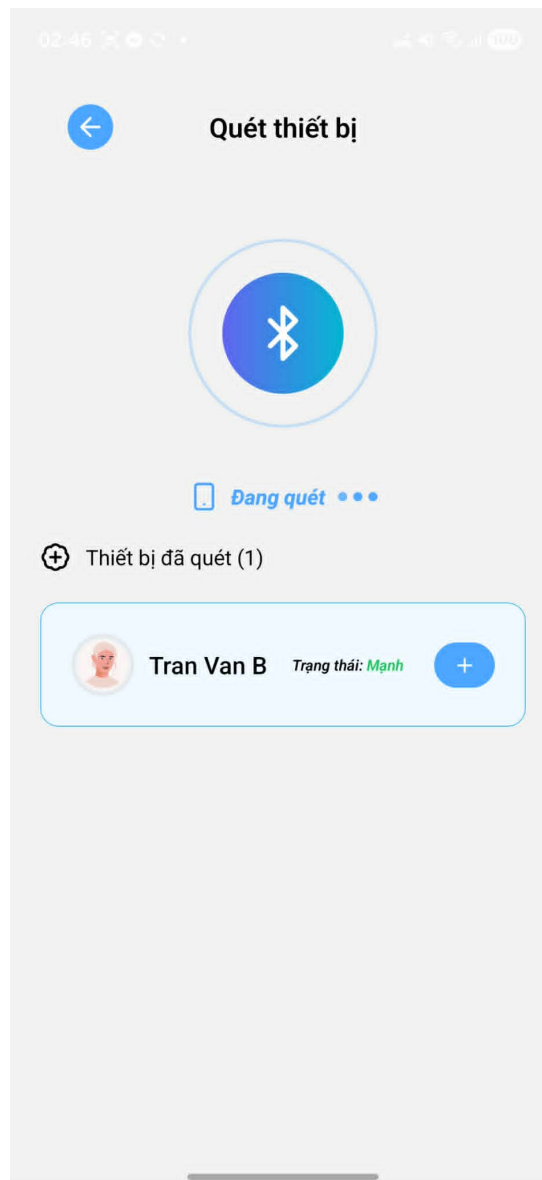
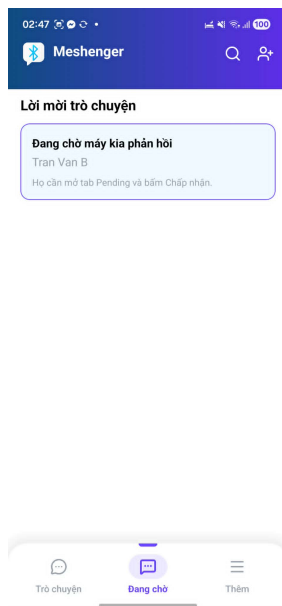
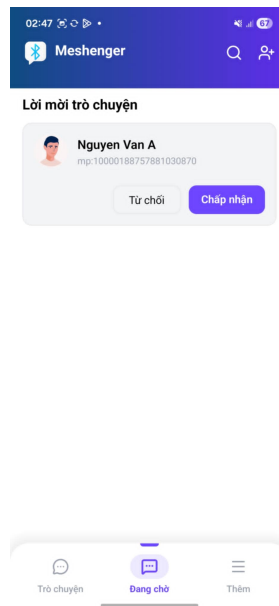


Figure 51: *Device Scanning Screen*

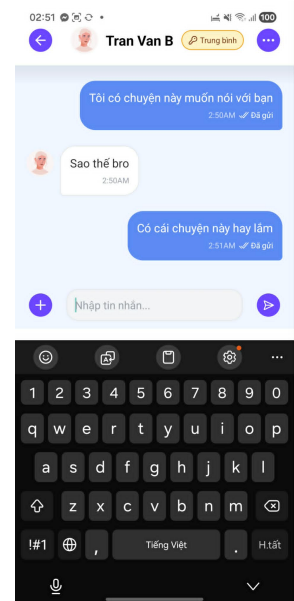
As shown in the **Figure 52**, the requesting user must wait for the selected user's response to the chat request. If the selected user accepts the request, both users are automatically navigated to a newly created one-to-one chatroom with a medium level of security.



(a) Requesting User Pending Screen



(b) Receiving User Pending Screen



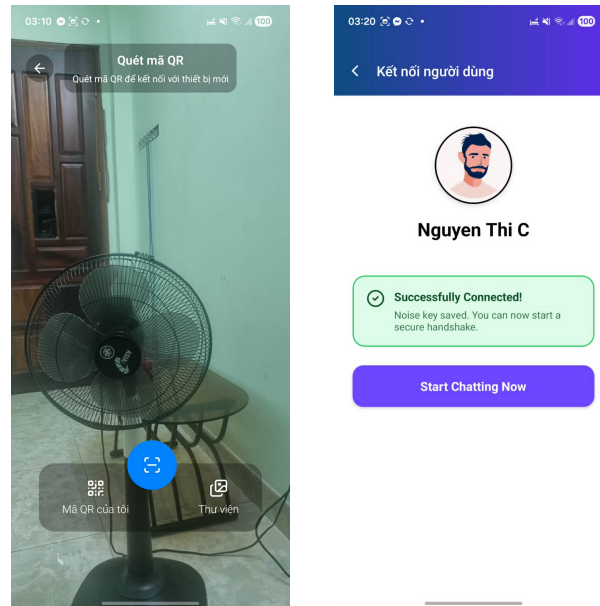
(c) 1:1 Chat Screen

Figure 52: Device Scanning Process

7.2.5 QR Scanning

QR Scanning is a more secure user-adding method compared to Device Scanning, as it provides a higher level of trustworthiness between users. By selecting the QR icon from the pop-up menu after pressing the add-user icon at the top-right corner of the screen, the user is navigated to the QR Scanning screen. The system provides two QR scanning methods: direct scanning and image-picker scanning.

Direct QR scanning allows the user to scan another peer's QR code using the device camera. After a successful scan, the system automatically creates a one-to-one chatroom with the highest security level. If the user wishes to start messaging immediately, they can press the provided button to navigate directly to the chatroom.



(a) QR Camera Screen (b) Receiver Scanned Info Screen

Figure 53: Direct QR Scanning Process

Image Picker Scanning allows users to select a previously saved QR code image from the device library for scanning. This method helps users save time by enabling them to store another user's QR information and scan it later when needed.

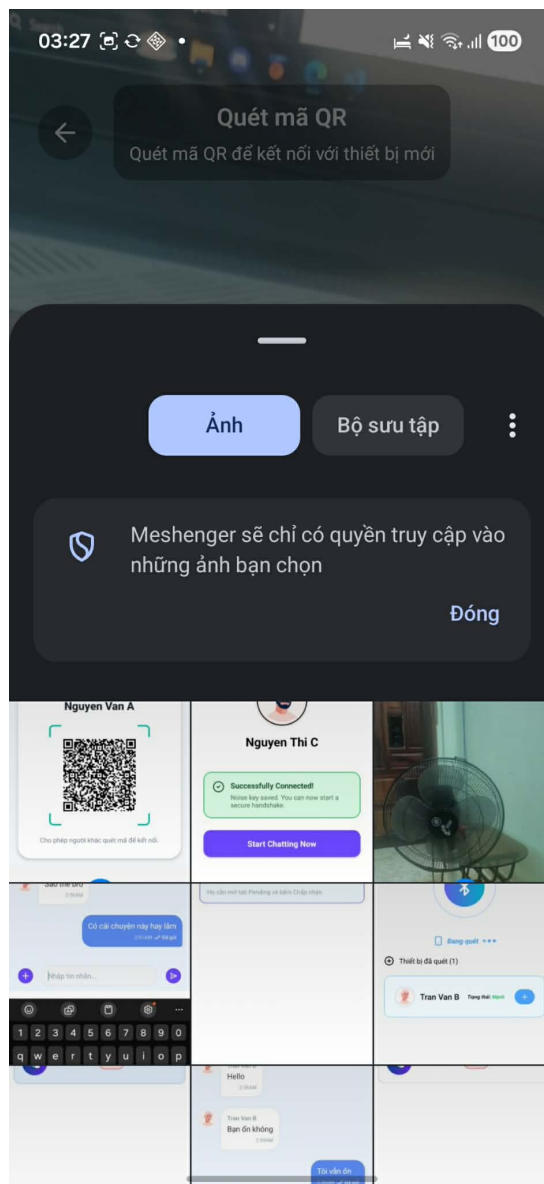


Figure 54: *Image Picker Screen*

For other users to scan, the My QR feature generates a QR code containing the device's information, including the username, avatar, MP address, public key, and private key.



Figure 55: *My QR Screen*

7.3 Feature Implementation

7.3.1 Secure Channel Establishment

Introduction

The primary objective of the application is to guarantee secure communication between peers. All messages exchanged between users or within channels are protected by end-to-end encryption, ensuring that only the intended participants can access the content. To accommodate different usage scenarios and security requirements, the application supports three distinct types of communication channels with increasing levels of assurance.

Channel Types and Security Levels

- **Global channel (Low-level security):** the global channel is designed for open communication among all devices connected to the network. Messages in this channel are protected using a basic symmetric encryption scheme managed by the application. To reduce the impact of potential key exposure, the symmetric key is periodically rotated whenever predefined update conditions are met.
- **Secure channel without out-of-band verification (Medium-level security):** this channel provides stronger protection for one-to-one and group communication. Messages are encrypted end-to-end, and peers are authenticated at the device level within the network. However, the application cannot guarantee that the real-world identity of the user behind a device matches their claimed identity.
- **Secure channel with out-of-band verification (High-level security):** this channel offers the highest level of security. In addition to end-to-end encryption and peer authentication, users verify each other's identities through an external process such as QR-code scanning to exchange public keys.

Cryptographic Primitives

The cryptographic primitives selected for secure channel establishment are:

- **Diffie–Hellman:** Curve25519.
- **Cipher:** ChaCha20-Poly1305.
- **Hash function:** SHA-256.

7.3.2 Identity Management

The application employs a three-tier identity management framework designed to balance usability, routing functionality, and cryptographic assurance. Each tier provides a distinct level of trust and serves a specific role within the overall security architecture.

Three-Tier Identity Model

- **Tier 1: Username.** Usernames are mutable, human-readable identifiers. They improve usability but are not cryptographically bound to either the device or the user.
- **Tier 2: Routing ID.** The routing identifier functions as the operational address for packet delivery between peers. It is refreshed periodically or upon application restart so that routing can work without exposing a stable long-term device identity.

- **Tier 3: Static Public Key.** The static public key is the most trusted tier. It uniquely binds a device within the network and serves as its definitive cryptographic identity.

Username

Username are displayed in chat interfaces and peer discovery views. To ensure consistency and usability, usernames are restricted to a maximum length of 100 characters. While users are permitted to modify their usernames, a cooldown period of 48 hours is enforced between changes to prevent excessive modifications and support stable peer recognition.

Routing Identifier

Using a static public key directly as a routing identifier is undesirable because prolonged observation of network traffic could allow adversaries to correlate packets and track devices. To mitigate this risk, the application employs a dynamic routing identifier referred to as the *Mesh Protocol Address (MP Address)*.

Dynamic routing identifiers provide several important security properties:

- **Replay protection:** prevents reuse of captured identifiers outside their valid epoch.
- **Proof of possession:** ensures that only peers holding the corresponding private key can establish secure channels.
- **Forward privacy:** compromise of one routing ID does not expose past or future identifiers.

Mesh Protocol Address Properties

The MP Address must be cryptographically bound to the device's static public key, one-way, and universally computable by devices that have the required information. To achieve interoperability and decentralization, the address construction incorporates both static and dynamic components.

Mesh Protocol Address Design

The MP Address derivation integrates static elements, including the device's public key, an application-specific salt, protocol metadata, and a static arbitrary key embedded within the application. Dynamic elements, primarily time-based epochs, introduce regular rotation to enhance privacy and reduce predictability.

Construction Flow of the Mesh Protocol Address

- **Static part:** the long-term static public key, protocol metadata, an application-specific salt, and an embedded arbitrary static key.
- **Dynamic part:** a time-based window derived from the current Unix time, divided into fixed one-day intervals.
- **Cryptographic function:** HKDF is used to derive a one-way routing identifier from the static and dynamic inputs [13].

The generation of the unique peer address is shown in equation (1)

$$\begin{aligned} \text{MP_Address} = \text{HKDF} \Big(& \text{IKM} = \text{SHA256}(PK_{\text{peer}}), \\ & \text{salt} = \text{salt_value}, \\ & \text{info} = \text{info_prefix} \parallel \left\lfloor \frac{t_{\text{unix}}}{86400} \right\rfloor \parallel \text{SHA256}(PK_{\text{app}}), \\ & \text{length} = L \Big) \end{aligned} \quad (1)$$

Static Public Key

The static public key is generated once during the initial launch of the application using a long-term Diffie–Hellman key pair based on the X25519 elliptic curve, as specified in the Noise Protocol Framework. It is used for authentication, secure channel establishment, and routing identifier derivation, and is only transmitted in encrypted form during handshake procedures.

7.3.3 Routing Protocol

In a Bluetooth-based peer-to-peer mesh system, routing must operate under strict constraints such as low bandwidth, limited MTU, short-range connectivity, and frequent topology changes. In addition, the system must continue delivering packets even when no stable end-to-end path exists between the sender and receiver.

For this reason, the project adopts an epidemic routing protocol as the fallback dissemination strategy for the mesh. Rather than depending on persistent route tables, nodes replicate packets opportunistically across directly connected neighbors, allowing delivery to continue even when the destination is temporarily absent from the current connected component [17].

Epidemic Routing Protocol

The epidemic routing protocol serves as a last-resort mechanism for delivering packets across the mesh network when no optimal path can be found or when the destination is temporarily unavailable. It is a flood-based routing protocol in which each node forwards eligible packets to all of its directly connected neighbors. A key feature of the protocol is its store-and-forward,

or anti-entropy, capability. Intermediate nodes retain packets in a local cache even when the destination is not currently reachable.

Whenever a device connects to a new peer, or when a periodic synchronization event is triggered, it shares a compact representation of the packets it currently stores. Neighboring nodes compare that summary with their own caches, identify missing packets, and request only the missing data. This mechanism reduces redundant retransmission while preserving resilience benefits.

Anti-Entropy in MANETs

The project operates in a Mobile Ad Hoc Network (MANET) environment in which devices frequently join, leave, and reconnect to the mesh. These changes create inconsistent network views and may interrupt packet delivery before the destination is reached. To mitigate this instability, the system implements anti-entropy as a store-and-forward synchronization mechanism that gradually reconciles packet state across neighboring nodes and restores delivery opportunities after temporary disconnections.

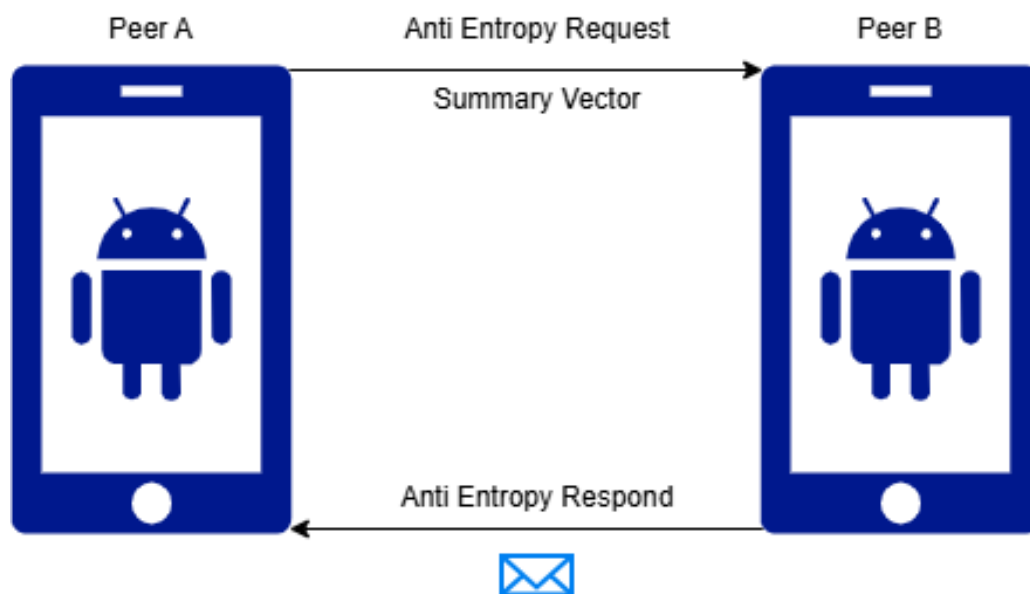


Figure 56: *Anti-entropy synchronization between two peers using a summary vector request and response*

Protocol Overview

Synchronization follows a request–response cycle:

1. **Request Phase:** at periodic intervals, or immediately after a new peer connection is established, a node sends an Anti-Entropy Request containing its local Summary Vector.
2. **Comparison Phase:** neighboring nodes compare the requester’s packet inventory against their own local caches.

3. **Response Phase:** if a neighbor identifies packets missing from the requester's cache, it encapsulates those packets into Anti-Entropy Response packets and transmits them back.
4. **Integration Phase:** the requester unwraps the received payloads, reassembles any fragmented packets, stores them in its cache, updates its Summary Vector, and continues processing them through the normal forwarding pipeline.

Probabilistic Trigger Mechanism

To reduce synchronization overhead, the system uses a probabilistic trigger:

- **Initialization:** each device starts with a 0% probability of triggering a request.
- **Incremental Logic:** for every interval in which a request is not sent, the probability increases by 5%.
- **Reset Logic:** once a request is triggered and transmitted, the probability resets to 0%.

This linear progression keeps synchronization traffic low during stable periods while guaranteeing eventual cache reconciliation after prolonged inactivity.

Forwarding Constraints

To prevent uncontrolled flooding, each forwarded packet is bounded by a TTL value, filtered through duplicate suppression, and checked against the local packet cache before retransmission. Packets are retained only for the configured store-and-forward window and are removed after acknowledgment, expiration, or successful delivery.

7.3.4 Mesh Maintainer

Mesh Maintainer Logic

The Mesh Maintainer is a background service designed to automate mesh integration over BLE. Once the required permissions are granted and the radio is active, the service manages the device's network presence through two primary states: *In-Mesh* (server mode) and *Out-Mesh* (client mode). This dual-state behavior keeps the network stable while enabling self-healing whenever connectivity degrades.

The maintenance loop operates as follows:

1. **Connectivity Assessment:** the device monitors its outbound connection count. If the count satisfies the configured minimum, the device remains in the *In-Mesh* state.
2. **On-Demand Scanning:** if the count falls below the threshold, the device initiates a synchronized scan for 2000 ms to discover nearby peers advertising the *In-Mesh* role.

3. **State Transition and Association:** if no suitable peers are found, the device assumes the role of a seed node and returns to the *In-Mesh* state. If peers are found, the device waits for a random jitter between 500 ms and 2000 ms, transitions to the *Out-Mesh* state, and attempts outbound connections until the limit is reached.
4. **Bidirectional Establishment:** every successful outbound connection is paired with the corresponding inbound registration and stored in the `MeshConnectionRegistry`.
5. **Resource Preservation:** after each maintenance cycle, the service sleeps for 5 seconds before reevaluating network status.

```
while the Mesh Service is running do
    retrieve the number of active outbound connections
    if outbound connections are below MIN_CONNECTIONS_LIMIT then
        start a synchronized scan for 2000 ms
        discover peers advertising "In-Mesh"
        if at least one suitable peer is found then
            wait a random jitter between 500 ms and 2000 ms
            transition to Out-Mesh state
            attempt connections until the limit is reached
            register each successful link in MeshConnectionRegistry
        end
    else
        transition to In-Mesh state
    end
end
else
    maintain In-Mesh state
end
sleep for 5 seconds
end
```

Algorithm 1: Mesh Maintenance Loop

7.3.5 Payload Structure and Protocol

This subsection describes the payload structure and protocol design. The discussion begins with the payload structure, where each field is introduced, described, and assigned a specific purpose within the communication framework. The protocol then distinguishes between two primary packet classes: system packets and user packets. The payload format is shown in 57.

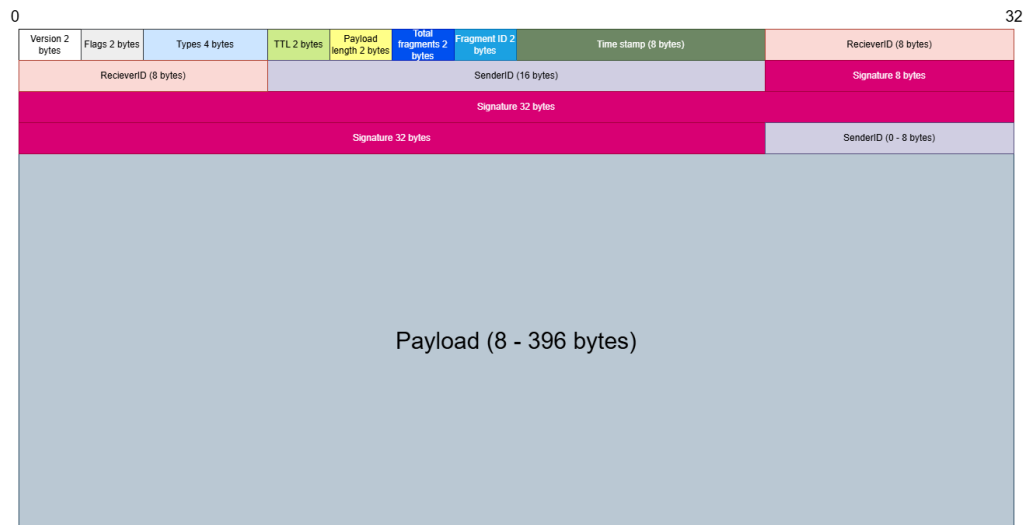


Figure 57: *Payload Format*

Payload Structure

- **Version** (2 bytes): indicates the protocol version used for compatibility and parsing logic.
- **Flag** (2 bytes): identifies whether special handling is required for the packet.
- **Type** (4 bytes): specifies the packet type to guide processing behavior.
- **TTL** (2 bytes): limits the lifespan of the packet across the network.
- **Payload Length** (2 bytes): defines the size of the payload for buffer allocation and parsing.
- **Total Fragments** (2 bytes): indicates the number of fragments belonging to the complete message.
- **Fragment ID** (2 bytes): identifies the index of the current fragment.
- **Timestamp** (8 bytes): records the Unix time at which the packet was created.
- **Receiver ID** (8 bytes): contains the Mesh Protocol Address of the intended recipient. For broadcast packets, a reserved broadcast value is used.
- **Sender ID** (8 bytes): contains the Mesh Protocol Address of the sender.
- **Signature** (8 bytes): stores a cryptographic signature used to verify integrity and authenticity.
- **Payload** (0-396 bytes): carries application-level data, which may include encrypted content or structured messages.

Protocol

System packets are generated automatically by the application to support mesh formation, secure channel establishment, and anti-entropy synchronization. User packets are created in response to explicit user actions and carry application-level communication data.

System Packets

- **Bootstrap Packet (0x00):** used to initiate network discovery and request entry into the mesh network.
- **Noise handshake packet (0x01):** carries handshake data required to establish a secure channel between peers.
- **Anti-Entropy Request Packet (0x02):** initiates cache synchronization between neighboring peers by sending a Summary Vector.
- **Anti-Entropy Response Packet (0x03):** returns cached packets or fragments missing from the requester.

User Packets

- **All Message Packet (0x10):** transmits user-generated messages for global chat and is the core communication packet type.
- **All Message Packet (0x11):** transmits user-generated messages for 1:1 sessions and is the core communication packet type.

8 Testing and Deployment

Software testing is a critical phase of the development lifecycle, typically executed concurrently with or immediately following software implementation. It involves a collaborative effort between dedicated Quality Assurance (QA) and Quality Control (QC) teams, developers, and project stakeholders to ensure the final product aligns with specified requirements, functions as expected, and is free of critical defects. For this project, the team implemented a tiered testing strategy consisting of three distinct methodologies to guarantee overall system robustness:

1. **Unit Testing:** This serves as the foundation of the testing architecture. Its primary objective is to isolate and verify the smallest testable parts of the application, such as individual functions, classes, or modules. By testing these components in isolation, we ensure that the core logic and critical requirements of each “building block” are fulfilled prior to system integration.
2. **Integration Testing:** Once individual units are validated, Integration Testing is conducted to focus on the interactions between disparate modules or services. The goal is to identify interface defects and confirm that separate components, which may operate correctly in isolation, function as intended and exchange data accurately when combined.
3. **Use Case Testing:** The next layer of the strategy is Use Case Testing, a form of functional testing designed to validate the system from the end-user’s perspective. By executing specific scenarios based on user requirements, we ensure that the application workflow is intuitive and that the system successfully fulfills the practical needs and expectations of the stakeholders.
4. **Non-Functional Testing:** The final layer of the strategy is Non-Functional Testing, which evaluates the operational attributes of the system rather than specific functional behaviors. By measuring quantifiable engineering metrics, such as throughput, end-to-end latency, and system resource consumption we ensure that the architecture is resilient, meets strict operational benchmarks, and remains stable in a distributed environment.

8.1 Unit Testing

8.1.1 Overview

Unit testing serves as the initial phase of the software testing lifecycle. It involves the isolation and independent verification of the smallest testable units of an application, typically a single function, method, class, or module. By decoupling these components from the rest of the system,

developers can ensure that each individual "building block" adheres strictly to its functional requirements and design specifications.

Testing Library: AndroidJUnit4

AndroidJUnit4 is a specialized test runner provided by the AndroidX Test library. It is designed to allow developers to write tests in a consistent format, whether they are running as local unit tests on a JVM or instrumented tests on a physical device or emulator.

Testing Focus

The unit testing phase for this project focuses on verifying the integrity of the core networking and cryptographic components. Each module is tested in isolation to ensure logical correctness and adherence to the protocol specifications.

Static Key Management: Validation of key generation, storage, and retrieval processes. Tests ensure that static keys are handled securely and remain consistent throughout the session life-cycle.

MP Address Calculation: Ensures the mathematical derivation of the Multi-Path (MP) address is accurate. Tests validate that the calculation yields the expected output based on specific input parameters.

Packet Creation & Serialization: Validation of the packet construction process. This ensures that headers are correctly formatted and that the serialization logic accurately parse legitimate packets while rejecting tampered or malformed ones.

Packet Signing & Verification: Validation of the packet signing feature. This ensures that the packet is not tampered with and accepting legitimate ones.

Fragmentation and Reassembly Logic: Tests the system's ability to detect payloads exceeding the Maximum Transmission Unit (MTU) and segment them accordingly. Additionally, it verifies the system's capability to identify fragmented packets and accurately restore the original payload.

8.1.2 Testing Approach

For the verification and validation phase of unit testing this project, a combined strategy of Data-Driven Testing (DDT) and Black-Box Testing was adopted. This methodology decouples test data from execution logic and focuses on validating system behavior against functional specifications.

Methodology

Data-Driven Testing (DDT): [6] DDT is a software testing methodology characterized by the strict separation of test execution logic from test data vectors. Rather than hardcoding inputs

and expected outcomes within individual test cases, data is centralized into structured datasets. A singular, reusable test harness iterates through this dataset, executing the same logical sequence against varying inputs and verifying the respective outputs. This approach maximizes test coverage, eliminates code redundancy, and minimizes maintenance overhead by allowing new test scenarios to be added purely as data entries.

Black-Box Testing (Testlio): [16] Black-Box testing is a software testing method in which testers examine the application's external functionality without knowing or accessing its internal structures, code implementations, or architectural workings. Instead of evaluating how the software executes a task, test cases are designed purely around user requirements and functional specifications to evaluate what the software outputs in response to specific inputs.

Rationale for Selection

The integration of Data-Driven Testing with a black-box strategy was selected to optimize the testing lifecycle based on the following engineering advantages:

- **Optimized Test Coverage:** Running a uniform test script across highly diverse datasets allows the system to evaluate numerous input permutations. This ensures that critical logical paths, validation boundaries, and edge cases are thoroughly exercised without requiring individual test scripts for every scenario.
- **Efficiency Under Strict Time Constraints:** Given the compressed timeline remaining immediately following the system implementation phase, this hybrid approach accelerates the testing cycle. Testing the application as a black box eliminates the time-consuming overhead of drafting highly complex internal unit mocks, while DDT automates execution across multiple test vectors simultaneously.
- **High Reusability of Test Logic:** Because the test harness code remains completely decoupled from the data, the exact same execution script is reused across positive, negative, and boundary test scenarios. This drastically reduces code duplication within the testing suite.
- **Simplified Maintenance:** If application requirements shift or a test case needs updating (e.g., modifying expected return values or data payloads), changes are confined strictly to the centralized data arrays or files. The underlying test logic remains untouched, minimizing regression risks during updates.
- **Scalability via Automation:** This architecture is inherently built for automation. Expanding test coverage to accommodate new features or newly discovered edge cases requires only the addition of a new data row to the existing dataset, allowing the test suite to scale effortlessly without expanding the codebase size.

8.1.3 Unit Testing Test Case Summary

Table 11 provides a comprehensive overview of the input parameters and verification objectives for each scenario executed within the unit testing suite.

Table 11: *Test Case Summary Matrix for Unit Testing*

Feature	Test Script	Inputs / Parameters Evaluated	Objective / Verification
Static Key Management	StaticKeyManager- IntegrationTest	Generated <i>X25519</i> private key	Verifies symmetric encryption safety (<i>AES/GCM</i>) via key wrapping and unwrapping round-trips.
		Random <i>X25519</i> public key bytes	Validates data transformation safety during Base64 string codec casting.
		Device local key storage state	Assures cryptographic identity consistency and retention across application lifecycles.
		Raw <i>X25519</i> public identity key structure	Confirms structural compatibility when encoding/decoding raw identity keys across wire formats.
MP Address Calculation	MPAddress- ParameterizedTest	0xA, 0xB, 0xC, 0xD byte identity matrices; 24-hour epoch lengths	Validates round-trip string-to-byte arrays and ULong bit conversions for multi-path address data.
		Two distinct node public keys, same time brackets	Ensures mathematical collision resistance between independent nodes calculating addresses in parallel.

		Single node public key, same time epoch	Assures calculation determinism (reproducible multi-path indexing addresses per epoch).
Packet Creation & Serialization	PacketFactory-ParameterizedTest	Global Broadcast, Protocol Bootstrap, Direct 1-to-1, and Noise Handshake message types	Validates unsigned header formatting, correct receiver ID mapping, and automated 64-byte signature block generation.
	PacketSerialization-ParameterizedTest	Plain messages, NEED_ACK flags, IS_COMPRESSED requests, and boundary 0-byte payloads	Confirms structural byte alignment, flags layout retention, and data payload symmetry after full binary encode/decode round-trips.
Packet Signing & Verification	PacketSigner-ParameterizedTest	Global Chat Path, Global App Protocol, and Direct Communication paths	Verifies cryptographic signature parsing logic (<i>HMAC/Ed25519</i>) and routing alignment across single-packet and fragmented blocks.
Fragmentation & Reassembly Logic	PacketFactory-ParameterizedTest	500-byte bulk array input	Assures payload segmentation splits correctly into multiple components when exceeding standard MTU limits.
	PacketSerialization-ParameterizedTest	500-byte byte array input	Validates correct sequence padding and payload reconstruction formatting for multiple tracking chunks.

PacketSigner- ParameterizedTest	10,000-byte bulk data array inputs	Ensures signature logic retains context verification symmetry when split across distinct fragmented header pieces.
ReassemblyQueue- ParameterizedTest	Short message string payload, sequential processing order	Checks that single-packet messages cleanly bypass multi-step reassembly processing without data overhead.
	2000-byte chunk array, ordered sequential stream	Validates the base case path where fragmented packets are cached sequentially to successfully rebuild a payload.
	2000-byte chunk array, shuffled/randomized stream	Emulates multi-path network routing to verify out-of-order slice tracking and precise index-based array reassembly.
	2000-byte chunk array, shuffled stream with dropped frame	Assures partial array tracks remain incomplete and safely drop final reassembly triggers if an expected packet drops.

8.1.4 Unit Testing Results Summary

The results of the unit testing phase are summarized in Table 12, indicating that all core modules successfully passed their respective validation suites.

Table 12: *Unit Testing Results Summary*

Index	Module	Test Suites	Test Cases	Result
1	Static Key Management	1	4	Passed
2	MP Address Calculation	1	12	Passed
3	Packet Creation & Serialization	2	10	Passed
4	Packet Signing & Verification	1	6	Passed
5	Fragmentation and Reassembly Logic	1	4	Passed

8.2 Integration Testing

Overview

Integration testing serves as the subsequent phase of the software testing lifecycle. While unit testing verifies components in isolation, integration testing evaluates how these individual modules collaborate and interact across architectural boundaries under diverse conditions. By executing these tests on a physical device via AndroidJUnit4, the validation encompasses native cryptographic operations, real-world memory caches, and inter-module data pipelines which cannot be accurately simulated on a local JVM.

Testing Focus

The integration testing phase for this project shifts the focus away from individual packet constructs towards the overarching system protocols. Specifically, it rigorously evaluates the comprehensive cryptographic pipelines governing multi-party and two-party communication sessions, followed by the Epidemic Flooding's synchronization capabilities (Anti-Entropy).

Session Cryptography Pipeline: This suite guarantees the strict enforcement of security properties (Confidentiality, Integrity, and Authenticity) across both chat modalities:

- **Global Chat Integrity (AES-GCM):** Tests validate the bidirectional encryption/decryption process. Furthermore, negative test cases ensure that the system aggressively rejects payloads encrypted with unauthorized keys (AEAD Tag mismatch) and strictly enforces Initialization Vector (IV) uniqueness by dropping packets with misaligned timestamps to prevent Replay Attacks.
- **Two-Party Noise Protocol (ChaCha20Poly1305):** Verifies the CipherState transition mechanics. The tests assert that both the sender and receiver mathematically increment their cryptographic Nonces synchronously after each transmission. It strictly validates that any out-of-order, dropped, or maliciously re-transmitted packets are outright rejected, thereby guaranteeing sequential data integrity.

Anti-Entropy Synchronization Mechanism: This test suite validates the system's ability to self-heal and recover missing data across the mesh network without flooding the bandwidth. The integration tests specifically verify:

- **Bloom Filter Serialization:** Ensures that the system can accurately compress thousands of packet signatures into a compact memory footprint (Bloom Filter) and successfully deserialize it across the network boundary while maintaining mathematical accuracy.
- **Missing Packet Detection:** Validates the logic where a node compares a peer's incoming Bloom Filter against its local memory caches to precisely identify the exact packets the peer is missing.
- **Respond Packaging & Reassembly:** Ensures that recovered packets are correctly nested within an ANTI_ENTROPY_RESPOND payload. It also verifies that if the recovered payload exceeds the BLE MTU limit, the transport layer automatically fragments and sequentially reassembles the payload at the destination.

Integration Testing Results Summary

The results of the integration testing phase are summarized in Table 13, indicating that the core security pipelines and synchronization mechanisms successfully passed their respective validation suites on the physical Android hardware.

Table 13: *Integration Testing Results Summary*

Index	Module	Test Suites	Test Cases	Result
1	Session Cryptography Pipeline	1	5	Passed
2	Anti-Entropy Mechanism	1	3	Passed

8.3 Use Case Testing

8.3.1 Use Case 1: Initial Application Launch

Main Flow

The main flow describes the process of how the user enters the application for the first time by following these steps: accepting application permissions, navigating through onboarding screens, filling in profile information, and finally accessing the chat screen.

Table 14: *Initial Application Launch Testing*

Step	Action	Expected Result	Status
1	Launch Meshenger applica- tion	Application starts successfully	Passed
2	Accept Meshenger permis- sions	Navigate from the splash screen to the onboarding screens	Passed
3	Navigate to the profile section	Profile section is displayed success- fully	Passed
4	Fill in avatar and profile infor- mation	Profile information is initialized suc- cessfully	Passed
5	Enter the chat screen	Chat screen is displayed normally	Passed

Alternative Flow 1: Reject Permissions

This flow describes the behavior of the application when the user rejects all required permissions during the splash screen. As a result, the application terminates immediately.

Table 15: *Reject Permissions Testing*

Step	Action	Expected Result	Status
1	Launch Meshenger app	Application starts successfully	Passed
2	Reject Meshenger permissions	Application exits and terminates	Passed

Exception Flow: Missing Username in Profile Section

To access the chat screen, the user must provide a username so that the system can initialize and broadcast the user information to other peers. This flow describes the behavior of the application when the user attempts to access the chat screen without entering a username or entering above 15 characters, causing an error message to be displayed. Avatar selection is optional because the system automatically assigns a default avatar image if the user does not choose one.

Table 16: *Missing Username Testing*

Preconditions: • The user launches the application for the first time. • The user accepts the app's permissions • The user goes through all four onboarding screens			
Step	Action	Expected Result	Status

1	Navigate to profile section	Profile section is displayed successfully	Passed
2	Navigate to chat screen	Error message is displayed	Passed
3	Enter above 15 characters in username section	Username is typed successfully	Passed
4	Navigate to chat screen	Error message is displayed	Passed

When an error message is displayed in the profile section while attempting to access the chat screen due to username validation failure, the user must re-enter the username until it satisfies the validation rules.

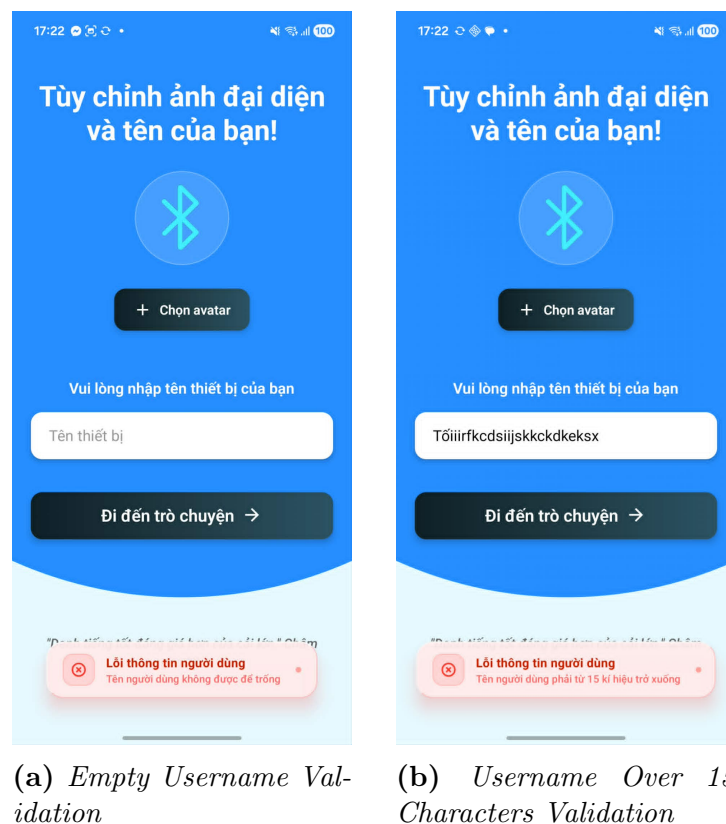


Figure 58: Username Validation

8.3.2 Use Case 2: Bluetooth Availability Check

Main Flow

In the main flow, Bluetooth is manually turned on and off to verify the availability detection mechanism. If Bluetooth is disabled, a popup dialog is displayed requiring the user to either enable Bluetooth or exit the application. In this report, Bluetooth availability will be checked in the chat screen only.

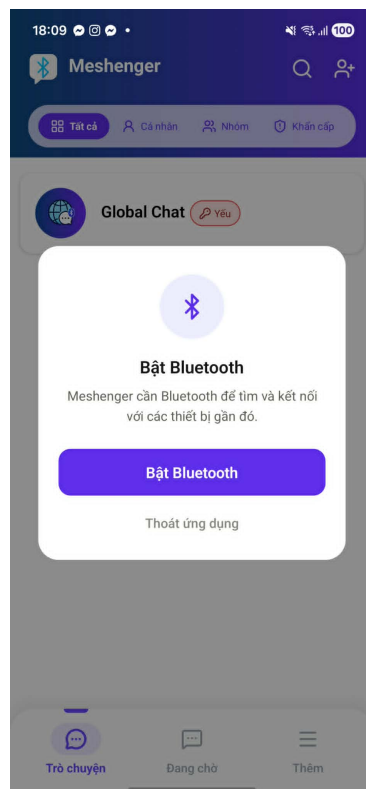


Figure 59: *Bluetooth Availability Check*

The testing for main flow is displayed in the table below:

Table 17: *Use Case 2 Main Flow Testing*

Preconditions: • The user launches the application • The user navigates to chat screen from profile section (first time user)			
Step	Action	Expected Result	Status
1	Navigate to chat screen	Chat screen is displayed successfully	Passed
2	Turn off Bluetooth	Bluetooth needed popup is displayed	Passed
3	Turn on Bluetooth	Popup disappear	Passed

Alternative Flow

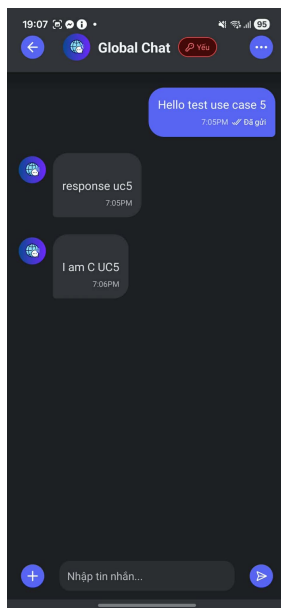
The alternative flow describes the behavior of the application when the user chooses to exit the application instead of enabling Bluetooth.

Table 18: *Use Case 2 Alternative Flow Testing*

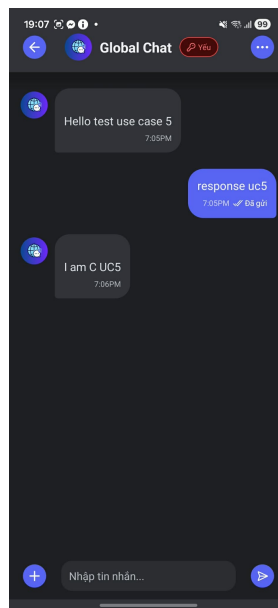
Preconditions: - The user launches the application - The user navigates to chat screen from profile section (first time user)			
Step	Action	Expected Result	Status
1	Navigate to chat screen	Chat screen is displayed successfully	Passed
2	Turn off Bluetooth	Bluetooth needed popup is displayed	Passed
3	Press exit app button	The application terminates	Passed

8.3.3 Use Case 3: Global Chat

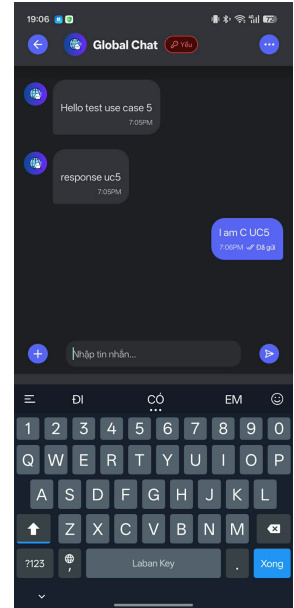
This use case describes how peers communicate within a mesh network. When a peer sends a message, all other connected peers in the network receive the message. In this test case, three devices are connected and exchange messages with one another to verify the communication process.



(a) *Device A Messaging Screen*



(b) *Device B Messaging Screen*



(c) *Device C Messaging Screen*

Figure 60: *Global Chat*

In Figure 60, when Device A sends a message to the global chat, all nearby peers, such as Devices B and C, receive the message instantly. The same behavior occurs when Devices B and C send messages to the global chat. The table below presents the test cases for the main flow of global chat communication.

Table 19: *Global Chat Testing Between Devices A, B, and C*

Preconditions:			
• Device A, B and C start the application with bluetooth on • Device A, B and C has initialized their profile • Device A, B and C navigates to chat box screen			
Step	Action	Expected Result	Status
1	A, B and C navigated to global chat	Global chat messaging screen for 3 devices displayed successfully	Passed
2	A sent a message	B and C received the message	Passed
3	B sent a message	A and C received the message	Passed
4	C sent a message	A and B received the message	Passed

8.3.4 Use Case 4: Bootstrap + 1:1 Chat

This use case illustrates the process of connecting and starting a chat session with a nearby peer in the mesh network. The bootstrapping process begins when the user navigates to the device scanning screen. After the scanning process is completed, a list of nearby peers will be displayed for the user to connect and chat with.

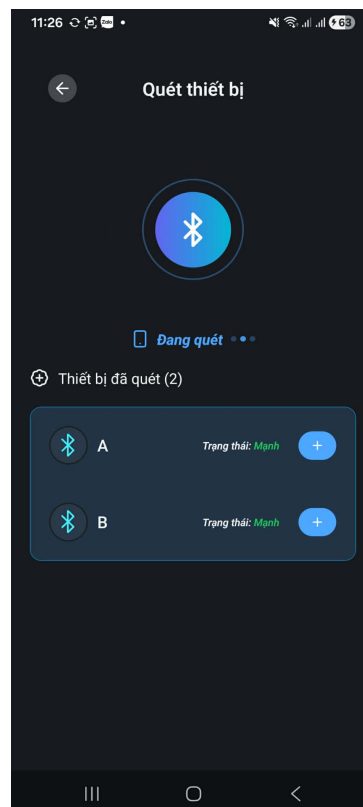
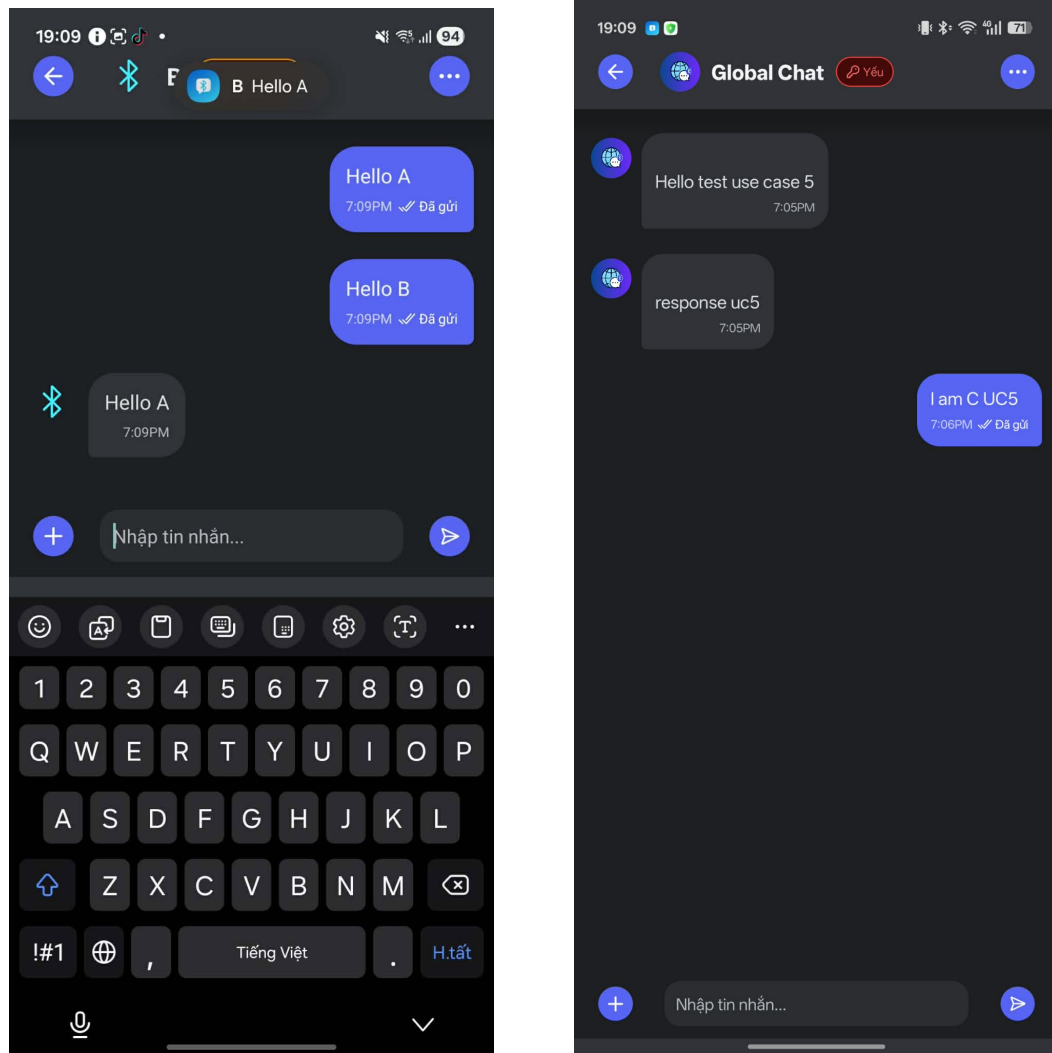


Figure 61: *Bootstrapping in Device C*

In Figure 61, when device B is selected for a one-to-one chat session, a private chat room will be created. Only devices B and C are able to exchange messages within this chat room, while other chat rooms cannot receive or access these messages. In the Figure 62, at a particular time of 19:09, when device C sends a message to device B, only device B receives the message. On the other hand, the global chat does not receive any message from device C at that time.



(a) 1:1 Chat with Device B at 19:09

(b) Global Chat at 19:09

Figure 62: Username Validation

The testing results for this use case are presented in the table below.

Table 20: *Bootstrapping and 1:1 Chat Testing Between Devices B and C*

Preconditions:			
• Device B and C start the application with Bluetooth enabled • Device B and C have initialized their profiles • Device B and C navigate to the chat box screen			
Step	Action	Expected Result	Status
1	C navigated to the device scanning screen	Device scanning screen displayed successfully	Passed
2	C selected B for connection	B received a messaging request from C	Passed
3	B accepted the request	A new chat user appeared in both B and C chat lists	Passed
4	C navigated to the newly created chat room	Messaging screen displayed successfully	Passed
5	C sent a message to B	B received the message successfully	Passed
6	C navigated to the global chat	Global chat displayed successfully and did not display C's message to B	Passed

8.3.5 Use Case 5: QR Scanning

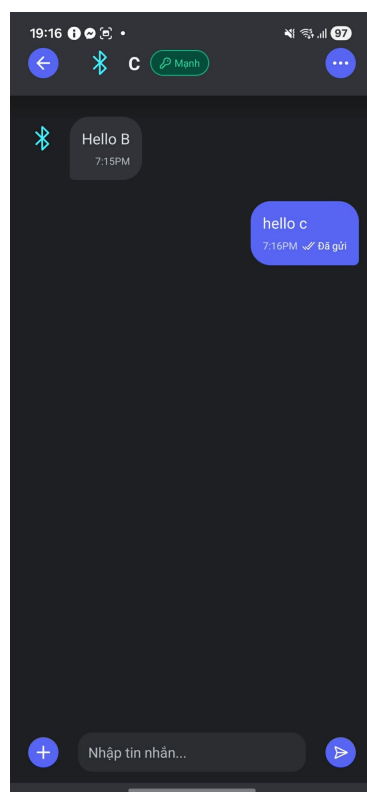
This use case testing checks how the application handles the process when a user wants to connect to another user outside the mesh network through a QR code. The testing process consists of one main flow, one alternative flow, and one exception flow

Main Flow

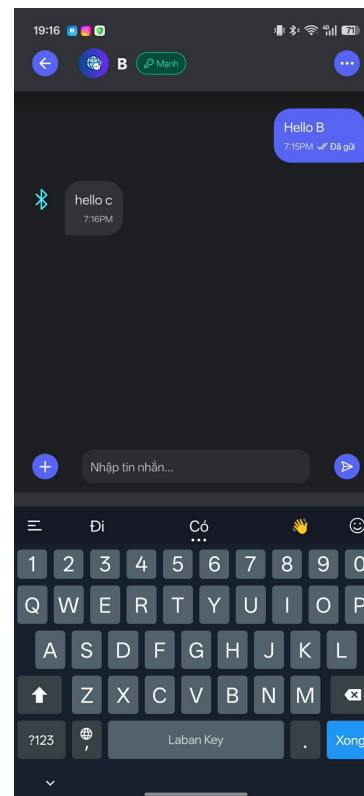
The main flow follows the standard process of scanning a QR code and creating a new chat room after the scan: the user opens the QR scanning screen, scans the QR code using the camera, and accepts the user after the QR code is successfully scanned. The table below displays the test cases for the main flow for device B and C connection.

Table 21: *Direct QR Scanning Main Flow Testing*

Preconditions:			
• Device B and C start the application with Bluetooth enabled • Device B opens QR Scanning screen • Device C gets QR code from My QR screen			
Step	Action	Expected Result	Status
1	B scanned the QR code from C	Navigate to scanned user detail screen	Passed
2	B accepted to chat with C	Navigate to messaging screen with C	Passed
3	C sent a message to B	B received the message	Passed
4	B sent a message to C	C received the message	Passed



(a) *Device B messaging screen after QR scanning*



(b) *Device C messaging screen after QR scanning*

Figure 63: *Messaging screen after QR scanning*

The Figure 63 illustrates step 3 and 4 of Table 24. Connections established through QR scanning provide a higher level of security compared to device scanning and global chat.

Alternative Flow: Image Picker QR Scanning

This flow is similar to direct QR scanning, but differs in that the user selects a QR code image from the device album and allows the application to scan it.

Table 22: *Image Picker QR Scanning Alternative Flow Testing*

Preconditions: • Device B and C start the application with Bluetooth enabled • Device B opens QR Scanning screen • Device B has device C's QR code image			
Step	Action	Expected Result	Status
1	B clicked album in QR scanning screen	Image picker displayed successfully	Passed
2	B select C's QR code image from image picker	Navigate to messaging screen with C	Passed
3	B accepted to chat with C	Navigate to messaging screen with C	Passed
4	C sent a message to B	B received the message	Passed
5	B sent a message to C	C received the message	Passed

Exception Flow: Scanning One's Own QR Code

This flow describes how the system reacts when a user scans their own QR code data using both scanning methods (direct scanning and image picker).

Table 23: *QR Scanning Exception Flow Testing*

Preconditions: • The user starts the application with Bluetooth enabled • The user opens QR Scanning screen • The user has their own profile's QR code image			
Step	Action	Expected Result	Status
1	Scan their own QR code by camera	Error message displayed	Passed
2	Click album to open the image picker	Image picker displayed successfully	Passed
3	Choose their own QR code	Error message displayed	Passed

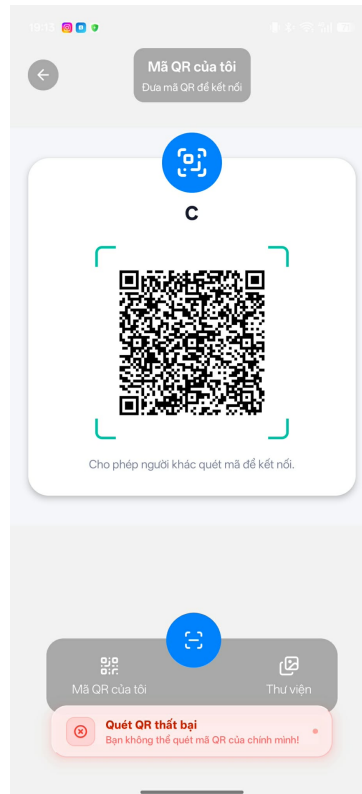


Figure 64: *Error message in QR scanning*

The Figure 64 displays the example error message example of step 1 and step 3.

8.3.6 Use Case 6: Change Profile

Changing user profiles is one of the essential features in most mobile applications that broadcast user information to others. When users want to update their profile, they need to click More and tap the profile bar, as shown in the Figure 65.

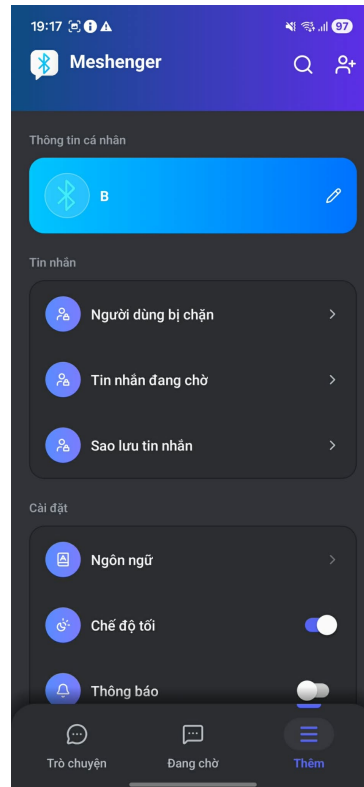


Figure 65: *Meshenger More Screen*

When clicking the profile bar, a popup will appear requiring the user to enter a username and select an avatar image. After the process is completed, the user information will be updated and broadcast to other peers. If a connection exists, the updated avatar and username will also be displayed from the perspectives of the connected peers.

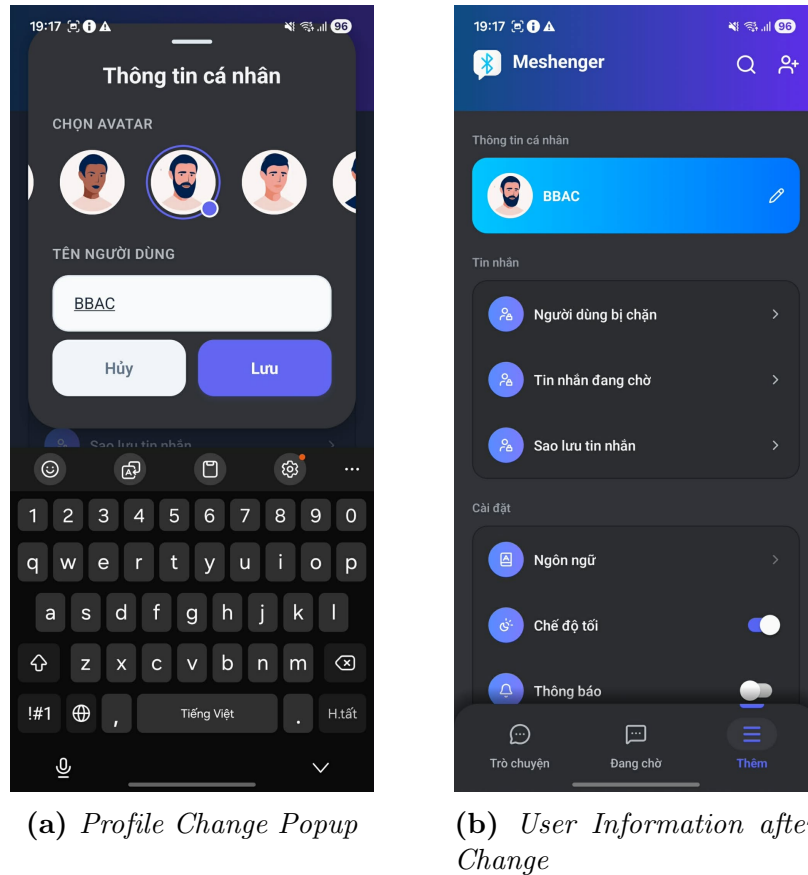


Figure 66: User Profile Information Changing Process

The table below demonstrates the main flow use case testing for the Change Profile feature.

Table 24: Change Profile Main Flow Testing

Preconditions:			
- The user starts the application with Bluetooth enabled - The user navigates to More			
Step	Action	Expected Result	Status
1	Click the profile bar	Change Profile Popup displayed successfully	Passed
2	Select an avatar different from the previously chosen one.	The selected avatar displays a check mark successfully.	Passed
3	Input a new username	New username input successfully	Passed
4	Click Save	Popup disappears and profile bar displays a new user information	Passed

8.3.7 Use Case 7: Test Input Payload

This test case evaluates how the system handles input payloads that exceed the maximum payload size of 396 bytes (described in 7.3.5). Emojis were used because each emoji consumes at least 4 bytes in UTF-8 encoding. In this test case, more than 99 emojis were entered to verify whether the message could still be transmitted properly to another peer.

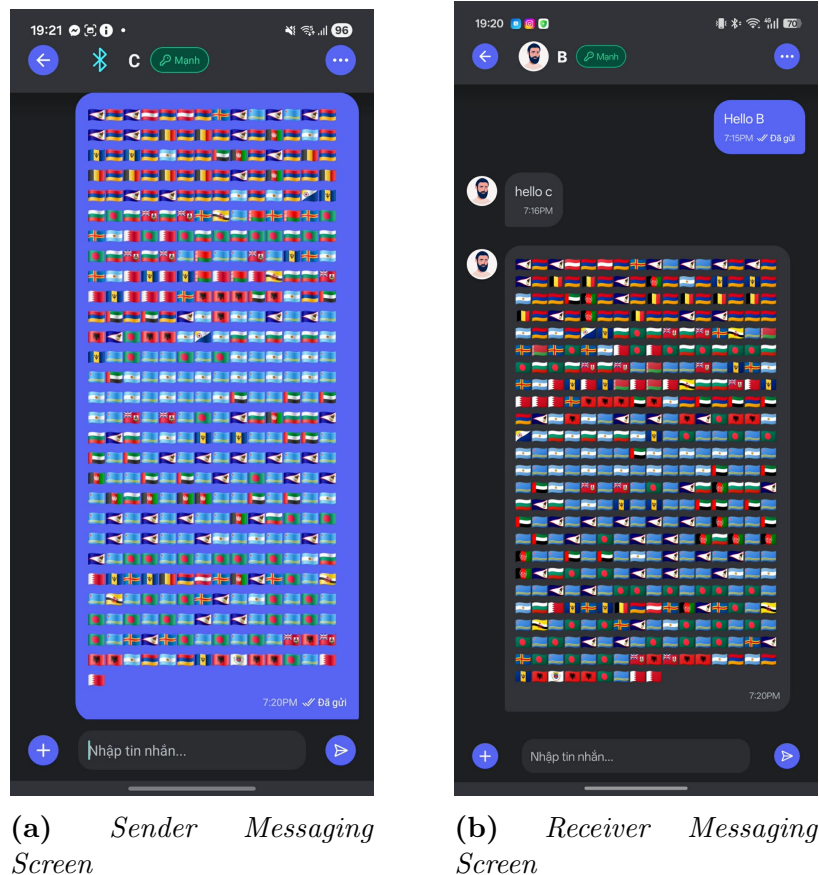


Figure 67: *Maximum Payload Test*

In the Figure 67, when the sender transmitted a message containing more than 99 emojis, the message was still processed and delivered successfully to the other peer.

8.3.8 Use Case 8: Anti-entropy

This test case describes how the system synchronizes message data across the mesh network when disconnected peers reconnect after being offline for a period of time. The test is conducted using three devices: device A, B, and C. In this scenario, device A sends a message to the global chat while device B remains online and successfully receives the message (Figure 68). Meanwhile, device C is temporarily offline and cannot receive the message at that moment. After reconnecting to the network, device C is expected to synchronize and receive the missed messages from the global chat.

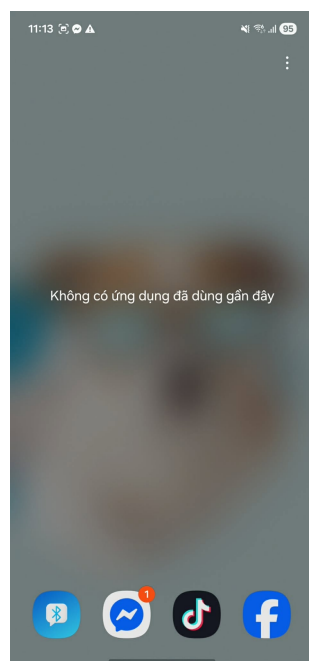


(a) *Device A Global Chat*



(b) *Device B Global Chat*

Figure 68: *Device A and B Anti-entropy Test*



(a) *Device C Offline Screen*



(b) *Device C Global Chat*

Figure 69: *Device A and B Anti-entropy Test*

In the Figure 69, device A sent a message at 11:12 AM (Figure 68), while device C remained offline until 11:14 AM. After device C reopened Meshenger and navigated to the global chat, it successfully received the message previously sent by device A. This demonstrates the anti-

entropy feature, where peers in the mesh network are able to synchronize and receive cached packets or missing fragments after reconnecting to the network.

8.4 Non-Functional Testing

To guarantee system efficiency and architectural resilience, the non-functional evaluation focuses on validating precise operational attributes against quantifiable engineering benchmarks. Two critical performance requirements were verified under standard deployment conditions:

- **P1 (Peer Discovery Performance):** Peer discovery and structural topology establishment must complete within a maximum threshold of 10 seconds under typical mesh conditions.
- **P2 (Message Transport Latency):** End-to-end message delivery latency must remain below a 1-second operational ceiling for directly connected adjacent peers.

8.4.1 P1: Peer Discovery Validation (10-Second Threshold)

Preconditions: A baseline mesh architecture consisting of two active nodes is deployed in an open-air environment within standard Bluetooth Low Energy (BLE) operational range (0–100 meters).

Scenario: An independent third node, Device C, initializes the application from a cold boot state, triggers an active scan for nearby network advertisements, and attempts to negotiate concurrent linkages with the existing cluster.

Expected Behavior: Device C dynamically discovers the mesh structure and successfully concludes low-level handshakes with Device A, Device B, or both endpoints concurrently.

```
1 14:31:01.265 MeshMaintainer com.meshenger D Outbound connections (0)
2 14:31:01.265 MeshMaintainer com.meshenger D Inbound connections (0)
3 14:31:01.265 MeshMaintainer com.meshenger D Under connection limit (0). Scanning...
4 14:31:09.124 MeshMaintainer com.meshenger D Outbound connections (2)
5 14:31:09.124 MeshMaintainer com.meshenger D Inbound connections (2)
6 14:31:09.124 MeshMaintainer com.meshenger D Satisfy connection limit (2).
```

Listing 1: Android Logcat: Mesh Linkage Discovery Lifecycle

Evaluation Result: Analysis of the execution traces in Listing 1 confirms that the peer discovery engine successfully fulfills requirement P1. The network topology engine transitions from an unconnected scanning state (14:31:01.265) to a fully synchronized state containing two active connections (14:31:09.124). The total elapsed delta for discovery, handshake negotiations, and connection table registration is calculated as:

$$\Delta t = 09.124\text{ s} - 01.265\text{ s} = 7.859\text{ seconds}$$

This falls well within the strict 10-second operational threshold, proving that Device C successfully scales out the mesh cluster layout efficiently.

8.4.2 P2: Message Delivery Latency Validation (1-Second Ceiling)

Preconditions: Device B and Device C maintain active, authenticated multi-path links within the routing table and are positioned within point-to-point physical transmission range.

Scenario: Device C initiates a broadcast transaction targeting the global chat channel containing the plaintext string: "Hello, I am C".

Expected Behavior: The end-to-end processing and propagation window—spanning serialization, cryptographic packet signing, mesh wireless transmission, frame validation, and extraction—must finish within 1 second.

```
1 14:39:18.780 SessionModule com.meshenger D Attempting to emit to JS: {"Action":"Send",  
  ↪ "SessionType":"GlobalChat","Message":"Hello, I am C","PeerID":"-1"}
```

Listing 2: Android Logcat: Device C Outbound Message Transmission Trace

```
1 14:39:19.509 GlobalChatSession com.meshenger D 4920088106597921027 said: Hello, I am C
```

Listing 3: Android Logcat: Device B Inbound Message Extraction Trace

Evaluation Result: Cross-referencing the synchronized system timestamps between the sending node in Listing 2 and the receiving endpoint in Listing 3 demonstrates compliance with requirement P2. The outbound payload event is dispatched by the session module at 14:39:18.780, and the corresponding packet is fully reassembled, verified, and parsed by Device B at 14:39:19.509. The total transit latency is computed as:

$$\Delta t = 19.509\text{ s} - 18.780\text{ s} = 0.729\text{ seconds}$$

This performance (729 ms) safely satisfies the sub-second design requirement, validating that the underlying network architecture processes packets and routes message objects efficiently.

8.5 Deployment

The final deployment target of the system is an Android smartphone environment in which each device runs the complete application stack locally. Every deployed device contains the mobile user interface, local database, packet cache, encryption logic, and Bluetooth mesh communication services, enabling the system to operate without dependence on external servers or internet connectivity.

Deployment was carried out in a staged manner. During development, Expo supported rapid interface prototyping and iterative testing, while Android Studio was used to build and debug the Android-native parts of the application. For release deployment, the project was packaged into an Android build artifact for installation on physical devices, permissions for Bluetooth

and related device features were granted during setup, and the application was then initialized through onboarding and profile creation. Final validation was performed on real Android phones because Bluetooth discovery, peer-to-peer channel establishment, and mesh behavior cannot be evaluated reliably through emulator-only deployment.

For future production-oriented releases, the same deployment pipeline can be extended with signed release builds, broader device compatibility checks, and controlled distribution to pilot users. In addition, interface-only updates can benefit from faster iteration workflows, whereas changes that affect native Bluetooth behavior or Android services should continue to be distributed through rebuilt Android release packages.

The source code and executable application for this project are publicly available at the following repositories:

- **Source Code (GitHub):** <https://github.com/HuyVietFeb4/P2P-chat-app>
- **Application Binary (.apk):** [Google Drive Repository](#)

9 Conclusion and Next Phase

This project demonstrates the feasibility of an infrastructure-independent peer-to-peer messaging application operating over Bluetooth mesh networks. By combining Bluetooth Low Energy communication, local-first data management, and end-to-end security mechanisms, the system provides a practical foundation for resilient communication in environments where conventional internet or cellular infrastructure is unavailable, unreliable, or intentionally restricted. The implementation also confirms that consumer Android devices can support secure decentralized messaging without requiring specialized hardware.

The conclusion of this work is therefore not only that the proposed system is technically viable, but also that it provides a meaningful starting point for broader real-world deployment and refinement. The following subsections summarize the main deliverables achieved in the current phase and the principal limitations that define the next phase of development.

9.1 Key Deliverables Achieved

The project completed several major deliverables across system design, implementation, security, and user interaction.

- **Decentralized communication architecture:** a layered application structure was designed to support peer discovery, packet forwarding, secure session establishment, local caching, and user-facing messaging workflows over a Bluetooth mesh environment.
- **Secure messaging model:** the system integrated modern cryptographic mechanisms, including Curve25519-based key agreement, ChaCha20-Poly1305 authenticated encryption, and the Noise Protocol Framework to support secure channel establishment under different trust levels.
- **Practical mobile implementation:** a working Android-based prototype was implemented using React Native for cross-platform-oriented interface development and Kotlin for Android-native integration, especially for Bluetooth and lower-level device services.
- **Core user features:** the application delivered essential user workflows such as onboarding, profile initialization, global messaging, one-to-one communication, QR-assisted verification, profile editing, and local persistence of messaging data.
- **Mesh communication support:** the protocol and implementation included bootstrapping, peer discovery, packet formation, fragmentation, reassembly, acknowledgment handling, and the foundation for anti-entropy-based delayed delivery.

- **Testing and deployment preparation:** the system was validated through unit, integration, and use-case-based testing, and it was prepared for deployment on physical Android devices for real Bluetooth communication scenarios.

Taken together, these deliverables show that the project successfully moved from conceptual design to an implemented prototype capable of supporting secure offline communication in realistic constrained environments.

9.2 Limitations and Constraints

Despite the progress achieved, the current system still operates under several important limitations. Bluetooth Low Energy provides low power consumption and broad device availability, but it also imposes constraints in communication range, bandwidth, interference tolerance, and connection stability. These factors limit message throughput and make large-scale or highly mobile mesh scenarios more difficult to support reliably.

In addition, the present implementation remains a prototype-oriented system rather than a production-ready platform. Some advanced capabilities are still incomplete or reserved for the next milestone, especially full anti-entropy validation, broader large-network testing, deeper resistance to adversarial behavior such as Sybil-style attacks or intentional jamming, and more extensive evaluation under diverse real-world deployment conditions. Features such as cross-platform support, large file-transfer optimization, and richer communication modes also remain outside the current implementation scope.

These constraints directly inform the next phase of the project. Future work should prioritize completing and validating anti-entropy synchronization, expanding field testing on multiple physical devices, improving routing efficiency and battery usage, strengthening trust and identity verification workflows, and preparing the application for broader pilot deployment through more mature release engineering and compatibility testing. In this sense, the current phase establishes a strong technical foundation, while the next phase is focused on robustness, scalability, and operational readiness.

9.3 Future Work

To transition the current mesh architecture from a functional prototype to a highly resilient, enterprise-grade decentralized communication system, the next development lifecycle phases will prioritize the following objectives:

- **Intensive Vulnerability and Stress Verification:** Expand the existing test suites by introducing automated fuzzing frameworks and high-density network emulation. This phase focuses on subjecting the routing engine and state layers to extreme boundary conditions to expose and eliminate hidden synchronization deadlocks, race conditions, and edge-case anomalies.

- **Architectural Mitigation and Refinement:** Systematically address and patch the architectural limitations identified during evaluation cycles, optimizing resource utilization and system stability across all target devices.
- **Anti-Entropy Synchronization Optimization:** Enhance the algorithmic efficiency of the anti-entropy framework. Future implementations will investigate adaptive Bloom filter sizing strategies, compressed bitmask transmissions, and delta-encoded synchronization sequences to minimize bandwidth consumption during high-frequency node handshakes.
- **ODMRP Routing Protocol Integration:** Implement the On-Demand Multicast Routing Protocol (ODMRP) to replace or augment the baseline flooding mechanism. Transitioning to an on-demand, mesh-based routing strategy will significantly reduce broadcast storms, mitigate packet collision rates, and dynamically discover optimal routing pathways within volatile Mobile Ad-Hoc Network (MANET) environments.
- **Group Chat Cryptographic Security Protocol:** Architect and integrate an advanced group chat security sub-system. This involves implementing end-to-end encryption (E2EE) tailored for dynamic mesh clusters, employing decentralized group key management schemes (such as Asynchronous Ratcheting Trees), and guaranteeing forward secrecy and post-compromise security without relying on a central authority.
- **Advanced Transport Layers and Feature Sets:** Scale the platform's functional capabilities by implementing:
 - *Transport-Layer Primitives:* A lightweight, custom TCP-like protocol designed for mesh topologies to guarantee ordered, reliable packet delivery and congestion control.
 - *Multimedia Payload Support:* Extended serialization codecs capable of segmenting, streaming, and reassembling complex, non-textual payloads including files, audio notes, and video data.
 - *Social Layer State Synchronization:* An optimized state-tracking protocol to handle real-time delivery confirmations, "read" receipts, and message reaction states across disconnected network clusters.

A Noise Protocol Framework

The application adopts the Noise Protocol Framework as the foundation for secure channel establishment. Noise is a flexible cryptographic framework based on Diffie–Hellman key agreement, enabling two peers to establish a shared secret over an insecure channel. It defines structured handshake patterns that provide confidentiality, integrity, and authentication. The session keys derived by the handshake are then used to encrypt and authenticate all subsequent communication [15].

A Noise handshake pattern consists of a pre-message pattern for the initiator, a pre-message pattern for the responder, and a sequence of message patterns exchanged during the handshake. In Noise notation, the symbols **e** and **s** denote ephemeral and static public keys, respectively. The tokens **ee**, **es**, **se**, and **ss** represent Diffie–Hellman operations.

A.1 Handshake Patterns Used in the Application

XX pattern

```
-> e
<- e, ee, s, es
-> s, se
```

The XX pattern is the default mechanism for establishing a secure channel between two peers. It ensures end-to-end encryption and mutual authentication of static keys, but does not by itself guarantee the physical authenticity of the device holding the key.

KK pattern

```
-> s
<- s
...
-> e, es, ss
<- e, ee, se
```

The KK pattern is used to reestablish secure channels between peers that have previously exchanged static keys. Its security depends on whether those keys were verified out-of-band.

XK pattern

```
<- s
...
```



-> e, es

<- e, ee

-> s, se

The XK pattern is applied during QR-code-based key sharing. If used immediately after key exchange, it establishes a high-security channel. If reused after a timeout, it provides asymmetric trust guarantees while preserving encryption.

B Bloom Filter

This application utilizes a Bloom Filter for the Summary Vector of the Anti-Entropy protocol. The primary reason for selecting a Bloom Filter over standard hash functions is its ability to represent large sets of data with significantly lower network overhead. The inherent trade-off of this approach is a higher false-positive rate. However, this is acceptable within the protocol's design: since the Anti-Entropy mechanism is cyclical and will re-request missing data in subsequent rounds, any initial failure to retrieve a packet due to a false positive will be resolved in a future synchronization cycle.

B.1 Bloom Filter Overview

A Bloom Filter is a probabilistic data structure used to test whether an element is a member of a set. It is highly space-efficient and supports fast insertions and queries, making it suitable for distributed systems. [1]

Let $\mathbf{bf}[]$ be a bit vector of size m , and let there be k independent hash functions $h_i : \mathcal{U} \rightarrow \{0, 1, \dots, m-1\}$ for $1 \leq i \leq k$, where $\mathcal{U}$ is the universal set of possible elements.

Insertion

To insert an element x into the Bloom Filter:

- Compute the hash values $h_1(x), h_2(x), \dots, h_k(x)$.
- Set the corresponding bits in the bit vector to 1:

$$\mathbf{bf}[h_1(x)] = 1, \quad \mathbf{bf}[h_2(x)] = 1, \quad \dots, \quad \mathbf{bf}[h_k(x)] = 1$$

Querying

To check whether an element x is in the set:

- Compute the hash values $h_1(x), h_2(x), \dots, h_k(x)$.
- If all corresponding bits are set to 1:

$$\mathbf{bf}[h_1(x)] = 1, \quad \mathbf{bf}[h_2(x)] = 1, \quad \dots, \quad \mathbf{bf}[h_k(x)] = 1$$

then return **true**; otherwise, return **false**.

Note that Standard Bloom Filters may yield false positives but never false negatives. Figure 70 illustrated the examples of 2 operations insert and query item.

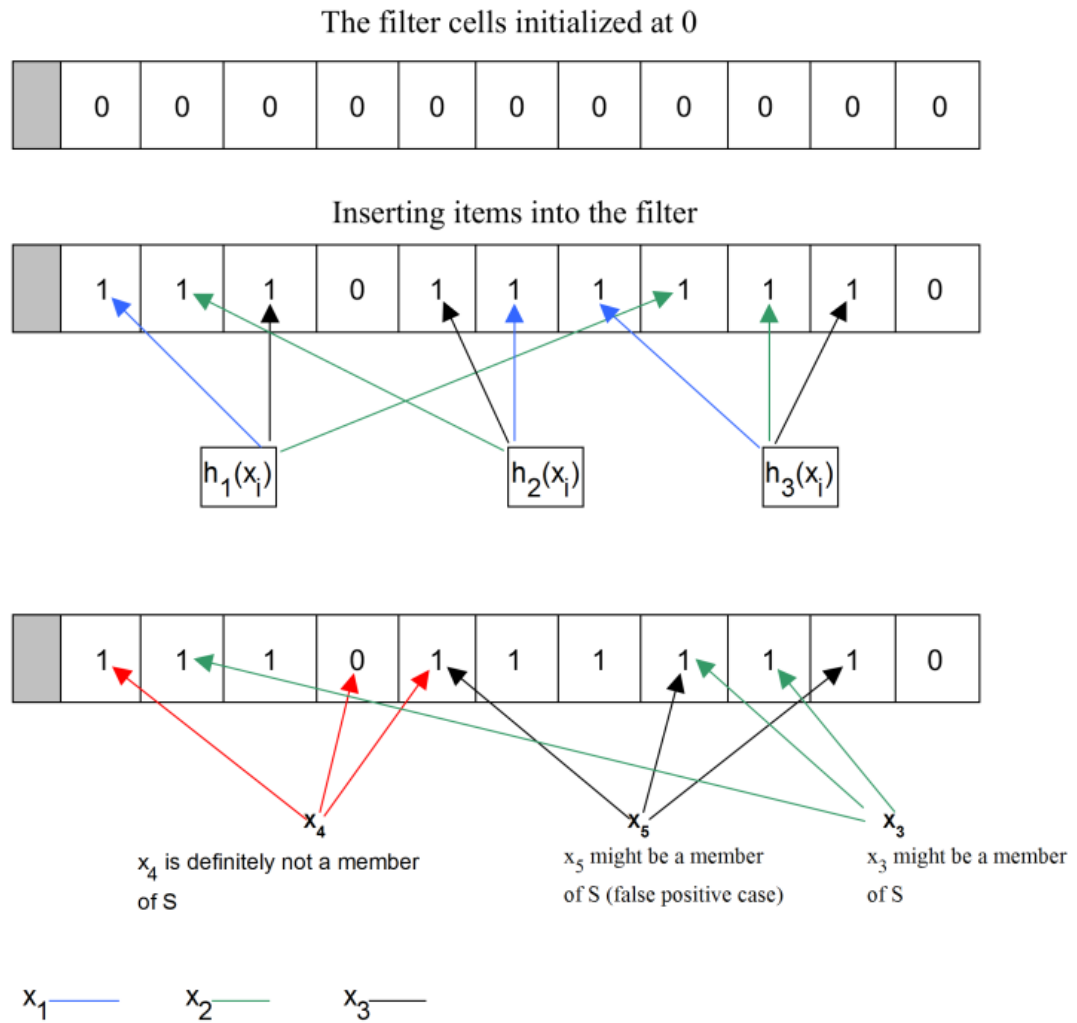


Figure 70: Bloom Filter Insert and Queries Operations

B.2 Types of Bloom Filters

To optimize the efficiency of the summary vector, we evaluated several probabilistic data structures. The specific Bloom Filter variants considered for implementation in this project include:

B.2.1 Standard Bloom Filter

The standard Bloom filter is the original and most widely used variant. It employs a fixed-size bit array and multiple independent hash functions to map elements into the array. Insertions set the corresponding bits to 1, and queries check whether all relevant bits are set. While simple and efficient, this design can produce false positives when different elements map to the same positions, though it never yields false negatives. [3]

B.2.2 Kirsch-Mitzenmacher Bloom Filter

The Kirsch-Mitzenmacher Bloom filter improves efficiency by reducing the number of hash computations required. Instead of computing k independent hash functions, it derives multiple

hash values from two base functions. This optimization lowers computational overhead while maintaining the same false positive probability as the standard Bloom filter, making it attractive for high-performance distributed systems. [12]

B.2.3 Compact Bloom Filter

Compact Bloom filters are designed to minimize memory usage in network, especially in environments with limited resources. They achieve this by compressing the bit array or by using more space-efficient representations. Although compact filters may slightly increase query time or false positive rates, they are useful when network constraints outweigh the need for maximum accuracy.

An enhancement to the Compact Bloom Filter (CmBF) was proposed to improve the utilization of index values in the mapping process. The refinement introduces a new rule that ensures all bit pattern values are effectively used when assigning indices. Specifically, if the bit pattern value $S_{ival} < 2^m - 1$, then the index in the CmBF is directly assigned the bit pattern's value. This adjustment reduces redundancy in index mapping, improves distribution across the filter, and ultimately enhances query accuracy while maintaining the compact memory footprint of the original design. [14]

B.2.4 Yes No Bloom Filter

The Yes-No Bloom filter enhances the standard Bloom filter by introducing a secondary structure, called the No filter, which specifically addresses false positives. When a query passes the initial Bloom filter check (the “Yes” stage), the element is then verified against the No filter, which stores items known to cause false positives. If the element is found in the No filter, the query is rejected, thereby eliminating the error. The No filter is updated whenever a false positive is detected: if an element passes the Yes filter but is not actually present in the dataset, it is inserted into the No filter so that future queries for the same element will be correctly identified as absent. This two-stage process significantly reduces false positives while preserving the efficiency of Bloom filter operations. [7]

B.2.5 Project decision

Drawing from the analysis from multiple sources [3, 7, 12, 14], a hybrid architectural design, integrating the KM Bloom Filter with the Compact Refined Bloom Filter, has been identified as the most effective solution for peer-to-peer environments. This hybrid approach is particularly well-suited for mesh networks where minimizing network bandwidth consumption is a critical constraint. Consequently, this specific filter implementation will be utilized for the Anti-Entropy Summary Vector, ensuring high-efficiency synchronization with minimal data overhead.

References

- [1] Abdennebi, A., & Kaya, K. (2021). A bloom filter survey: Variants for different domain applications. *arXiv preprint arXiv:2106.12189*. <https://doi.org/10.48550/arxiv.2106.12189>
- [2] Android Developers. (2025). *Bluetooth low energy overview* [Official Android Developer Documentation]. Retrieved December 21, 2025, from <https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview>
- [3] Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7), 422–426. <https://doi.org/10.1145/362686.362692>
- [4] Bluetooth SIG Regulatory Expert Group. (2023, March). *Bluetooth low energy – regulatory aspects (rad)* (Informational Publication) (Prepared by the Regulatory Expert Group). Bluetooth SIG. Retrieved December 21, 2025, from <https://www.bluetooth.com/>
- [5] Briar Project. (2025). *Briar project*. Retrieved December 21, 2025, from <https://briarproject.org/>
- [6] BrowserStack. (2025). Data-driven testing: What it is, how it works, and tools to use [Accessed: 2026-05-18].
- [7] Carrea, L., Vernitski, A., & Reed, M. (2016). Yes-no bloom filter: A way of representing sets with fewer false positives. *arXiv preprint arXiv:1603.01060*. <https://doi.org/10.48550/arxiv.1603.01060>
- [8] Computerworld. (2024, July 18). *What's a peer-to-peer (p2p) network?* Retrieved December 21, 2025, from <https://www.computerworld.com/article/1355655/whats-a-peer-to-peer-p2p-network.html>
- [9] Dorsey, J. (2025). *Bitchat*. Retrieved December 21, 2025, from <https://github.com/permissionlesstech/bitchat>
- [10] Gnutella Developer Forum. (2003). *The annotated gnutella protocol specification v0.4* [Document Revision 1.6, Clip2]. Retrieved December 21, 2025, from <https://rfc-gnutella.sourceforge.net/developer/stable/>
- [11] IBM. (2025, November 17). *End-to-end encryption*. Retrieved December 21, 2025, from <https://www.ibm.com/think/topics/end-to-end-encryption>
- [12] Kirsch, A., & Mitzenmacher, M. (2008). Less hashing, same performance: Building a better bloom filter. *Random Structures and Algorithms*, 33(2), 187–218. <https://doi.org/10.1002/rsa.20208>

- [13] Krawczyk, H., & Eronen, P. (2010, May). *Hmac-based extract-and-expand key derivation function (hkdf)* (RFC No. 5869) (Category: Informational). Internet Engineering Task Force. Retrieved December 21, 2025, from <https://datatracker.ietf.org/doc/html/rfc5869>
- [14] Mosharraf, N., Jayasumana, A. P., & Ray, I. (2016). Compacted bloom filter. *2016 IEEE 2nd International Conference on Collaboration and Internet Computing (CIC)*, 304–311. <https://doi.org/10.1109/cic.2016.048>
- [15] Perrin, T. (2018). *The noise protocol framework* (Technical Report No. Revision 34) (Official/unstable specification). Noise Project. Retrieved December 21, 2025, from <https://noiseprotocol.org/noise.html>
- [16] Testlio. (2025). The top black box testing techniques [Accessed: 2026-05-18].
- [17] Vahdat, A., & Becker, D. (2000, April). *Epidemic routing for partially connected ad hoc networks* (Technical Report No. CS-200006). Duke University.